



universidad
de león



Escuela de Ingenierías Industrial, Informática y Aeroespacial

GRADO EN INGENIERÍA INFORMÁTICA

Trabajo de Fin de Grado

Aplicación Web para Comercio Electrónico con IA
Generativa y Recuperación de Información

E-commerce Web App with Generative AI and
Information Retrieval

Autor: Carlos Honrado Cristobalena

Tutora: M^a Teresa García Ordás

Cotutor: Martín Bayón Gutiérrez

(Julio, 2025)

UNIVERSIDAD DE LEÓN

Escuela de Ingenierías Industrial, Informática y Aeroespacial

GRADO EN INGENIERÍA INFORMÁTICA

Trabajo de Fin de Grado

ALUMNO: Carlos Honrado Cristobalena

TUTOR: M^a Teresa García Ordás

COTUTOR: Martín Bayón Gutiérrez

TÍTULO: Aplicación Web para Comercio Electrónico con IA Generativa y Recuperación de Información.

TITLE: E-commerce Web App with Generative AI and Information Retrieval

CONVOCATORIA: Julio, 2025

RESUMEN:

Esta aplicación web integra un chatbot basado en inteligencia artificial generativa que transforma la experiencia de búsqueda, análisis y gestión en plataformas de comercio electrónico. Su diseño responde a tres objetivos principales: agilizar la exploración de productos para los consumidores, proporcionar un motor de búsqueda semántico avanzado y facilitar herramientas analíticas a los administradores.

El núcleo de esta solución reside en la conversión tanto de las características de los productos como de las consultas de los usuarios a un mismo espacio semántico mediante representaciones vectoriales. Esto permite realizar búsquedas más precisas, priorizando la similitud conceptual sobre las coincidencias exactas de palabras. Así, cuando un usuario busca algo como “quiero un portátil ligero con buena batería”, el sistema evalúa la proximidad entre su intención y la descripción de los productos, devolviendo los resultados más relevantes.

Además, este mismo enfoque vectorial se aplica a las reseñas de usuarios, lo que habilita el uso de técnicas como RAG (*Retrieval-Augmented Generation*). Al recibir una consulta,

Carlos Honrado Cristobalena

el chatbot puede recuperar opiniones cercanas semánticamente y utilizarlas para generar respuestas fundamentadas, ofreciendo resúmenes o comparativas basadas en experiencias reales.

Para gestionar estas capacidades, el sistema utiliza *Function Calling*, una técnica que permite al modelo de lenguaje invocar funciones específicas del backend cuando detecta que la consulta lo requiere. Por ejemplo, puede activar funciones para generar reportes personalizados sobre comportamiento de usuarios, tendencias de búsqueda o rendimiento de productos, sin intervención manual. También es capaz de construir gráficos interactivos, facilitando así la toma de decisiones basada en datos.

En resumen, esta aplicación combina procesamiento del lenguaje natural, búsqueda semántica avanzada y generación contextualizada de respuestas mediante RAG y Function Calling. Esta arquitectura permite una experiencia conversacional fluida, adaptada tanto a consumidores como a administradores, superando las limitaciones de los motores de búsqueda tradicionales. El chatbot no solo asiste, sino que actúa como un asistente estratégico central, proporcionando una experiencia personalizada, inteligente y orientada a la acción dentro del entorno del comercio electrónico.

ABSTRACT:

This web application integrates a generative AI-based chatbot that transforms the way users search, analyze, and manage content on e-commerce platforms. It is designed with three main objectives: to streamline product exploration for consumers, provide an advanced semantic search engine, and offer analytical tools for administrators.

At the core of this solution lies the transformation of both product features and user queries into a shared semantic space through vector representations. This enables more accurate searches by prioritizing conceptual similarity over exact word matches. For instance, when a user searches for something like “I want a lightweight laptop with good battery life,” the system evaluates the proximity between the user’s intent and product descriptions, returning the most relevant results.

This same vector-based approach is also applied to user reviews, enabling the use of techniques such as RAG (Retrieval-Augmented Generation). Upon receiving a query, the chatbot can retrieve semantically similar opinions and integrate them into its responses, offering well-grounded summaries or comparisons based on real user feedback.

To manage these capabilities, the system uses Function Calling—a technique that allows the language model to invoke specific backend functions when it detects a relevant request. For example, it can trigger functions to generate personalized reports on user behavior, search trends, or product performance, all without manual intervention. It can also generate interactive charts, thereby facilitating data-driven decision-making.

In summary, this application combines natural language processing, advanced semantic search, and contextual response generation through RAG and Function Calling. This architecture enables a seamless conversational experience tailored to both consumers and

administrators, surpassing the limitations of traditional search engines. The chatbot not only assists but acts as a strategic core assistant, delivering a personalized, intelligent, and action-oriented experience within the e-commerce environment.

Palabras clave: Aplicación web, inteligencia artificial generativa, chatbot, búsqueda semántica, embeddings, RAG, Function Calling, comercio electrónico.

Firma del alumno:

VºBº Tutor/es:

Índice de contenidos

1 INTRODUCCIÓN.....	1
1.1 OBJETIVOS.....	3
1.2 METODOLOGÍA EMPLEADA.....	4
1.3 TECNOLOGÍA EMPLEADA.....	6
1.3.1 EVALUACIÓN DE ALTERNATIVAS.....	6
1.3.2 ALTERNATIVAS ELEGIDAS.....	9
1.4 ESTRUCTURA DEL TRABAJO.....	12
2 ESTUDIO DEL PROBLEMA.....	14
2.1 EL CONTEXTO DEL PROBLEMA.....	14
2.2 EL ESTADO DEL ARTE.....	14
2.2.1 ALTERNATIVAS SIMILARES.....	15
2.2.1.1 AMAZON A9.....	15
2.2.1.2 EBAY CASSINI.....	17
2.2.1.3 WALMART POLARIS.....	19
2.3 LA DEFINICIÓN DEL PROBLEMA.....	20
2.4 MARCO TEÓRICO.....	21
2.4.1 MODELOS DE LENGUAJE DE GRAN ESCALA (LLM).....	21
2.4.2 FUNCTION CALLING EN LLMS.....	22
2.4.3 EMBEDDINGS SEMÁNTICOS.....	24
2.4.4 BASES DE DATOS VECTORIALES.....	25
3 GESTIÓN DEL PROYECTO SOFTWARE.....	27
3.1 ALCANCE DEL PROYECTO.....	27
3.1.1 DEFINICIÓN DEL PROYECTO.....	27
3.1.2 OBJETIVOS.....	29
3.2 PLAN DE TRABAJO.....	29
3.2.1 IDENTIFICACIÓN DE TAREAS.....	30
3.2.1.1 INICIO DEL PROYECTO.....	30
3.2.1.2 PROCESO ITERATIVO (E INCREMENTAL).....	30
3.2.1.3 VALIDACIÓN Y ENTREGA.....	32
3.2.2 ESTIMACIÓN DE TAREAS.....	33
3.2.3 PLANIFICACIÓN DE TAREAS.....	33
3.3 GESTIÓN DE RECURSOS.....	40
3.3.1 ESPECIFICACIÓN DE RECURSOS.....	40
3.3.2 ASIGNACIÓN DE RECURSOS.....	45
3.4 PRESUPUESTO.....	46
3.5 LEGISLACIÓN Y NORMATIVA.....	47
4 SOLUCIÓN.....	50
4.1 DESCRIPCIÓN DE LA SOLUCIÓN.....	50
4.2 ANÁLISIS DE REQUISITOS.....	55
4.2.1 DEFINICIÓN DE REQUISITOS.....	55
4.2.2 ESPECIFICACIÓN DE REQUISITOS.....	58

4.3 DISEÑO.....	81
4.3.1 DISEÑO DE LA ARQUITECTURA.....	82
4.3.2 DISEÑO DEL FRONTEND.....	84
4.3.3 DISEÑO DEL BACKEND.....	88
4.3.4 CONTENERIZACIÓN Y PORTABILIDAD DE LA SOLUCIÓN.....	90
4.3.5 DISEÑO DEL SISTEMA KAFKA.....	91
4.3.6 DISEÑO DE LA BASE DE DATOS.....	92
4.4 IMPLEMENTACIÓN.....	94
4.4.1 IMPLEMENTACIÓN DEL FRONTEND.....	95
4.4.2 IMPLEMENTACIÓN DEL BACKEND.....	123
4.4.3 IMPLEMENTACIÓN DEL MÓDULO DEL CHATBOT.....	147
4.4.4 IMPLEMENTACIÓN DEL SISTEMA KAFKA.....	161
4.5 PRUEBAS.....	166
4.6 EL PRODUCTO DEL DESARROLLO.....	172
4.6.1 SISTEMA DE DATOS.....	172
4.6.2 LA APLICACIÓN WEB.....	178
5 CONCLUSIONES.....	196
5.1 APORTACIONES REALIZADAS.....	197
5.2 TRABAJOS FUTUROS.....	198
5.3 PROBLEMAS ENCONTRADOS.....	200
5.4 OPINIÓN PERSONAL.....	201
REFERENCIAS.....	203

Índice de figuras

Figura 1.1 Predicción del tamaño de mercado de la IA Generativa en millones de USD [51].....	1
Figura 1.2 Fases de la metodología iterativa.....	5
Figura 1.3 Estructura general de la solución.....	11
Figura 2.1 Diagrama de interacción con Function Calling.....	23
Figura 2.2 Conversión de texto a Embedding.....	24
Figura 2.3 Comparación de similitud entre embeddings de frases mediante ángulos vectoriales [31].....	26
Figura 3.1 Diagrama de Gantt Plan inicial.....	34
Figura 3.2 Diagrama de Gantt Plan iteración 1.....	36
Figura 3.3 Diagrama de Gantt Plan iteración 2.....	36
Figura 3.4 Diagrama de Gantt Plan iteración 3.....	36
Figura 3.5 Diagrama de Gantt Plan iteración 4.....	37
Figura 3.6 Diagrama de Gantt Plan iteración B1.....	37
Figura 3.7 Diagrama de Gantt Plan iteración B2.....	37
Figura 3.8 Diagrama de Gantt Plan iteración B3.....	38
Figura 3.9 Diagrama de Gantt Plan iteración B4.....	38
Figura 3.10 Diagrama de Gantt Plan iteración B5.....	38
Figura 3.11 Diagrama de Gantt Plan iteración B6.....	39
Figura 3.12 Diagrama de Gantt Plan iteración B7.....	39
Figura 3.13 Diagrama de Gantt Plan iteración B8.....	39
Figura 3.14 Diagrama de Gantt Plan Validación y entrega final.....	40
Figura 4.1 Subsistema de ingesta y procesamiento de datos.....	52
Figura 4.2 Vistas y acciones del usuario.....	53
Figura 4.3 Vistas y acciones del Administrador.....	54
Figura 4.4 Sub-proyectos de la aplicación.....	82
Figura 4.5 Ejemplo de interacción cliente-servidor.....	83
Figura 4.6 Estructura de carpetas del frontend.....	85
Figura 4.7 Prototipo de la paleta de colores de la aplicación.....	86
Figura 4.8 Prototipo del icono de la Aplicación.....	86
Figura 4.8 Boceto Página de Productos.....	87
Figura 4.9 Boceto Página Ver Producto.....	87
Figura 4.10 Estructura de carpetas del backend.....	89
Figura 4.11 Estructura de la parte Docker.....	90
Figura 4.12 Servicios de Kafka desplegados.....	91
Figura 4.13 Esquema de la base de datos.....	94
Figura 4.14 Fragmento de código de inicialización en main.tsx.....	96
Figura 4.15 Fragmento de código de inicialización en main.tsx.....	97
Figura 4.16 Función para RootRedirect.....	98
Figura 4.17 Declaración de las rutas.....	99

Figura 4.18 Lógica del componente ProtectedRoute.....	100
Figura 4.19 Lógica del componente AdminRoute.....	100
Figura 4.20 Funciones e inicialización del archivo Authcontext.....	102
Figura 4.21 Lógica de autenticación tradicional.....	103
Figura 4.22 Proceso de autenticación con proveedores externos.....	104
Figura 4.23 Archivo apiClient.tsx.....	105
Figura 4.24 Búsqueda manual con medición del tiempo de respuesta.....	107
Figura 4.25 Mecanismo de autocompletado al escribir en el campo de búsqueda.....	107
Figura 4.26 Funcionamiento de la pestaña principal de Productos.....	109
Figura 4.27 Funciones del servicio de productos.....	110
Figura 4.28 Carga inicial de detalles y reseñas.....	111
Figura 4.29 Componente de reseñas con scroll.....	112
Figura 4.30 Inicialización del mapa de calor.....	115
Figura 4.31 Renderizado de gráficos personalizados a partir de consultas del chatbot.....	117
Figura 4.32 Construcción del informe de actividad de usuarios en PDF.....	118
Figura 4.33 Llamada al backend para usar el servicio del chat.....	120
Figura 4.34 Procesamiento de los chunks recibidos desde el backend.....	120
Figura 4.35 Ejemplo de un chunk de additional data.....	121
Figura 4.36 Ejemplo de un chunk de status.....	121
Figura 4.37 Impresión de los chunks recibidos desde el backend.....	122
Figura 4.38 Asociación de namespaces a las rutas.....	124
Figura 4.39 Documentación de la API de la Aplicación.....	125
Figura 4.40 Endpoints de autenticación disponibles (Swagger).....	126
Figura 4.41 Registro seguro de usuario con hash de contraseña.....	127
Figura 4.42 Endpoints de usuario disponibles en Swagger.....	129
Figura 4.43 Método create_user en UserService.....	132
Figura 4.44 Filtrado de usuarios por parámetros.....	133
Figura 4.45 Añadir un producto a favoritos.....	133
Figura 4.46 Guardado de búsquedas del usuario.....	135
Figura 4.47 Endpoints de productos.....	138
Figura 4.48 Creación de producto y generación de embeddings para los detalles.....	139
Figura 4.49 Cálculo de la similitud textual entre la consulta y los campos title y description. 140	
Figura 4.50 Distancia de los embeddings de detalles y características respecto de la consulta.....	141
Figura 4.51 Formato devuelto por el motor de búsqueda.....	142
Figura 4.52 Lógica detrás de /products/most-favorited.....	143
Figura 4.53 Lógica detrás de /api/products/autocomplete.....	144
Figura 4.54 Endpoints de reseñas.....	145
Figura 4.55 Generación del embedding para la reseña.....	146
Figura 4.56 Consulta que devuelve la distancia coseno entre la pregunta y una reseña.....	147
Figura 4.57 Endpoints del chatbot.....	148
Figura 4.58 Instrucciones iniciales del chatbot.....	149

Figura 4.59 Implementación del streaming de respuestas en el método generate_stream...	151
Figura 4.60 Ejemplo de prompt para administradores.....	153
Figura 4.61 Ejemplo de tool call para search_product.....	155
Figura 4.62 Ejemplo de solicitud de los datos del heatmap.....	157
Figura 4.63 Implementación del metodo get_heatmap_data.....	160
Figura 4.64 Salida de generate_dynamic_chart.....	162
Figura 4.65 Creación de base de datos SQLite auxiliar.....	165
Figura 4.66 Envío de datos al topic.....	165
Figura 4.67 Procesamiento de reseñas en el consumer.....	166
Figura 4.68 Contenedores del sistema de datos en ejecución.....	175
Figura 4.69 Tópicos Activos en Kafka-UI.....	176
Figura 4.70 Consola del Productor Mostrando Procesamiento y Envío de Dato.....	176
Figura 4.71 Contenido de un Mensaje de Tópico en Kafka-UI Procesado por el Consumidor.....	177
Figura 4.72 Logs del Consumidor Detallando una Solicitud POST Exitosa.....	178
Figura 4.73 Tablas almacenadas en PostgreSQL.....	179
Figura 4.74 Bienvenida a la aplicación.....	180
Figura 4.75 Landing page.....	181
Figura 4.76 Página de login.....	181
Figura 4.77 Página de inicio de sesión.....	182
Figura 4.78 Página principal de los usuarios.....	183
Figura 4.79 Búsqueda de producto con búsquedas recientes.....	184
Figura 4.80 Búsqueda de producto con sugerencias de búsqueda.....	184
Figura 4.81 Resultados de búsqueda de producto.....	185
Figura 4.82 Vista detallada de producto.....	186
Figura 4.83 Consulta del usuario al chatbot con respuesta basada en reseñas del producto.....	186
Figura 4.84 Landing page del chatbot.....	187
Figura 4.85 Búsqueda de producto mediante el chatbot.....	188
Figura 4.86 Página de favoritos.....	188
Figura 4.89 Página principal del administrador.....	191
Figura 4.90 Productos más guardados en favoritos en la página del administrador.....	192
Figura 4.91 Página de manejo de usuarios.....	192
Figura 4.92 Página de creación de informes.....	193
Figura 4.93 Ejemplo de manejo de usuarios con el chatbot.....	194
Figura 4.94 Ejemplo de generación de un reporte con el chatbot.....	195
Figura 4.95 Ejemplo de consulta para crear gráficos.....	196
Figura 4.96 Gráficos creados por el chatbot.....	196

Índice de tablas

Tabla 3.1 Estimación temporal de tareas.....	47
Tabla 3.2 Planificación general del proyecto.....	48
Tabla 3.3 Planificación de Proceso Iterativo.....	50
Tabla 3.4 Recursos humanos del proyecto y sus sueldos.....	55
Tabla 3.5 Coste de los RRHH para la empresa.....	56
Tabla 3.6 Coste de los RRHH para la empresa con impuestos.....	57
Tabla 3.7 Coste de recursos software.....	59
Tabla 3.8 Coste de recursos hardware.....	59
Tabla 3.9 Asignación de Recursos Humanos.....	60
Tabla 3.10 Costos totales del proyecto.....	61
Tabla 4.1 Requisitos no funcionales.....	69
Tabla 4.2 Requisitos funcionales.....	71
Tabla 4.3 RNF-01: Adaptabilidad a distintos dispositivos (responsive design).....	71
Tabla 4.4 RNF-02: Navegación clara mediante header persistente.....	72
Tabla 4.5 RNF-03: Integración no intrusiva del chatbot.....	72
Tabla 4.6 RNF-04: Uso de React, Flask y PostgreSQL.....	73
Tabla 4.7 RNF-05: Contenerización con Docker.....	73
Tabla 4.8 RNF-06: Integración de Apache Kafka para ingesta de datos.....	74
Tabla 4.9 RNF-07: Soporte del estándar HTTP.....	74
Tabla 4.10 RNF-08: Tiempo de respuesta del chatbot.....	75
Tabla 4.11 RNF-09: Tiempo de carga de productos.....	75
Tabla 4.12 RNF-10: Paginación en respuestas con grandes volúmenes de datos.....	76
Tabla 4.13 RNF-11: Almacenamiento cifrado de credenciales.....	76
Tabla 4.14 RNF-12: Acceso restringido al panel de administración.....	77
Tabla 4.15 RNF-13: Cumplimiento con la normativa de protección de datos.....	77
Tabla 4.16 RNF-14: Autenticación mediante token en la API.....	78
Tabla 4.17 RNF-15: Continuidad de servicio ante fallos de Kafka.....	78
Tabla 4.18 RNF-16: Registro y notificación de errores críticos.....	79
Tabla 4.19 RNF-17: Entorno de desarrollo reproducible con Docker Compose.....	79
Tabla 4.20 RNF-18: Código modular.....	80
Tabla 4.21 RNF-19: Mostrar logs para análisis posterior.....	80
Tabla 4.22 RF-01: Registro con email/contraseña o mediante Google/GitHub.....	81
Tabla 4.23 RF-02: Inicio de sesión con credenciales o proveedores sociales.....	81
Tabla 4.24 RF-03: Cierre de sesión seguro.....	82
Tabla 4.25 RF-04: Visualización del listado de usuarios por parte del administrador.....	82
Tabla 4.26 RF-05: Eliminación de usuarios por el administrador.....	83
Tabla 4.27 RF-06: Cambio de rol de usuario por el administrador.....	83
Tabla 4.28 RF-07: Diferenciación de accesos para usuario y administrador.....	84
Tabla 4.29 RF-08: Bloqueo de funciones protegidas sin autenticación.....	84
Tabla 4.30 RF-09: Limitación de funciones del bot según rol.....	85
Tabla 4.31 RF-10: Mostrar productos tras iniciar sesión.....	85

Tabla 4.32 RF-11: Guardar las búsquedas del usuario.....	86
Tabla 4.33 RF-12: Motor de búsqueda de productos.....	86
Tabla 4.34 RF-13: Vista detallada del producto (info, reviews, enlace).....	87
Tabla 4.35 RF-14: Añadir y quitar favoritos.....	87
Tabla 4.36 RF-15: Ver favoritos del usuario.....	88
Tabla 4.37 RF-16: Acceso al chatbot desde cualquier vista.....	88
Tabla 4.38 RF-17: Consultas generales al bot sobre productos.....	89
Tabla 4.39 RF-18: Respuestas del bot contextualizadas por producto.....	89
Tabla 4.40 RF-19: Generación de contenido para administradores por el bot (gráficos, reportes).....	90
Tabla 4.41 RF-20: Selección del proveedor de IA por parte del usuario.....	90
Tabla 4.42 RF-21: Acceso a métricas y estadísticas.....	91
Tabla 4.43 RF-22: Visualización de gráficos y mapa de calor de inicio de sesiones.....	91
Tabla 4.44 RF-23: Creación de reportes mediante formulario.....	92
Tabla 4.45 RF-24: Endpoint HTTP para verificar el estado del sistema.....	92
Tabla 4.46 RF-25: Lectura de archivos desde Kafka Producer.....	93
Tabla 4.47 RF-26: Envío de datos a topic.....	93
Tabla 4.48 RF-27: Consumo de datos y reenvío al backend.....	94
Tabla 4.49 RF-28: Procesamiento y almacenamiento en backend.....	94
Tabla 4.50 Flujo de interacción con el chatbot.....	168
Tabla 4.51 Flujo de generación de gráficos e informes.....	174
Tabla 4.52 Pruebas de sistema para el sistema de datos.....	181
Tabla 4.53 Pruebas de sistema para la aplicación.....	185

Glosario de términos

- **Dataset:** Un conjunto estructurado de datos, generalmente organizado en forma de tablas, archivos o bases de datos, que contiene información recopilada para análisis, investigación o aprendizaje automático [\[58\]](#).
- **Embeddings:** Un embedding es la representación numérica de datos (como texto o imágenes) en forma de vectores, donde cada número captura una característica específica. Esto permite procesar y analizar datos no estructurados en modelos de machine learning [\[34\]](#).
- **Flask:** Un framework web ligero de código abierto en Python, diseñado para el desarrollo rápido de aplicaciones web. Ofrece flexibilidad y simplicidad, permitiendo a los desarrolladores crear aplicaciones del lado del servidor escalables con una configuración mínima [\[42\]](#).
- **Framework:** Un marco de desarrollo web que proporciona una estructura robusta y completa para construir aplicaciones web, incluyendo herramientas y bibliotecas predefinidas para tareas como enrutamiento, gestión de bases de datos, autenticación y renderizado de plantillas. Está diseñado para acelerar el desarrollo al ofrecer componentes reutilizables y una arquitectura organizada, manteniendo un equilibrio entre flexibilidad y productividad [\[14\]](#).
- **Function Calling:** La invocación de funciones es una capacidad que permite a los modelos de IA interactuar con código externo o APIs para ejecutar tareas específicas, como obtener datos o realizar acciones, ampliando su utilidad y aplicabilidad [\[40\]](#).
- **IA Generativa:** La inteligencia artificial generativa (IA generativa) es un campo de la inteligencia artificial que se centra en crear modelos y sistemas capaces de generar contenido nuevo, como imágenes, videos, música o texto. Al entrenarse con grandes conjuntos de datos, los modelos de IA generativa aprenden patrones y

estructuras dentro de los datos para producir resultados novedosos y realistas que imitan la distribución original de los datos [\[51\]](#).

- **Inteligencia Artificial:** Capacidad de los sistemas computacionales para realizar tareas típicamente asociadas con la inteligencia humana, como el aprendizaje, el razonamiento, la resolución de problemas, la percepción y la toma de decisiones. Es un campo de investigación dentro de la informática que desarrolla y estudia métodos y software que permiten a las máquinas percibir su entorno y utilizar el aprendizaje y la inteligencia para tomar acciones que maximicen sus posibilidades de alcanzar objetivos definidos [\[6\]](#).
- **Kafka:** Apache Kafka es un sistema de mensajería distribuido que sigue el modelo pub/sub (publicador/suscriptor) para procesar flujos de datos en tiempo real [\[43\]](#).
- **Kafka Consumer:** Un Consumer (consumidor) en Kafka es una aplicación o servicio que se suscribe a uno o más topics para leer y procesar mensajes escritos por los producers. Opera dentro de un consumer group para permitir escalabilidad y balanceo de carga entre múltiples instancias [\[43\]](#).
- **Kafka producer:** Un Producer (productor) en Kafka es una aplicación o servicio que publica mensajes en topics específicos del clúster Kafka. Funciona de manera asíncrona, enviando datos a particiones del topic según una clave o mediante balanceo round-robin (si la clave es nula) [\[43\]](#).
- **LLM:** De sus siglas en inglés un *Large Language Model* es un modelo de IA basado en transformadores que procesa texto en paralelo para comprender y generar lenguaje con alta precisión [\[26\]](#).
- **Microframework:** Un marco de desarrollo web ligero que ofrece herramientas básicas como enrutamiento y manejo de solicitudes HTTP, destacando por su simplicidad y flexibilidad. En este proyecto, se usa Flask como microframework para la aplicación web. [\[42\]](#).
- **Pgvector:** Una extensión de código abierto para PostgreSQL que permite el almacenamiento, indexación y consulta eficiente de datos vectoriales de alta

dimensionalidad para búsquedas de similitud. Soporta operaciones como distancia L2, distancia coseno y búsquedas de vecinos más cercanos, ideal para aplicaciones de procesamiento de lenguaje natural, sistemas de recomendación y aprendizaje automático [\[44\]](#).

- **PostgreSql:** Un sistema de gestión de bases de datos relacionales de código abierto, avanzado y conocido por su robustez, extensibilidad y cumplimiento con SQL. Soporta una amplia gama de tipos de datos y extensiones, como pgvector, para el almacenamiento y consulta de datos complejos en diversas aplicaciones [\[45\]](#).
- **Openrouter:** OpenRouter es una plataforma que actúa como un enrutador de modelos de inteligencia artificial, proporcionando una API que permite a los usuarios acceder y utilizar múltiples modelos de IA de diferentes proveedores a través de una interfaz unificada. Facilita la integración y el uso eficiente de modelos de lenguaje para desarrolladores y aplicaciones [\[41\]](#).
- **React:** Una biblioteca de JavaScript para construir interfaces de usuario dinámicas e interactivas, especialmente para aplicaciones de página única. Permite crear componentes de interfaz reutilizables y gestionar el estado de la aplicación de manera eficiente, facilitando el desarrollo de aplicaciones web y nativas [\[47\]](#).
- **Retrieval-Augmented Generation (RAG):** RAG es una técnica que mejora la generación de texto recuperando información relevante desde la base de datos mediante búsqueda vectorial y pasándola como contexto al LLM [\[31\]](#).
- **Stella_en_v5:** Stella_en_v5 es un modelo de lenguaje desarrollado por BleLabs, disponible en Hugging Face, optimizado para generar representaciones vectoriales de texto en inglés. Está diseñado para tareas de procesamiento de lenguaje natural, como búsqueda semántica, clasificación de texto o clustering en aplicaciones [\[18\]](#).
- **Zookeeper:** ZooKeeper es un servicio centralizado esencial para la operación de Kafka, actuando como sistema de coordinación y gestión de metadatos del clúster. Su función principal es mantener el estado consistente de la arquitectura distribuida de Kafka [\[43\]](#).

1 INTRODUCCIÓN

La inteligencia artificial generativa ha emergido como una de las tecnologías más disruptivas en los últimos años, con aplicaciones que van desde la creación de contenido hasta la automatización de procesos en múltiples sectores. En particular, los modelos de lenguaje de gran escala (*LLMs*) han revolucionado la forma en que los sistemas comprenden y generan texto, permitiendo interacciones más naturales y eficientes con los usuarios. Su uso se ha extendido a áreas como el servicio al cliente, la generación de texto automatizada y la mejora de la búsqueda de información en plataformas digitales.

Este crecimiento ha sido impulsado por avances en modelos de lenguaje y un aumento en la inversión en tecnologías de procesamiento del lenguaje natural (NLP). Se estima que el mercado de la IA generativa alcanzará los **62.720 millones de dólares en 2025**, lo que refleja su creciente adopción en diversas industrias.

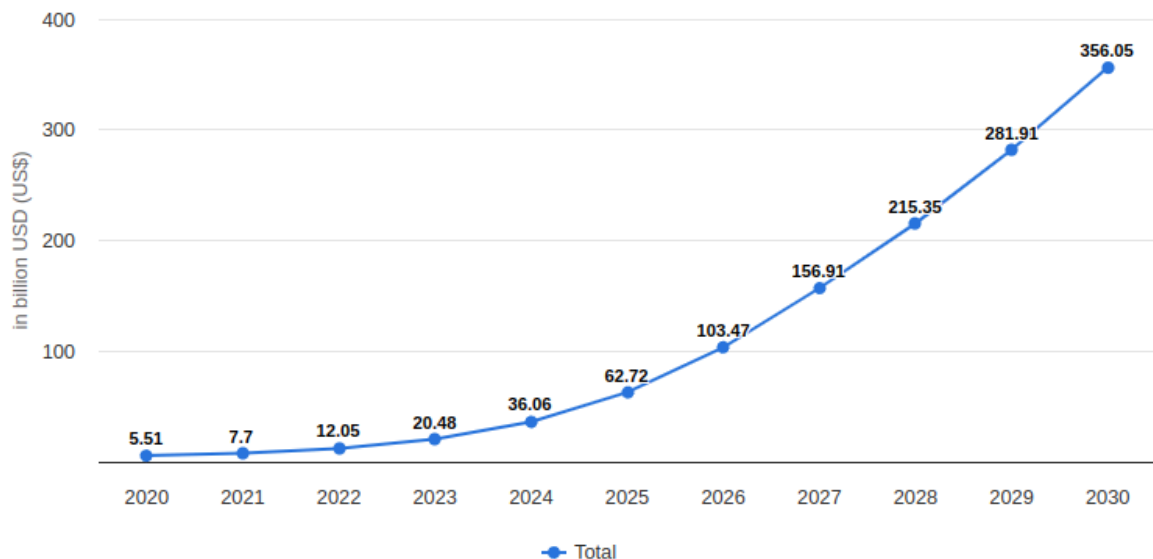


Figura 1.1 Predicción del tamaño de mercado de la IA Generativa en millones de USD [\[51\]](#).

Dentro de este contexto, una de las áreas donde la IA generativa puede marcar una diferencia significativa es en la búsqueda y recuperación de información en plataformas de comercio electrónico. A medida que el volumen de productos disponibles en línea sigue en aumento, los usuarios enfrentan el desafío de encontrar aquellos que mejor se ajusten a sus necesidades. Los motores de búsqueda tradicionales, basados en palabras clave, presentan limitaciones al no comprender la intención del usuario ni las relaciones semánticas entre productos y la consulta del usuario, lo que puede resultar en resultados poco relevantes. Este problema es especialmente relevante considerando que una familia estadounidense promedio dedica casi seis horas a la semana a la planificación y compra de productos para el hogar [\[57\]](#).

Para abordar este problema, en este proyecto se ha desarrollado una aplicación que combina IA generativa con técnicas avanzadas de recuperación de información. En el centro de esta solución se encuentra un **asistente conversacional basado en IA generativa**, que actúa como interfaz principal para gestionar consultas en lenguaje natural. Este chatbot no solo permite a los usuarios interactuar de manera intuitiva con la plataforma, sino que también facilita el acceso a recomendaciones personalizadas y resultados relevantes mediante el uso de técnicas como Retrieval-Augmented Generation (RAG), que permite a la IA generar respuestas basadas en información recuperada de la base de datos en tiempo real. Además, se emplea Function Calling, una técnica que permite a la IA interactuar con funciones definidas en nuestro código para realizar tareas específicas, como recuperar datos estructurados o ejecutar consultas avanzadas. A través de *embeddings*, los datos de los productos son representados en un espacio vectorial, lo que permite identificar similitudes con la pregunta del usuario de manera más efectiva y proporcionar recomendaciones más precisas.

Un problema adicional identificado es la necesidad de herramientas administrativas para gestionar y analizar datos de manera efectiva. Para abordar esto, la aplicación incorpora al bot funciones para administradores, permitiéndoles generar reportes, crear gráficos y gestionar usuarios, lo que amplía las capacidades de la plataforma. Esta integración de tecnologías no solo mejora la experiencia de búsqueda, sino que también agiliza la

interacción de los usuarios con grandes volúmenes de datos, posicionando a la IA generativa como un asistente conversacional avanzado y una solución eficiente para entornos digitales.

1.1 OBJETIVOS

El objetivo principal de este proyecto es mejorar la experiencia de los usuarios en las plataformas de comercio electrónico, minimizando el tiempo que dedican a la búsqueda de productos y optimizando la accesibilidad a la información relevante. A través de la integración de inteligencia artificial se busca desarrollar una aplicación fluida y responsiva que permita a los usuarios obtener respuestas rápidas y precisas a sus consultas mediante el chatbot, sin tener que pasar horas en menús de búsqueda. Para abordar la necesidad de herramientas administrativas, la aplicación incluye funciones para administradores, como la generación de reportes, creación de gráficos y gestión de usuarios, potenciando la eficiencia y las capacidades de la plataforma.

Este proyecto ofrece una valiosa oportunidad para adquirir conocimientos y explorar en profundidad el uso de embeddings e inteligencia artificial generativa, dado que se inicia sin experiencia previa en estas áreas. La aplicación de estas tecnologías facilitará una comprensión más completa de su funcionamiento, sus usos prácticos y los retos que implica su implementación en entornos reales, como la búsqueda de información en plataformas de comercio electrónico.

Los objetivos específicos son los siguientes:

- **Ahorrar tiempo a los consumidores:** Proporcionar una herramienta que guíe a los usuarios en la búsqueda de productos mediante lenguaje natural de manera rápida y eficiente, eliminando la necesidad de realizar múltiples consultas y filtrados manuales.
- **Construir un motor de búsqueda eficiente:** Desarrollar un sistema que combine tecnologías avanzadas de recuperación de información, como la búsqueda semántica y el análisis de datos en tiempo real, para ofrecer resultados precisos y personalizados.

- **Crear una interfaz fluida y responsiva:** Desarrollar una interfaz de usuario intuitiva y adaptable que se ajuste a diferentes dispositivos y tamaños de pantalla, garantizando una experiencia de búsqueda cómoda y eficiente en ordenadores de sobremesa.
- **Proveer herramientas de análisis de datos a administradores:** Equipar a los administradores con funcionalidades integradas en el bot que permitan generar reportes detallados y gráficos personalizados sobre el comportamiento de los usuarios, tendencias de búsqueda y rendimiento de productos, facilitando la toma de decisiones basada en datos en tiempo real.

En resumen, el propósito fundamental de esta aplicación es transformar la experiencia de búsqueda de productos y administración de la plataforma en una experiencia conversacional fluida, eliminando la necesidad de búsquedas tradicionales y ofreciendo una interacción directa con el bot que guía al usuario de manera rápida y eficiente hacia la información que busca. Al mismo tiempo, este proyecto servirá como un aprendizaje práctico sobre cómo utilizar embeddings e IA generativa, explorando sus aplicaciones y beneficios en entornos digitales.

1.2 METODOLOGÍA EMPLEADA

En este proyecto se ha adoptado una metodología ágil e iterativa, que facilita un desarrollo continuo con entregas periódicas y la adaptación constante a los cambios. El trabajo se ha estructurado en dos fases diferenciadas, cada una con una duración específica de los ciclos de desarrollo. En la primera fase, los periodos de trabajo fueron de una semana, con el propósito de establecer una base funcional sólida y validar los aspectos fundamentales del sistema. En la segunda fase, los ciclos se ampliaron a dos semanas, facilitando la implementación de características más avanzadas y la optimización de las ya existentes.

Durante cada iteración, el desarrollo se ha organizado en cuatro etapas:

- **Análisis y planificación:** En esta fase se definen los objetivos del ciclo. En una reunión con el tutor, se identifican los requisitos técnicos y se establece un plan de acción para su implementación. Se tiene en cuenta la retroalimentación de iteraciones previas para ajustar la dirección del proyecto.
- **Implementación:** Se desarrollan las funcionalidades planificadas, incorporando tanto nuevas características como mejoras sobre los componentes existentes. Se procura que cada entrega aporte un valor añadido al sistema.
- **Pruebas y validación:** Se realizan pruebas para garantizar que las nuevas incorporaciones funcionarán correctamente sin comprometer la estabilidad global del sistema. Se emplearon pruebas funcionales para verificar el cumplimiento de los requisitos.
- **Evaluación y ajustes:** Se llevó a cabo una revisión de los resultados obtenidos, identificando posibles fallos o áreas de mejora. Los aprendizajes de cada ciclo se integraron en la planificación de la siguiente iteración para perfeccionar el desarrollo.

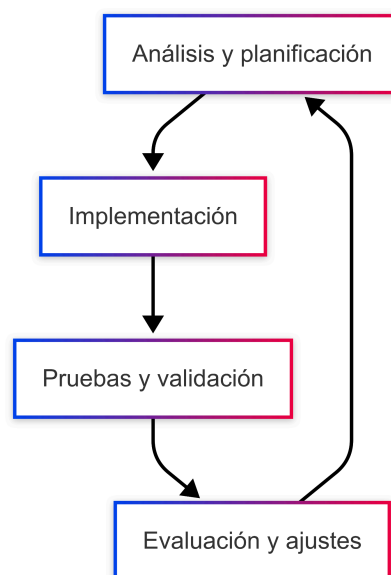


Figura 1.2 Fases de la metodología iterativa.

Este enfoque ha permitido un progreso continuo, minimizando riesgos y adaptando el sistema a las necesidades que surgieron a lo largo del proceso. Gracias a la retroalimentación constante y a la flexibilidad del modelo iterativo, se ha logrado un producto robusto y alineado con los objetivos iniciales del proyecto.

1.3 TECNOLOGÍA EMPLEADA

1.3.1 EVALUACIÓN DE ALTERNATIVAS

1. Bases de datos

Como se explica en el marco teórico (sección [2.4.3](#) y [2.4.4](#)), el uso de embeddings vectoriales en este proyecto requiere una base de datos capaz de gestionar operaciones sobre vectores de alta dimensionalidad. A continuación, se analizan las principales alternativas tecnológicas para este propósito..

- **Bases de datos relacionales:**

Las bases de datos relacionales, como PostgreSQL [\[45\]](#) y MySQL/MariaDB, han sido ampliamente utilizadas por su robustez y capacidad para manejar transacciones complejas. PostgreSQL destaca por su flexibilidad y soporte para extensiones, como *pgvector* [\[44\]](#), que permite el almacenamiento y la búsqueda eficiente de vectores de alta dimensionalidad, esencial para aplicaciones que trabajan con embeddings. Por otro lado, MySQL [\[35\]](#) y MariaDB [\[27\]](#), aunque populares, carecen de soporte nativo para operaciones eficientes con vectores, lo que limita su uso en este ámbito.

- **Bases de datos no relacionales:**

Las bases de datos no relacionales, como MongoDB [\[33\]](#), han ganado relevancia por su capacidad para manejar datos no estructurados y escalar horizontalmente. MongoDB, en particular, ha incorporado funcionalidades avanzadas para el manejo de vectores, incluyendo algoritmos optimizados como el Approximate Nearest Neighbor (ANN), que permite búsquedas rápidas de similitud en espacios de alta dimensionalidad [\[34\]](#). Esto lo convierte en una opción ideal para aplicaciones como sistemas de recomendación o

procesamiento de lenguaje natural, donde los embeddings son clave. Además, su flexibilidad y escalabilidad lo hacen adecuado para entornos dinámicos y en constante evolución.

2. Tecnologías de desarrollo

Frontend

Para el desarrollo de la interfaz de usuario, React [\[47\]](#) es una de las opciones más populares debido a su eficiencia en la creación de interfaces dinámicas y su amplia comunidad de soporte. Alternativas como Vue.js [\[56\]](#) y Angular [\[4\]](#) también son viables, ofreciendo enfoques similares pero con diferentes filosofías de desarrollo.

Backend

En el backend, Flask [\[42\]](#) con Python [\[46\]](#) es una excelente opción para construir APIs ligeras y escalables, ideal para manejar operaciones con embeddings y bases de datos vectoriales. Otras alternativas incluyen Django [\[14\]](#) (también con Python), que proporciona un *framework* más completo con herramientas integradas, y Node.js con Express [\[30\]](#), una opción popular para aplicaciones basadas en JavaScript.

3. Modelo de embeddings

En el contexto del proyecto, la generación de embeddings es fundamental para realizar operaciones avanzadas como búsquedas semánticas y recomendaciones. Existen varias alternativas para implementar esta funcionalidad.

La primera opción es ejecutar modelos locales como BERT o stella_en_v5, lo que ofrece mayor control y privacidad sobre los datos, aunque requiere recursos computacionales significativos. BERT, basado en transformadores bidireccionales, es ideal para tareas de similitud semántica gracias a su preentrenamiento con objetivos como el modelado de lenguaje enmascarado (MLM) y la predicción de la siguiente frase (NSP) [\[12\]](#). Por su parte, stella_en_v5, derivado de modelos como Alibaba-NLP/gte-large-en-v1.5, se especializa en recuperación de información y similitud semántica, utilizando dimensiones

de embedding que van desde 512 hasta 8192, con 1024 dimensiones como punto óptimo entre rendimiento y eficiencia [18]. Ambos modelos son robustos y versátiles, aunque demandan un esfuerzo computacional considerable.

Otra alternativa es utilizar proveedores externos, como OpenAI Embeddings [38], que simplifican el proceso al delegar la generación de embeddings a servicios en la nube, reduciendo la carga local y ofreciendo escalabilidad inmediata, aunque con costos adicionales y dependencia de terceros.

4. LLM

En el ámbito de los Modelos de Lenguaje de Gran Escala (LLM), existen diversas alternativas para integrar y aprovechar estas tecnologías en aplicaciones avanzadas. Entre estas opciones, se encuentra la posibilidad de utilizar modelos locales, como LLaMA [25] o DeepSeek-R1 [17], que permiten un control total sobre los datos y la infraestructura, ideal para entornos donde la privacidad y la personalización son prioritarias. Sin embargo, es importante destacar que ejecutar un modelo avanzado en local resulta inviable sin contar con una infraestructura de hardware de alto rendimiento, cuyo costo puede ascender a varios millones de dólares.

La alternativa más viable y común es recurrir a servicios en la nube ofrecidos por proveedores como por ejemplo OpenAI (GPT-4) [39], que simplifican la implementación y ofrecen escalabilidad inmediata. Estos servicios son ideales para proyectos que necesitan rapidez en la implementación y no desean gestionar infraestructura compleja, aunque implican costos adicionales y dependencia de terceros.

Otra alternativa es usar plataformas como Eden AI [15] o OpenRouter [41] permiten integrar múltiples LLM de diferentes proveedores en una sola API, ofreciendo flexibilidad y la posibilidad de comparar y elegir el mejor modelo para cada caso de uso. Estas herramientas son especialmente útiles en entornos donde se requiere combinar diferentes capacidades de LLM o evitar la dependencia de un único proveedor.

5. Integración de Datos

Para facilitar el flujo de información desde una API o un archivo hacia la base de datos del sistema, tecnologías como Apache Kafka [\[19\]](#) y Amazon Kinesis [\[3\]](#) son opciones destacadas. Ambas permiten gestionar el streaming de datos y procesar grandes volúmenes de información en tiempo real. Sin embargo, Kafka requiere una configuración más compleja, ya que depende de la gestión de clústeres, nodos, replicaciones y particiones. Por otro lado, Amazon Kinesis se presenta como una solución más sencilla, lista para producción en pocas horas, ideal para proyectos que buscan rapidez en la implementación [\[19\]\[3\]](#).

Finalmente para obtener un *dataset* de productos de Amazon, una opción práctica es utilizar plataformas como Hugging Face [\[16\]](#), que ofrece una variedad de conjuntos de datos predefinidos, incluyendo aquellos relacionados con reseñas y características de productos. Otra alternativa es el web scraping, aunque esta técnica tiene muchas limitaciones, como restricciones legales, políticas de Amazon y la complejidad técnica para manejar dinámicas de páginas web.

Por otro lado, se puede acceder a datos estructurados mediante la Amazon Product Advertising API [\[1\]](#), que proporciona información detallada de productos, aunque para usarla es necesario ser afiliado del programa de asociados de Amazon.

1.3.2 ALTERNATIVAS ELEGIDAS

Para el desarrollo de la aplicación, se optó por una base de datos relacional **PostgreSQL**, ya que su esquema relacional encajaba perfectamente con la estructura de la aplicación. Además, dado que ya se contaba con experiencia previa en bases de datos NoSQL como MongoDB, se decidió explorar una nueva tecnología, utilizando pgvector para manejar vectores dentro de PostgreSQL. En el backend, se utilizó **Flask** junto con **Python**, aprovechando SQLAlchemy como ORM para facilitar la interacción con la base de datos. Para la gestión del flujo de datos desde el dataset hacia la base de datos, se eligió **Apache Kafka** junto con Zookeeper (para la coordinación y gestión de los nodos del clúster), gracias a su capacidad para procesar grandes volúmenes de datos en tiempo real. Tanto el productor (encargado de ingestar los datos) como el consumer (que los procesa y almacena)

fueron implementados en **Python**, utilizando la librería confluent-kafka. Para el frontend, se seleccionó **React** junto con React Router, permitiendo una interfaz de usuario dinámica y con enrutamiento eficiente.

- **Dataset**

Se optó por el dataset Amazon Reviews'23 [\[2\]](#) debido a que ofrece varias ventajas sobre otros conjuntos de datos similares. Estas ventajas incluyen:

- Mayor volumen de datos: Contiene 571.54 millones de reseñas, por lo que es suficiente para el desarrollo de la aplicación.
- Metadatos más ricos: Incluye características descriptivas adicionales en los metadatos de los productos, lo que facilita análisis más detallados y personalizados.
- Procesamiento más limpio: Los metadatos de los productos están mejor estructurados y limpios, lo que reduce las tareas de preprocesamiento.

- **Modelos de Inteligencia Artificial**

En cuanto a los modelos de inteligencia artificial, se seleccionó el modelo local **stella_en_v5** [\[18\]](#) para la generación de embeddings, debido a su eficiencia y precisión en la representación de datos. Además es un modelo local por lo que no se incurrirá en costes adicionales.

Para el modelo de lenguaje (**LLM**), inicialmente se utilizó Eden AI, pero tras encontrar varios problemas con su API, se decidió migrar a **OpenRouter** [\[41\]](#). A través de esta plataforma, se probaron diversos modelos de lenguaje, destacando principalmente el uso de GPT-4o-mini por su equilibrio entre rendimiento y coste.

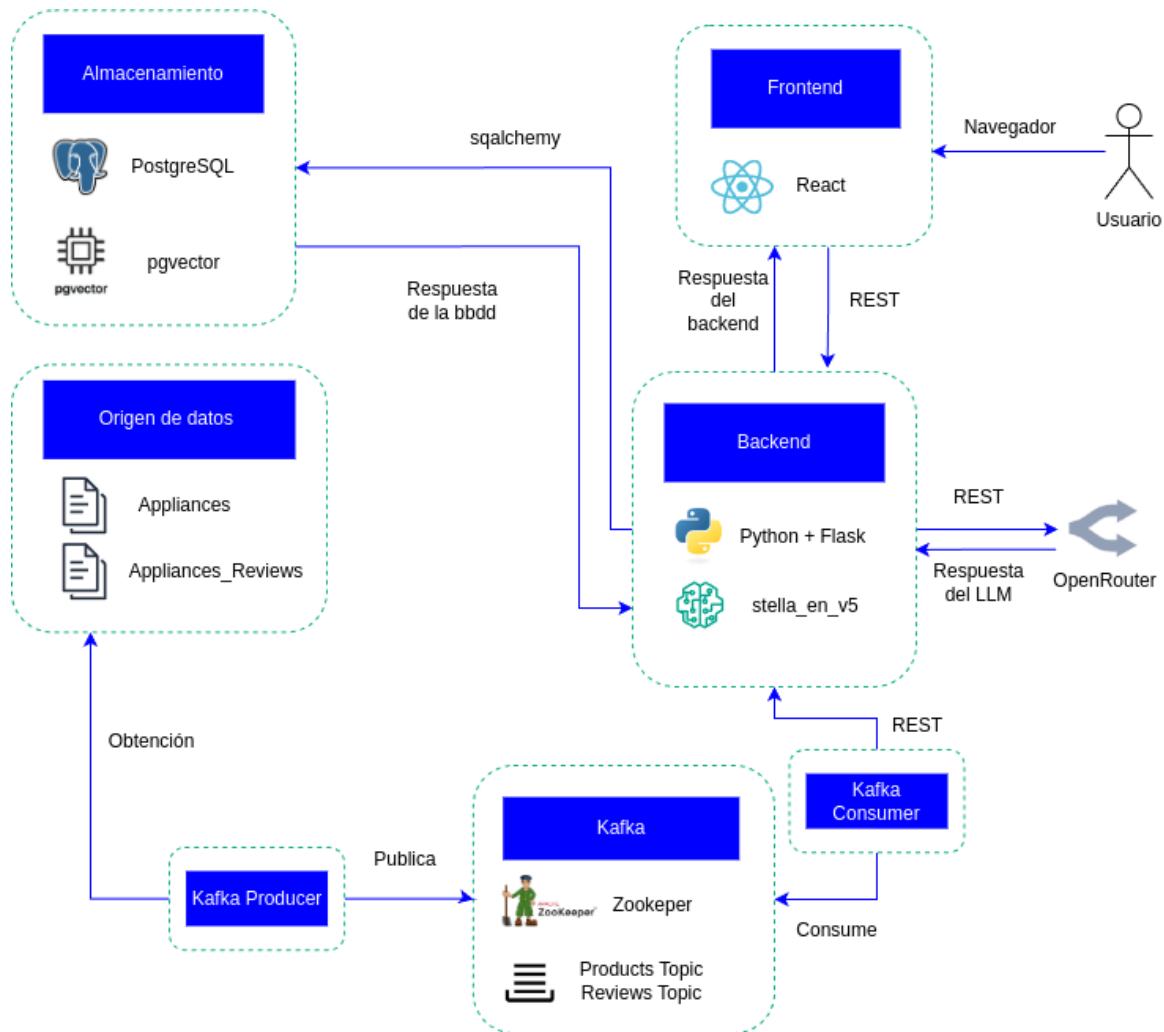


Figura 1.3 Estructura general de la solución

- Docker y Contenerización

Para facilitar el despliegue y la gestión de los diferentes servicios, se utilizó Docker. Cada componente de la aplicación (frontend, backend, kafka y base de datos) fue contenerizado, y se ejecutaron todos los servicios de manera coordinada mediante un archivo `docker-compose.yml`. Esto permitió un entorno de desarrollo y producción consistente, simplificando la configuración y el escalado de la aplicación.

Herramientas Adicionales

- **Docker Desktop:** Para la gestión de contenedores.
- **Postman:** Para realizar pruebas exhaustivas del backend.
- **VSCode:** Como entorno de desarrollo.
- **Google Docs:** Para la documentación del proyecto.
- **Microsoft Teams y Google Meet:** Para las reuniones con los tutores.
- **GitHub:** Como sistema de control de versiones, permitiendo un trabajo colaborativo y organizado.

1.4 ESTRUCTURA DEL TRABAJO

La memoria del proyecto se organiza en cuatro secciones principales, cada una de las cuales aborda una fase clave del desarrollo de la aplicación:

- **Análisis del problema:** En esta sección se describe el problema que se pretende resolver, analizando su contexto y las tecnologías, herramientas o trabajos previos relacionados con el ámbito del proyecto. Se realiza una comparativa de las opciones disponibles y se selecciona la más adecuada en función de las necesidades del proyecto.
- **Gestión del proyecto:** En esta sección se define el alcance del proyecto, se diseña un plan de trabajo detallado y se administran los recursos y los riesgos asociados. Además, se analizan los aspectos legales y normativos relevantes que deben considerarse a lo largo del desarrollo del proyecto.
- **Desarrollo de la solución:** Esta parte constituye el núcleo del proyecto. Se detalla la solución propuesta, describiendo el proceso de desarrollo, los requisitos funcionales y no funcionales, la fase de diseño, la implementación y las pruebas realizadas. También se presenta el producto final, explicando su funcionalidad y el proceso de despliegue.
- **Conclusiones:** En esta última sección se valida el cumplimiento de los requisitos iniciales, evaluando el producto final y verificando si se han alcanzado los



objetivos planteados. Se incluye un análisis detallado de los resultados obtenidos y su correspondencia con las expectativas iniciales.

2 ESTUDIO DEL PROBLEMA

En este apartado se analiza el contexto del problema, centrándose en las limitaciones de los motores de búsqueda tradicionales y su impacto en la experiencia del usuario. Se examina el estado del arte en torno a la recuperación de información, así como las alternativas tecnológicas disponibles en el mercado, evaluando su viabilidad para optimizar la búsqueda en plataformas de comercio electrónico.

2.1 EL CONTEXTO DEL PROBLEMA

En el comercio electrónico, los usuarios demandan la búsqueda de productos de manera rápida y eficiente, demandando respuestas precisas y relevantes. Tradicionalmente, esto se realiza mediante búsquedas basadas en palabras clave, donde el usuario introduce términos específicos y el sistema devuelve resultados que coinciden con esos términos.

Como alternativa, los chatbots han ganado popularidad al permitir a los usuarios realizar consultas en lenguaje natural, ofreciendo una interacción más intuitiva y cercana. A través de estas herramientas, los usuarios pueden formular preguntas complejas y recibir respuestas directas, eliminando la necesidad de navegar manualmente por múltiples opciones.

Una vez realizada la consulta, ya sea mediante un chatbot o un motor de búsqueda tradicional, el sistema accede a la base de datos para recuperar la información relevante. Este proceso implica filtrar y clasificar grandes volúmenes de datos, como descripciones de productos, reseñas y metadatos, para presentar los resultados más adecuados. Sin embargo, la eficacia de este paso depende en gran medida de la capacidad del sistema para comprender y procesar la consulta del usuario de manera precisa y contextualizada.

2.2 EL ESTADO DEL ARTE

En el ámbito del comercio electrónico, la búsqueda y recuperación de información es un componente crítico para garantizar una experiencia de usuario satisfactoria. Los motores de

búsqueda tradicionales, basados en palabras clave, han sido la norma durante años, pero presentan limitaciones significativas en términos de comprensión semántica y relevancia de los resultados. A medida que el volumen de productos disponibles en línea sigue creciendo, los usuarios enfrentan el desafío de encontrar aquellos que mejor se ajusten a sus necesidades, lo que ha llevado a la adopción de tecnologías más avanzadas para mejorar la precisión y la eficiencia de las búsquedas.

En este contexto, los principales actores del comercio electrónico han desarrollado motores de búsqueda que combinan técnicas tradicionales con algoritmos más sofisticados para ofrecer resultados más relevantes y personalizados. Estos sistemas no solo se basan en la coincidencia de palabras clave, sino que también incorporan factores como la popularidad del producto, la reputación del vendedor y el comportamiento del usuario para mejorar la precisión de los resultados.

En el siguiente apartado, se estudiará cómo funcionan los principales motores de búsqueda de productos en plataformas líderes como Amazon, eBay y Walmart analizando sus enfoques y las técnicas que utilizan para optimizar la experiencia del usuario.

2.2.1 ALTERNATIVAS SIMILARES

En este apartado se analizará cómo otras plataformas de comercio electrónico abordan el proceso de búsqueda y recuperación de productos, desde la interacción del usuario hasta la presentación de resultados. Existen miles de motores de búsqueda y enfoques para lograrlo, pero nos centraremos en los más destacados y reconocidos del sector. Se examinarán las soluciones implementadas por líderes del mercado, evaluando tanto los métodos tradicionales basados en palabras clave como las innovaciones recientes que incorporan inteligencia artificial, algoritmos avanzados y chatbots para mejorar la experiencia de compra.

2.2.1.1 AMAZON A9

Basándonos en el estudio de “How Amazon's e-commerce works?” [\[26\]](#), el algoritmo A9 de Amazon es el sistema que la plataforma utiliza para clasificar los productos en los

resultados de búsqueda, con el objetivo de maximizar las ventas. Funciona en dos pasos principales: primero, filtra los productos que no son relevantes para la consulta del usuario basándose en las palabras clave. Si un producto no contiene las palabras clave de la búsqueda, no aparecerá en los resultados. En segundo lugar, Amazon evalúa el rendimiento de los productos restantes, considerando factores como la tasa de clics (CTR), la tasa de conversión (CR) y las ventas. Estos indicadores determinan la probabilidad de compra, que es crucial para la clasificación final. A diferencia de Google, el A9 prioriza la conversión de ventas, ya que Amazon tiene un interés directo en promover productos que generen transacciones. Este enfoque asegura que los productos más relevantes y con mayor probabilidad de venta aparezcan en las primeras posiciones, beneficiando tanto a los compradores como a los vendedores. Además, el algoritmo analiza meticulosamente cada detalle de las características del producto, como la calidad de las imágenes (optimizadas para móviles) y la claridad de las descripciones, ya que estos elementos influyen directamente en la visibilidad y el rendimiento del listado.

El algoritmo A9 de Amazon destaca por su capacidad para maximizar las ventas al priorizar productos con alta probabilidad de conversión, beneficiando tanto a vendedores como a compradores. Sin embargo, la complejidad en la optimización obliga a los vendedores a invertir tiempo y recursos en mejorar constantemente sus listados, mientras que su enfoque comercial en la conversión limita la diversidad de resultados. Además, la sensibilidad a cambios en el algoritmo puede afectar significativamente la visibilidad y ventas. Estas limitaciones podrían mejorarse mediante la implementación de búsqueda vectorial, que permitiría una comprensión más profunda de la intención del usuario y una mayor diversidad en los resultados, reduciendo la dependencia de palabras clave exactas y adaptándose dinámicamente a las necesidades del mercado. Por otro lado, Amazon tampoco cuenta con un chatbot con el que los usuarios puedan interactuar en tiempo real utilizando lenguaje natural, lo que podría ser una oportunidad para mejorar la experiencia del cliente y ofrecer un soporte más inmediato y personalizado.

2.2.1.2 EBAY CASSINI

El algoritmo de búsqueda de eBay, conocido como Cassini, está diseñado específicamente para manejar las necesidades únicas de un mercado en línea donde los productos cambian constantemente y los resultados deben ser precisos, personalizados y actualizados en tiempo real. A continuación, se presenta una explicación simplificada de su funcionamiento basándose en el artículo [\[53\]](#).

1. Arquitectura Distribuida:

eBay divide su enorme colección de productos en fragmentos llamados shards, que se buscan en paralelo en diferentes servidores. Esto permite manejar un gran volumen de consultas de manera eficiente y escalable.

2. Actualización en Tiempo Real:

Dado que los productos en eBay cambian rápidamente (alrededor del 20% se actualizan diariamente), el sistema utiliza una técnica llamada probing para calcular estadísticas globales de términos, como la frecuencia de términos (TF) y la frecuencia inversa de documentos (IDF), en tiempo real. Estas estadísticas permiten ajustar el ranking de resultados, garantizando que sean precisos, relevantes y reflejan la información más actualizada.

3. Transformación de Consultas:

Cuando un usuario realiza una búsqueda, el sistema transforma automáticamente la consulta para mejorar su precisión. Esto incluye correcciones ortográficas, expansión de términos (añadir palabras relacionadas) y relajación de consultas (eliminar términos demasiado restrictivos). Esto ayuda a evitar resultados vacíos y mejora la relevancia.

4. Ranking Personalizado:

El orden de los resultados se determina mediante un sistema de ranking en múltiples etapas, que utiliza modelos de aprendizaje automático. Estos modelos consideran cientos

de factores, como el precio, la reputación del vendedor, la ubicación del comprador y detalles específicos del producto. Esto permite que los resultados se adapten a las preferencias individuales de cada usuario.

5. Indexación Ágil:

Para mantener el índice de productos actualizado, eBay combina reconstrucciones completas del índice cada 8 horas con actualizaciones parciales (llamadas "minis") que se realizan cada pocos minutos. Esto garantiza que los cambios en los productos se reflejen rápidamente en los resultados de búsqueda.

6. Integración de Datos del Vendedor:

eBay mantiene un índice separado para los vendedores, lo que permite que características como su reputación o ubicación se integren en el ranking de resultados sin necesidad de actualizar todos los productos de ese vendedor.

En resumen, el algoritmo de búsqueda de eBay está optimizado para un entorno dinámico y altamente personalizado, donde la precisión, la velocidad y la actualización en tiempo real son fundamentales. Su arquitectura distribuida, combinada con técnicas avanzadas de transformación de consultas y ranking, lo convierte en un sistema robusto y eficiente para un mercado en línea tan complejo como eBay. Aunque enfrenta desafíos como la complejidad del sistema, estas limitaciones son el resultado natural de manejar un entorno tan dinámico y cambiante. La capacidad de personalizar resultados basados en el comportamiento del usuario y mantener índices actualizados en tiempo real demuestra la innovación y adaptabilidad del sistema, que sigue siendo un referente en la industria del comercio electrónico. Destacar que eBay no incluye un chatbot con el que los usuarios puedan interactuar en lenguaje natural, lo que podría ser una oportunidad de mejora para el sistema en el futuro.

2.2.1.3 WALMART POLARIS

El motor de búsqueda Polaris, desarrollado por Walmart, es una herramienta avanzada que combina tecnologías semánticas y análisis de datos para mejorar la experiencia de búsqueda en su plataforma de comercio electrónico. Polaris utiliza la tecnología Social Genome, adquirida a través de la startup Kosmix, para analizar señales sociales y relaciones semánticas entre productos, personas, eventos y lugares [22]. Esto permite al motor comprender el contexto detrás de las búsquedas de los usuarios, como diferenciar entre "Sugar" (el alimento) y "Sugar" (la canción de Maroon 5), gracias a su capacidad para procesar datos no estructurados y complejos [24]. Además, Polaris integra señales sociales como los "Me gusta" en Facebook y los "pines" en Pinterest, junto con reseñas y calificaciones de productos, para mejorar la relevancia de los resultados [22].

Otra característica clave de Polaris es su sistema de puntuación de compromiso (engagement scoring), que se basa en el comportamiento real de los usuarios, como los productos que hacen clic, añaden al carrito o compran después de una búsqueda. Esto permite al motor aprender y adaptarse a las preferencias individuales, mejorando la precisión de las recomendaciones. Por ejemplo, si un usuario busca "flats", Polaris sabe que es más probable que esté buscando zapatos planos en lugar de pantallas planas, gracias a este sistema de puntuación [22].

En cuanto a su impacto, Polaris ha demostrado aumentar entre un 10% y un 15% la probabilidad de que los usuarios completen una compra después de realizar una búsqueda [22]. Esta mejora se debe a su capacidad para categorizar productos de manera semántica, extrayendo atributos clave de las descripciones (como "14 quilates" en un collar de oro) y haciendo que estos atributos sean buscables. Además, Polaris ha sido diseñado para escalar globalmente, con planes de implementación en mercados internacionales como Brasil y otros 26 países donde Walmart tiene presencia [22].

En resumen, Polaris representa un avance significativo en la búsqueda de productos en línea, combinando tecnologías semánticas, análisis de datos sociales y aprendizaje basado en el comportamiento del usuario para ofrecer resultados más relevantes y personalizados.

Sin embargo, al igual que ebay o Amazon, Walmart no incluye un chatbot con el que los usuarios puedan interactuar en lenguaje natural, lo que podría ser una área de mejora para futuras actualizaciones del sistema.

2.3 LA DEFINICIÓN DEL PROBLEMA

En el ámbito del comercio electrónico, los usuarios buscan una experiencia de compra rápida, eficiente y personalizada. Sin embargo, los motores de búsqueda tradicionales, basados en palabras clave, presentan limitaciones significativas en términos de comprensión semántica y relevancia de los resultados. Estas limitaciones pueden generar frustración en los consumidores, quienes dedican tiempo a realizar múltiples búsquedas y filtrar manualmente los resultados para encontrar productos que se ajusten a sus necesidades.

El problema central radica en la falta de herramientas integrales que permitan a los usuarios interactuar de manera intuitiva con las plataformas de comercio electrónico mediante lenguaje natural, mientras proporcionan a los administradores capacidades avanzadas de análisis. Aunque los motores de búsqueda de Amazon, eBay y Walmart han avanzado en personalización, ninguno incorpora un chatbot completo que combine: (1) respuestas contextualizadas en tiempo real mediante IA conversacional, (2) procesamiento semántico avanzado. Esta carencia representa una oportunidad para transformar la experiencia del usuario ahorrando tiempo en búsquedas mientras se dota a los gestores de herramientas de business intelligence basadas en datos de interacción en tiempo real.

El objetivo de este proyecto es abordar estas limitaciones mediante la creación de un sistema que combine un motor de búsqueda eficiente con un chatbot basado en inteligencia artificial. Este sistema permitirá a los usuarios realizar consultas en lenguaje natural, recibiendo respuestas precisas y relevantes que se adapten a sus necesidades específicas. Además, el motor de búsqueda estará optimizado para procesar grandes volúmenes de datos en tiempo real, garantizando resultados actualizados y personalizados.

En resumen, este proyecto busca resolver tres problemas principales:

- **Ahorrar tiempo a los consumidores:** Proporcionar una herramienta que guíe a los usuarios en la búsqueda de productos de manera rápida y eficiente, eliminando la necesidad de realizar múltiples consultas y filtrados manuales.
- **Construir un motor de búsqueda eficiente:** Desarrollar un sistema que combine tecnologías avanzadas de recuperación de información, como la búsqueda semántica y el análisis de datos en tiempo real, para ofrecer resultados precisos y personalizados.
- **Proporcionar a los administradores capacidades analíticas** avanzadas para la toma de decisiones que puedan usarse mediante lenguaje natural

Al integrar un chatbot con capacidades de lenguaje natural y un motor de búsqueda optimizado, este proyecto mejorará la experiencia del usuario y servirá como base para plataformas de comercio electrónico.

2.4 MARCO TEÓRICO

Para abordar el problema planteado y desarrollar una solución que integre búsqueda semántica y capacidades conversacionales, es necesario comprender una serie de conceptos clave relacionados con el procesamiento del lenguaje natural y la recuperación de información avanzada. En esta sección se definen y explican las tecnologías fundamentales que sustentan el sistema propuesto: los modelos de lenguaje de gran escala (LLMs), los embeddings semánticos, las bases de datos vectoriales y las llamadas a funciones (function calling), todos ellos pilares esenciales de la inteligencia artificial moderna aplicada al comercio electrónico.

2.4.1 MODELOS DE LENGUAJE DE GRAN ESCALA (LLM)

Los Modelos de Lenguaje de Gran Escala (Large Language Models, o LLMs) son sistemas de inteligencia artificial entrenados con enormes volúmenes de datos textuales para comprender, generar y razonar sobre lenguaje natural. Estos modelos, como GPT-4, Claude o Gemini, han revolucionado el campo del procesamiento del lenguaje natural (NLP),

permitiendo tareas como traducción automática, generación de texto, resumen, clasificación de sentimientos o interacción conversacional [\[37\]\[5\]](#).

La potencia de los LLMs radica en su arquitectura basada en transformers, un tipo de red neuronal que permite modelar relaciones complejas entre palabras, frases y conceptos, incluso cuando no se presentan de forma secuencial [\[54\]](#). Una red neuronal es un modelo computacional inspirado en el cerebro humano, compuesto por capas de nodos interconectados (neuronas artificiales), que transforman datos de entrada a través de conexiones ponderadas. Cada una de estas conexiones tiene un peso que se ajusta durante el proceso de entrenamiento, con el objetivo de minimizar el error y mejorar la precisión del modelo [\[36\]](#).

En el contexto de este proyecto, los LLMs se utilizan como base para implementar un chatbot conversacional capaz de interpretar consultas complejas, acceder a información contextual almacenada y devolver respuestas personalizadas. Para lograr esto de forma efectiva, es necesario complementar el modelo con mecanismos externos de recuperación de información, como embeddings o llamadas a funciones externas, lo cual se detalla en los siguientes apartados.

2.4.2 FUNCTION CALLING EN LLMS

El mecanismo de Function Calling en LLMs permite a un modelo no solo generar texto, sino también invocar funciones externas de forma estructurada. Esta técnica es esencial cuando el modelo debe interactuar con sistemas reales, como APIs, bases de datos o motores de búsqueda, para proporcionar respuestas precisas y actualizadas.

En lugar de responder directamente, el modelo puede devolver una estructura JSON que describe la función a llamar (nombre y argumentos). Esta llamada es procesada por el sistema externo (por ejemplo, una API REST), que ejecuta la lógica correspondiente y devuelve el resultado al modelo, el cual lo integra en su respuesta final al usuario.

Este enfoque permite separar la inteligencia conversacional (interpretación del lenguaje y generación de prompts) de la lógica de negocio, permitiendo sistemas más fiables,

auditables y seguros. En este proyecto, se utiliza el function calling para que el modelo pueda recuperar reseñas, realizar búsquedas semánticas, generar gráficos o acceder a datos específicos del producto solicitados por el usuario, entre otras funciones.

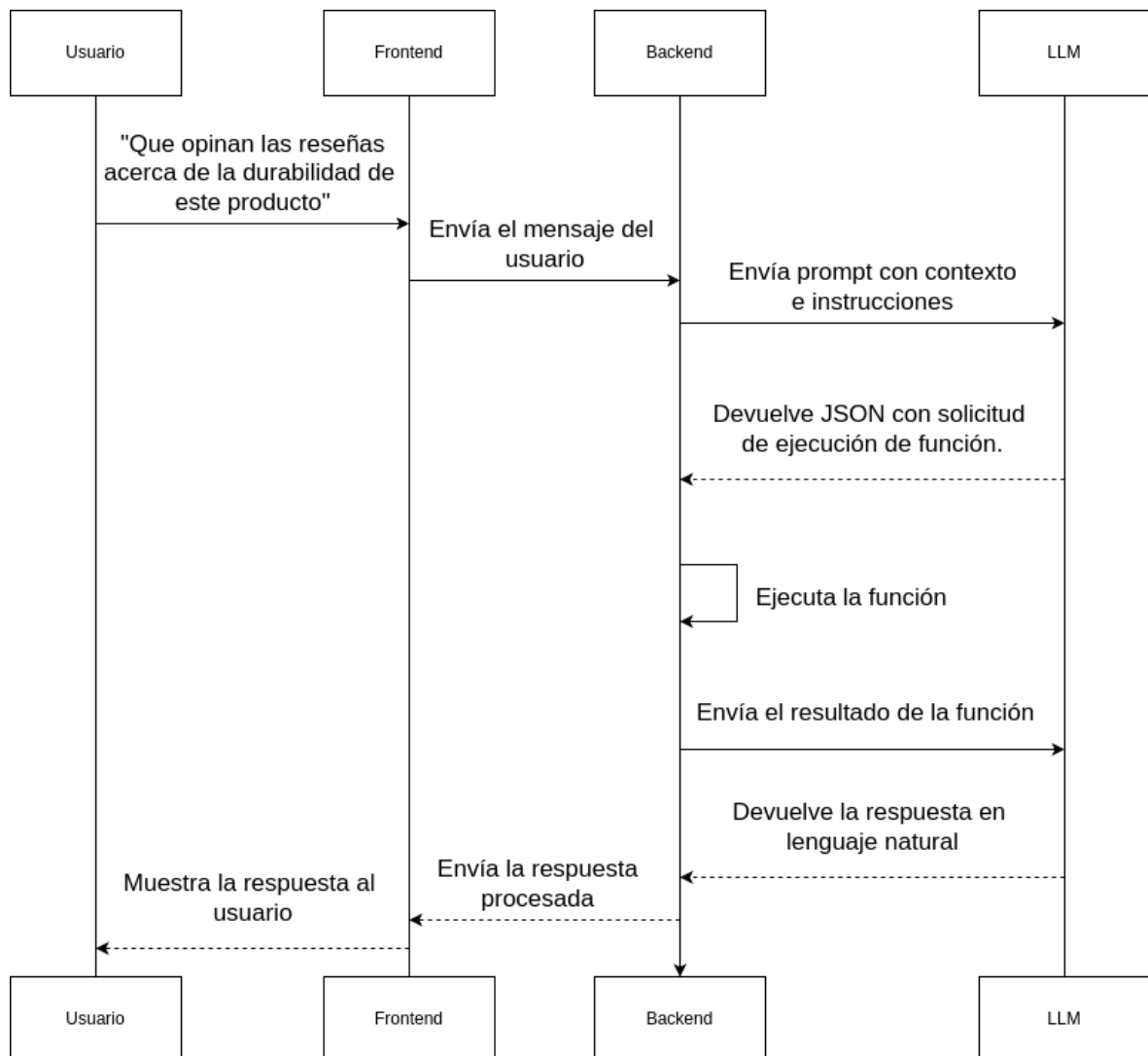


Figura 2.1 Diagrama de interacción con Function Calling

2.4.3 EMBEDDINGS SEMÁNTICOS

Los vectores, en el contexto de la ciencia de datos y el machine learning, son listas ordenadas de números que representan información compleja en un espacio multidimensional. Este proceso se conoce como vectorización o embedding, y permite representar texto, imágenes u otros datos en un formato que puede ser procesado matemáticamente. Por ejemplo, una frase como “el producto llegó rápido y en buen estado” puede convertirse en un vector numérico que captura su significado.

Estos vectores son fundamentales en tareas como búsqueda semántica o recomendación de contenido, ya que permiten comparar el significado de diferentes fragmentos de texto a través de operaciones como la distancia euclidiana o el coseno de similitud.

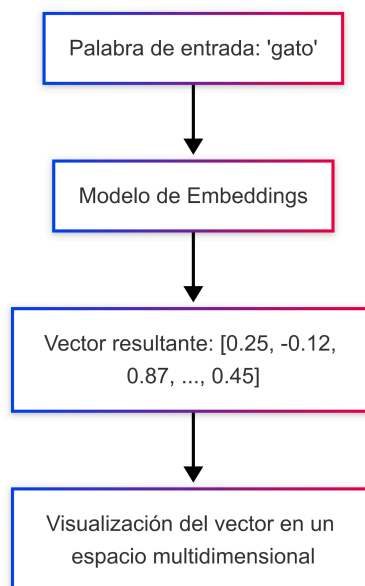


Figura 2.2 Conversión de texto a Embedding

2.4.4 BASES DE DATOS VECTORIALES

Una base de datos vectorial, también conocida como motor de búsqueda de similitud vectorial, es un sistema optimizado para almacenar y recuperar vectores en espacios de alta dimensionalidad. A diferencia de las bases de datos relacionales tradicionales, que organizan los datos en filas y columnas, las bases vectoriales representan la información como puntos en un espacio n-dimensional. Esta representación permite realizar búsquedas basadas en la similitud semántica de los datos, en lugar de buscar coincidencias exactas de palabras clave.

Esta capacidad es especialmente útil en tareas como la recuperación de reseñas o productos relacionados. Por ejemplo, cuando un usuario introduce una consulta como “productos resistentes para montaña”, esta se convierte en un vector de consulta (query embedding). Paralelamente, todas las reseñas o descripciones de producto han sido previamente transformadas en embeddings durante el proceso de indexación. La base de datos vectorial compara el vector de la consulta con los vectores almacenados, recuperando aquellos que se encuentren más cercanos en el espacio vectorial.

La cercanía entre vectores se evalúa utilizando métricas como el coseno de similitud, que mide el ángulo entre dos vectores. Cuanto más pequeño es el ángulo, mayor es la similitud semántica entre los textos representados.

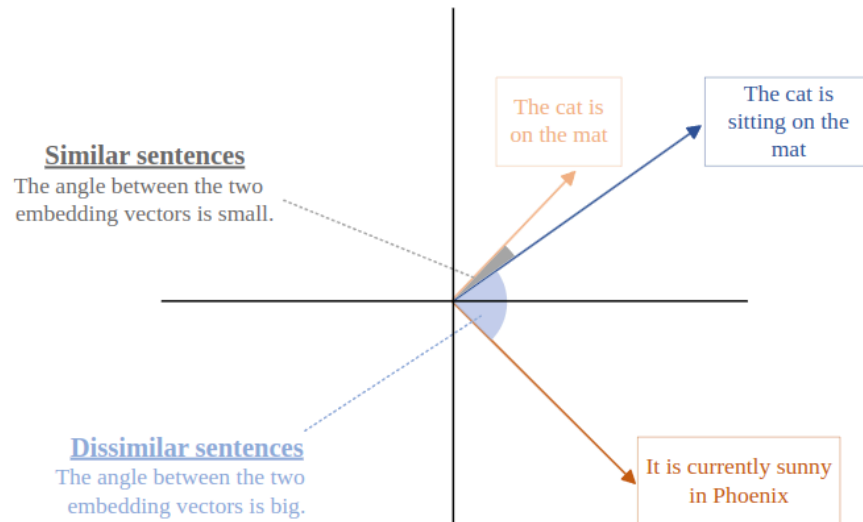


Figura 2.3 Comparación de similitud entre embeddings de frases mediante ángulos vectoriales [31].

Este enfoque permite recuperar, por ejemplo, reseñas que no contienen exactamente las mismas palabras que la consulta del usuario, pero que expresan ideas o experiencias similares. Así, si un usuario pregunta “¿es cómodo para largas caminatas?”, el sistema podría recuperar una reseña que diga “lo usé durante una excursión de 10 km y no tuve molestias”, aunque no haya una coincidencia literal entre los términos utilizados.

En este proyecto, esta lógica se implementa tanto para potenciar la interacción del chatbot como para optimizar la búsqueda de productos. Las alternativas se describen en la sección [1.3](#) (Tecnologías Utilizadas), donde se comparan las bases de datos candidatas según su capacidad para gestionar vectores y realizar búsquedas aproximadas de manera eficiente.

3 GESTIÓN DEL PROYECTO SOFTWARE

Esta sección describe el enfoque del proyecto desde una perspectiva empresarial, detallando su estructura, los objetivos necesarios para su implementación y la metodología empleada para gestionar los recursos y costes asociados. Asimismo, aborda la planificación temporal, asegurando el éxito del despliegue.

3.1 ALCANCE DEL PROYECTO

3.1.1 DEFINICIÓN DEL PROYECTO

El proyecto se estructura en tres componentes principales que operan de forma integrada para ofrecer una solución completa de búsqueda inteligente y gestión de datos:

Backend

El primer componente corresponde al backend, que actúa como el núcleo funcional del sistema. Este módulo se encarga de gestionar toda la interacción con la base de datos, incluyendo el almacenamiento de información de productos y reseñas, así como su transformación en representaciones vectoriales que permiten búsquedas semánticas avanzadas. Además, integra llamadas a modelos de inteligencia artificial para procesar consultas de los usuarios, proporcionando a los modelos de lenguaje acceso estructurado a los datos almacenados mediante funciones y permitiendo así que generen respuestas contextualizadas y precisas. El backend recibe las solicitudes del usuario desde el frontend, las procesa combinando recuperación de información y generación de respuestas, y devuelve los resultados listos para ser mostrados.

Frontend

Este subproyecto ofrece a los usuarios una interfaz web completa e interactiva. La aplicación presenta un menú principal desde donde realizar búsquedas generales de

productos, ya sea mediante términos específicos o a través de interacciones conversacionales. Cada producto cuenta con una página detallada donde consultar sus características técnicas, precios y reseñas de otros compradores. Los usuarios autenticados pueden crear y gestionar listas de deseos personalizadas, añadiendo o eliminando productos según sus preferencias. La interfaz está diseñada para ofrecer una navegación intuitiva, mostrando en todo momento opciones relevantes y permitiendo un acceso rápido a las funcionalidades principales.

Kafka

Kafka en este sistema actúa exclusivamente como canal de transporte para los datos de productos. Un producer lee los archivos fuente (JSON) y los envía a un topic específico tras procesarlos. Por otro lado, un consumer independiente recupera estos mensajes del topic y los reenvía al backend mediante llamadas API REST. Es importante destacar que Kafka nunca interactúa directamente con la base de datos - esta comunicación sólo ocurre a través del backend, que recibe las peticiones del consumer, aplica la lógica de negocio necesaria y finalmente persiste los datos en PostgreSQL. Esta separación clara de responsabilidades garantiza que: (1) Kafka solo maneje el flujo de mensajes, (2) el backend mantenga el control absoluto sobre las operaciones de base de datos, y (3) el sistema pueda escalar horizontalmente añadiendo más instancias del consumer sin afectar la integridad de los datos almacenados.

El sistema utiliza Kafka y un backend con inteligencia artificial para procesar consultas y gestionar una base de datos PostgreSQL. Kafka se encarga de la ingesta inicial de datos a partir de archivos, enviando los productos al backend, que los procesa y almacena. Esta arquitectura permite una carga eficiente, escalabilidad y una separación clara de responsabilidades entre la ingesta de datos y su procesamiento.

3.1.2 OBJETIVOS

Los objetivos del proyecto se definen en función de su alcance general, centrándose en la implementación de un sistema completo para búsqueda y recomendación de productos mediante inteligencia artificial. Así, se pueden definir como la implementación de:

- Funcionalidades básicas de registro y gestión de usuarios
- Sistema de guardado de productos
- Desarrollar módulo de almacenamiento y búsqueda semántica de productos
- Crear interfaz web interactiva con capacidades de búsqueda conversacional
- Compatibilidad con los principales navegadores web
- Integración con servicios externos de inteligencia artificial
- Generar reportes analíticos y visualizaciones de datos
- Proporcionar herramientas de administración y monitoreo
- Implementar pipeline de datos escalable con Kafka
- Interfaz de programación (API) para gestionar todas las operaciones del sistema
- Sistema de procesamiento y almacenamiento de datos escalable

3.2 PLAN DE TRABAJO

En esta parte se detalla la planificación temporal del proyecto, estructurada en etapas clave y visualizada mediante un diagrama de Gantt elaborado con Google Sheets. Este permite observar la duración, fechas de inicio y fin de cada actividad, así como su distribución en el calendario global.

El desarrollo se organiza en tres grandes bloques, usando un enfoque ágil y adaptativo:

- **Fase inicial:** Incluye reuniones de análisis con el cliente para identificar necesidades, definir requisitos técnicos y funcionales, y evaluar tecnologías adecuadas. Se establecen también los plazos y el presupuesto inicial.
- **Desarrollo iterativo:** Las tareas se dividen en sprints de una y dos semanas, generando entregables parciales con funcionalidades progresivas. Las iteraciones

bisemanales se indicarán con la nomenclatura Bx, donde "x" representa el número de la iteración (por ejemplo, B1, B2, etc.). Tras cada iteración, se revisan los avances con el cliente para incorporar feedback y ajustar prioridades, asegurando que los entregables parciales reflejen un desarrollo progresivo de las funcionalidades.

- **Pruebas y cierre:** Se ejecutan pruebas integrales para garantizar la calidad del sistema, corrigiendo errores y verificando el cumplimiento de los requisitos. Finalmente, se realiza la entrega formal al cliente, dejando abierta la opción de soporte post-implementación.

3.2.1 IDENTIFICACIÓN DE TAREAS

Cada una de las fases descritas anteriormente se compone de un conjunto de tareas específicas. A continuación, se desglosan en detalle las actividades correspondientes a cada etapa, junto con una explicación clara de su propósito y desarrollo.

3.2.1.1 INICIO DEL PROYECTO

Durante esta fase inicial del proyecto, se establece un plan de comunicación entre el equipo y el cliente para garantizar una alineación constante, mientras se definen los hitos principales y los entregables críticos. Además, se revisan las capacidades técnicas y los recursos necesarios, identificando posibles limitaciones desde el inicio para evitar riesgos futuros en el desarrollo.

3.2.1.2 PROCESO ITERATIVO (E INCREMENTAL)

El proyecto sigue un desarrollo mediante iteraciones semanales estructuradas que permiten un avance progresivo y controlado. Las iteraciones bisemanales se indican con la nomenclatura **Bx** (por ejemplo, B1, B2, etc.).

Iteración 1: Se diseña el esquema de base de datos para el sistema de recomendaciones, sentando las bases del desarrollo. A continuación, se completa la definición del contrato

API con los endpoints esenciales para las operaciones básicas del sistema, formalizando las estructuras de datos en formato JSON y documentándolas mediante OpenAPI/Swagger para garantizar la consistencia en las comunicaciones.

Iteración 2: Se configura e implementa el backend inicial utilizando Flask y PostgreSQL en entorno Docker. Se desarrollan los primeros endpoints funcionales para inserción y recuperación de datos, validando la conexión con la base de datos.

Iteración 3: El enfoque se traslada al frontend con la configuración del entorno React y creación de la estructura base del proyecto. Se implementan las vistas iniciales para visualización de productos con sus características principales.

Iteración 4: Esta iteración ocupará el tiempo de dos iteraciones semanales debido a su complejidad. Se integra el chatbot implementando la arquitectura RAG con GPT-4 como modelo base. Se configuran los sistemas de embeddings y búsqueda semántica utilizando pgvector en PostgreSQL para procesar las consultas de usuarios.

Fase de expansión (iteraciones bisemanales)

Iteración B1: Se comienza a desarrollar el LLM como agente autónomo, ampliando su funcionalidad más allá del sistema RAG inicial. Implementa las primeras capacidades de toma de decisiones y procesamiento autónomo.

Iteración B2: Se inicia la creación de un motor de búsqueda basado en embeddings avanzados para recuperación de características específicas. Paralelamente, se continúa expandiendo las capacidades del LLM como agente, mejorando su capacidad de comprensión contextual y generación de respuestas más precisas.

Iteración B3: Se finaliza el desarrollo del motor de búsqueda semántica, realizando ajustes finos en los algoritmos de recuperación de información. Se llevan a cabo pruebas para validar la precisión en la recuperación de características y la relevancia de los resultados obtenidos.

Iteración B4: El foco se traslada a mejoras en el frontend: se rediseña la página de menú de productos, se optimiza la interfaz del chat y se enriquece la página de detalles del producto. Se implementa la funcionalidad de selección de LLM.

Iteración B5: Se completa la mejora del frontend con la implementación final del selector de LLM y se optimizan todas las interfaces de usuario. Se añaden características avanzadas de visualización y se perfecciona la experiencia de usuario en todas las pantallas principales.

Iteración B6: Se desarrolla un bot administrador para gestionar el LLM, moderar contenido y ajustar el motor de búsqueda, junto con un dashboard que incluye gráficas interactivas (líneas, barras, mapas de calor) para métricas en tiempo real de uso, precisión y tendencias de productos.

Iteración B7: Se implementa el sistema Kafka para consumo automatizado de productos con reviews desde el dataset externo. Se desarrollan los pipelines necesarios para procesar y almacenar esta información en la base de datos, estableciendo los mecanismos de actualización periódica.

Iteración B8: Se completaron los gráficos generados por el LLM (líneas, barras, mapas de calor), se optimizó el código del chatbot y se implementó la localización de usuarios por IP para personalización regional.

3.2.1.3 VALIDACIÓN Y ENTREGA

Para garantizar la calidad del producto final, se realizan pruebas en las que el cliente prueba la aplicación en un entorno real, con el desarrollador presente para resolver cualquier incidencia. Tras corregir los errores, se revisan los requisitos iniciales con el cliente y se optimizan las funcionalidades antes de la entrega definitiva. Este proceso asegura que el software cumple con las expectativas y funciona correctamente en condiciones reales de uso.

3.2.2 ESTIMACIÓN DE TAREAS

La planificación temporal de las tareas del proyecto puede presentar variaciones debido a factores como la complejidad o la experiencia en ciertas tecnologías. Las estimaciones proporcionadas deben tomarse como referencia, ya que imprevistos o requerimientos adicionales pueden modificar los plazos iniciales. Cada día de trabajo se calcula en base a una jornada estándar de 8 horas, aunque algunas tareas podrían requerir más o menos tiempo según su naturaleza.

TIPO DE TAREA	DURACIÓN
Reuniones iniciales y toma de requisitos	2 días
Investigación estado del arte	5 días
Investigación del problema	2 días
Análisis y selección de tecnologías	2 días
Instalación y preparación del entorno	1 día
Implementación de requisitos (iterativa)	103 días
Documentación	1-5 días
Pruebas iterativas	1-3 días
Corrección de errores	1-2 días
Pruebas validación	3-4 días

Tabla 3.1 Estimación temporal de tareas

3.2.3 PLANIFICACIÓN DE TAREAS

La gestión de las actividades de este proyecto se ha organizado mediante la herramienta *Google Sheets* en la cual se han realizado diagramas de Gantt. El proyecto se ha segmentado en diferentes etapas.

El inicio del proyecto está fijado para el 2 de diciembre de 2024, mientras que el cierre está programado para el 30 de Mayo de 2025. Esto da como resultado una duración aproximada de 121 días laborables y 180 días naturales, considerando los festivos nacionales y regionales en España [13]. En cuanto al esfuerzo requerido, se han estimado unas 968 horas de trabajo, una cantidad razonable para alcanzar los objetivos planteados en este plazo.

Las tareas se han ordenado de forma secuencial, de modo que el comienzo de cada etapa depende de la conclusión de la anterior.

TAREA	DURACIÓN	INICIO	FIN
Plan inicial	12	2-12-24	19-12-24
Proceso iterativo e incremental	103	20-12-2024	22-05-2025
Validación y entrega	6	23-05-25	31-05-25

Tabla 3.2 Planificación general del proyecto

PLAN INICIAL

La duración de la fase inicial del proyecto fue de 12 días hábiles, comprendida entre el 2 de diciembre de 2024 y el 19 de diciembre de 2024. Las subtareas se organizaron de acuerdo con el diagrama de Gantt correspondiente.

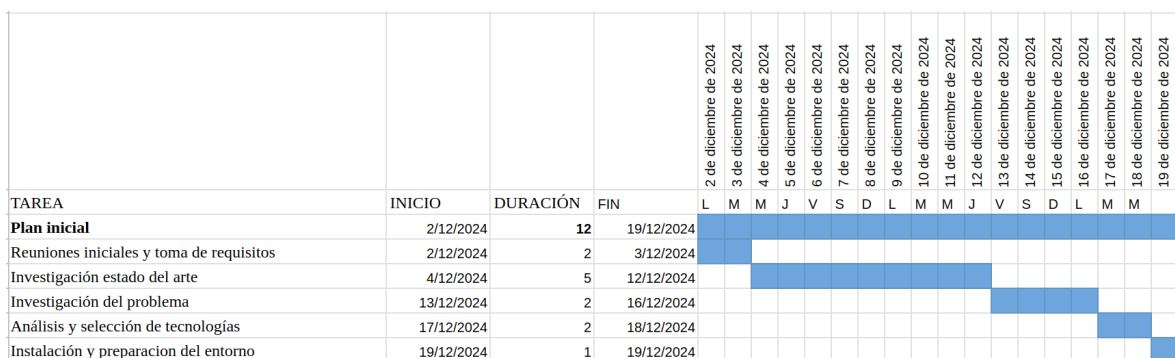


Figura 3.1 Diagrama de Gantt Plan inicial

PROCESO ITERATIVO E INCREMENTAL

En un proceso iterativo e incremental, el producto se desarrolló a lo largo de 13 iteraciones. Inició el 19 de diciembre de 2024 y finalizó el 22 de mayo de 2025, con una duración total de 104 días hábiles.

TAREA	DURACIÓN	INICIO	FIN
Plan inicial	12	2-12-24	19-12-24
Proceso iterativo e incremental	103	20-12-2024	22-05-2025
Iteración 1	5	20-12-2024	29-12-2024
Iteración 2	5	30-12-2024	07-01-2025
Iteración 3	5	08-01-2025	14-01-2025
Iteración 4	10	15-01-2025	28-01-2025
Iteración B1	10	29-01-2025	11-02-2025
Iteración B2	10	12-02-2025	25-02-2025
Iteración B3	10	26-02-2025	11-03-2025
Iteración B4	10	12-03-2025	25-03-2025
Iteración B5	10	26-03-2025	08-04-2025
Iteración B6	10	09-04-2025	27-04-2025
Iteración B7	10	28-04-2025	12-05-2025
Iteración B8	10	12-05-2025	22-05-2025
Validación y entrega	6	23-05-25	31-05-25

Tabla 3.3 Planificación de Proceso Iterativo

Las subtarefas de cada iteración, junto con su duración, se detallan en los apartados siguientes.

- **ITERACIÓN 1:**

				20 de diciembre de 2024	21 de diciembre de 2024	22 de diciembre de 2024	23 de diciembre de 2024	24 de diciembre de 2024	25 de diciembre de 2024	26 de diciembre de 2024	27 de diciembre de 2024
TAREA	INICIO	DURACIÓN	FIN	V	S	D	L	M	M	J	V
Iteración 1	20/12/2024	5	29/12/2024								
Diseñar el esquema de base de datos	20/12/2024	2	23/12/2024								
Completar la definición del contrato API	24/12/2024	1	25/12/2024								
Formalizar las estructuras de datos en formato JSON	26/12/2024	1	26/12/2024								
Pruebas	27/12/2024	1	29/12/2024								

Figura 3.2 Diagrama de Gantt Plan iteración 1

- **ITERACIÓN 2:**

				30 de diciembre de 2024	31 de diciembre de 2024	1 de enero de 2025	2 de enero de 2025	3 de enero de 2025	4 de enero de 2025	5 de enero de 2025	6 de enero de 2025	7 de enero de 2025
TAREA	INICIO	DURACIÓN	FIN	L	M	M	J	V	S	D	L	M
Iteración 2	30/12/2024	5	7/1/2025									
Configurar el entorno Docker para Flask y PostgreSQL	30/12/2024	1	30/12/2024									
Implementar el backend inicial	31/12/2024	1	1/01/2025									
Desarrollar endpoints para inserción de datos	2/01/2025	1	2/01/2025									
Desarrollar endpoints para recuperación de datos	3/01/2025	1	6/01/2025									
Realizar pruebas	7/01/2025	1	7/01/2025									

Figura 3.3 Diagrama de Gantt Plan iteración 2

- **ITERACIÓN 3:**

				8 de enero de 2025	9 de enero de 2025	10 de enero de 2025	11 de enero de 2025	12 de enero de 2025	13 de enero de 2025	14 de enero de 2025
TAREA	INICIO	DURACIÓN	FIN	M	J	V	S	D	L	M
Iteración 3	8/01/2025	5	14/1/2025							
Configurar el entorno React	8/01/2025	1	8/01/2025							
Crear la estructura base del proyecto	9/01/2025	1	9/01/2025							
Implementar vistas iniciales para visualización de productos	10/01/2025	2	13/01/2025							
Realizar pruebas	14/01/2025	1	14/01/2025							

Figura 3.4 Diagrama de Gantt Plan iteración 3

- **ITERACIÓN 4:**

				15 de enero de 2025	16 de enero de 2025	17 de enero de 2025	18 de enero de 2025	19 de enero de 2025	20 de enero de 2025	21 de enero de 2025	22 de enero de 2025	23 de enero de 2025	24 de enero de 2025	25 de enero de 2025	26 de enero de 2025	27 de enero de 2025	28 de enero de 2025
TAREA	INICIO	DURACIÓN	FIN	M	J	V	S	D	L	M	M	J	V	S	D	L	M
Iteración 4	15/01/2025	10	28/1/2025														
Implementar la arquitectura RAG con GPT-4	15/01/2025	4	20/01/2025														
Configurar sistemas de embeddings y búsqueda semántica con pgvector	21/01/2025	4	26/01/2025														
Realizar pruebas	27/01/2025	2	28/01/2025														

Figura 3.5 Diagrama de Gantt Plan iteración 4

- **ITERACIÓN B1:**

				29 de enero de 2025	30 de enero de 2025	31 de enero de 2025	1 de febrero de 2025	2 de febrero de 2025	3 de febrero de 2025	4 de febrero de 2025	5 de febrero de 2025	6 de febrero de 2025	7 de febrero de 2025	8 de febrero de 2025	9 de febrero de 2025	10 de febrero de 2025	11 de febrero de 2025
TAREA	INICIO	DURACIÓN	FIN	M	J	V	S	D	L	M	M	J	V	S	D	L	M
Iteración B1	29/01/2025	10	11/2/2025														
Diseñar la estructura del LLM como agente autónomo	29/01/2025	3	2/02/2025														
Implementar lógica para toma de decisiones independientes	3/02/2025	5	9/02/2025														
Realizar pruebas	10/02/2025	2	11/02/2025														

Figura 3.6 Diagrama de Gantt Plan iteración B1

- **ITERACIÓN B2:**

				12 de febrero de 2025	13 de febrero de 2025	14 de febrero de 2025	15 de febrero de 2025	16 de febrero de 2025	17 de febrero de 2025	18 de febrero de 2025	19 de febrero de 2025	20 de febrero de 2025	21 de febrero de 2025	22 de febrero de 2025	23 de febrero de 2025	24 de febrero de 2025	25 de febrero de 2025
TAREA	INICIO	DURACIÓN	FIN	M	J	V	S	D	L	M	M	J	V	S	D	L	M
Iteración B2	12/02/2025	10	25/2/2025														
Desarrollar el código del motor de búsqueda con embeddings	12/02/2025	3	16/02/2025														
Crear funciones para buscar características específicas	17/02/2025	2	18/02/2025														
Mejorar el código de comprensión del LLM	19/02/2025	2	20/02/2025														
Ajuste de las respuestas del agente	21/02/2025	1	23/02/2025														
Realizar pruebas	24/02/2025	2	25/02/2025														

Figura 3.7 Diagrama de Gantt Plan iteración B2

- **ITERACIÓN B3:**

				26 de febrero de 2025	27 de febrero de 2025	28 de febrero de 2025	1 de marzo de 2025	2 de marzo de 2025	3 de marzo de 2025	4 de marzo de 2025	5 de marzo de 2025	6 de marzo de 2025	7 de marzo de 2025	8 de marzo de 2025	9 de marzo de 2025	10 de marzo de 2025	11 de marzo de 2025
TAREA	INICIO	DURACIÓN	FIN	M	J	V	S	D	L	M	M	J	V	S	D	L	M
Iteración B3	26/02/2025	10	11/3/2025														
Terminar el código del motor de búsqueda semántica	26/02/2025	3	2/03/2025														
Hacer pequeños cambios en el código de recuperación de información	3/03/2025	2	4/03/2025														
Preparar ejemplos para comprobar la precisión de los resultados	5/03/2025	3	9/03/2025														
Realizar pruebas	10/03/2025	2	11/03/2025														

Figura 3.8 Diagrama de Gantt Plan iteración B3

- **ITERACIÓN B4:**

				12 de marzo de 2025	13 de marzo de 2025	14 de marzo de 2025	15 de marzo de 2025	16 de marzo de 2025	17 de marzo de 2025	18 de marzo de 2025	19 de marzo de 2025	20 de marzo de 2025	21 de marzo de 2025	22 de marzo de 2025	23 de marzo de 2025	24 de marzo de 2025	25 de marzo de 2025
TAREA	INICIO	DURACIÓN	FIN	M	J	V	S	D	L	M	M	J	V	S	D	L	M
Iteración B4	12/03/2025	10	25/3/2025														
Rediseñar el código de la página de menú de productos	12/03/2025	2	13/03/2025														
Mejorar el código de la interfaz del chat	14/03/2025	2	17/03/2025														
Agregar más detalles al código de la página de productos	18/03/2025	2	19/03/2025														
Desarrollar el código para que el usuario elija LLM	20/03/2025	2	23/03/2025														
Realizar pruebas	24/03/2025	2	25/03/2025														

Figura 3.9 Diagrama de Gantt Plan iteración B4

- **ITERACIÓN B5:**

				26 de marzo de 2025	27 de marzo de 2025	28 de marzo de 2025	29 de marzo de 2025	30 de marzo de 2025	31 de marzo de 2025	1 de abril de 2025	2 de abril de 2025	3 de abril de 2025	4 de abril de 2025	5 de abril de 2025	6 de abril de 2025	7 de abril de 2025	8 de abril de 2025
TAREA	INICIO	DURACIÓN	FIN	M	J	V	S	D	L	M	M	J	V	S	D	L	M
Iteración B5	26/03/2025	10	8/4/2025														
Terminar el código del selector de LLM	26/03/2025	3	30/03/2025														
Mejorar el código y optimizar las funcionalidades del frontend	31/03/2025	5	6/04/2025														
Realizar pruebas	7/04/2025	2	8/04/2025														

Figura 3.10 Diagrama de Gantt Plan iteración B5

- **ITERACIÓN B6:**

TAREA	INICIO	DURACIÓN	FIN	9 de abril de 2025	10 de abril de 2025	11 de abril de 2025	12 de abril de 2025	13 de abril de 2025	14 de abril de 2025	15 de abril de 2025	16 de abril de 2025	17 de abril de 2025	18 de abril de 2025	19 de abril de 2025	20 de abril de 2025	21 de abril de 2025	22 de abril de 2025	23 de abril de 2025	24 de abril de 2025	25 de abril de 2025	26 de abril de 2025	27 de abril de 2025
				M	J	V	S	D	L	M	M	J	V	S	D	L	M	M	J	V	S	D
Iteración B6	9/04/2025	10	27/4/2025																			
Definir las funciones y las gráficas a mostrar	9/04/2025	3	13/04/2025																			
Desarrollar el código de las funciones	14/04/2025	2	15/04/2025																			
Crear el código del dashboard con gráficas interactivas	16/04/2025	3	23/04/2025																			
Realizar pruebas	24/04/2025	2	27/04/2025																			

Figura 3.11 Diagrama de Gantt Plan iteración B6

- **ITERACIÓN B7:**

TAREA	INICIO	DURACIÓN	FIN	28 de abril de 2025	29 de abril de 2025	30 de abril de 2025	1 de mayo de 2025	2 de mayo de 2025	3 de mayo de 2025	4 de mayo de 2025	5 de mayo de 2025	6 de mayo de 2025	7 de mayo de 2025	8 de mayo de 2025	9 de mayo de 2025	10 de mayo de 2025	11 de mayo de 2025	12 de mayo de 2025
				L	M	M	J	V	S	D	L	M	M	J	V	S	D	L
Iteración B7	28/04/2025	10	12/5/2025															
Configurar el producer en Kafka para enviar datos de la API	28/04/2025	3	1/05/2025															
Desarrollar el consumer para procesar los datos recibidos	2/05/2025	3	6/05/2025															
Crear el código para guardar la información en la base de datos	7/05/2025	2	8/05/2025															
Realizar pruebas	9/05/2025	2	12/05/2025															

Figura 3.12 Diagrama de Gantt Plan iteración B7

- **ITERACIÓN B8:**

TAREA	INICIO	DURACIÓN	FIN	13 de mayo de 2025	14 de mayo de 2025	15 de mayo de 2025	16 de mayo de 2025	17 de mayo de 2025	18 de mayo de 2025	19 de mayo de 2025	20 de mayo de 2025	21 de mayo de 2025	22 de mayo de 2025
				M	M	J	V	S	D	L	M	M	J
Iteración B8	13/05/2025	8	22/5/2025										
Desarrollar el código para que el LLM pueda generar gráficos	13/05/2025	4	18/05/2025										
Implementar la localización por IP de los usuarios	19/05/2025	2	20/05/2025										
Realizar pruebas	21/05/2025	2	22/05/2025										

Figura 3.13 Diagrama de Gantt Plan iteración B8

VALIDACIÓN Y ENTREGA

La validación y entrega dará comienzo el día 23 de mayo de 2025 y finalizará el 30 de mayo de 2025. Este periodo tendrá una duración de 6 días laborables, durante los cuales se llevará a cabo los procesos correspondientes.

				23 de mayo de 2025	24 de mayo de 2025	25 de mayo de 2025	26 de mayo de 2025	27 de mayo de 2025	28 de mayo de 2025	29 de mayo de 2025	30 de mayo de 2025	31 de mayo de 2025	1 de junio de 2025
TAREA	INICIO	DURACIÓN	FIN	V	S	D	L	M	M	J	V	S	D
Validación y entrega	23/05/2025	6	1/6/2025										
Testeo preliminar del sistema	23/05/2025	2	26/05/2025										
Evaluación de cumplimiento de especificaciones	27/05/2025	2	28/05/2025										
Presentación definitiva al destinatario	29/05/2025	2	1/06/2025										

Figura 3.14 Diagrama de Gantt Plan Validación y entrega final

3.3 GESTIÓN DE RECURSOS

En esta sección se abordará la planificación y asignación de los recursos necesarios para la ejecución del proyecto. A continuación, se detallarán los recursos humanos, hardware y software requeridos, especificando sus roles y funciones dentro del proceso. Posteriormente, estos recursos serán asignados a las tareas correspondientes, haciendo un análisis del coste total.

3.3.1 ESPECIFICACIÓN DE RECURSOS

En un proyecto de desarrollo de software, los recursos humanos constituyen el pilar fundamental para su ejecución, abarcando roles como programadores, diseñadores y gestores de proyecto, entre otros. A continuación, se adjunta una tabla que detalla los recursos humanos necesarios para este proyecto, junto con sus salarios estimados, obtenidos a partir de datos recopilados en la página web Indeed [\[20\]](#), la cual proporciona una aproximación a los sueldos basada en las ofertas de empleo disponibles actualmente, permitiendo así estimar los costes asociados a cada perfil profesional.

Recurso	Número necesario	Salario bruto anual	Salario bruto mensual	Salario bruto por hora
Desarrollador Backend	1	49.961€	3568 €	24,01 €
Desarrollador Frontend	1	45.493 €	3249 €	21,87 €
Administrador de Bases de Datos	1	45.503 €	3250.21 €	21.87 €
Ingeniero de inteligencia artificial	1	33.264 €	2376 €	15,99 €
Gestor de Proyecto	1	46.986 €	3356,14 €	22,58 €

Tabla 3.4 Recursos humanos del proyecto y sus sueldos

Las fórmulas que se han empleado han sido:

$$\text{Salario mensual} = \text{salario anual} / 14 \quad (2.1)$$

$$\text{Salario por hora} = \text{salario anual} / (40 * 52) \quad (2.2)$$

En la fórmula 2.1, el sueldo se distribuye en 14 pagos: 12 correspondientes a los meses del año y 2 adicionales.

En la fórmula 2.2, 40 representa las horas trabajadas por semana, mientras que 52 son las semanas que componen un año típico.

Ahora, para determinar el coste real de cada trabajador para la empresa, se deben tener en cuenta los siguientes aspectos:

- Contingencias comunes: 23,6%
- Prestaciones por desempleo: 5,5%
- Baja por enfermedad o accidentes de trabajo: 3,5%

- Formación profesional: 0,6%
- FOGASA: 0,2%

Recurso	Salario anual	Contingencias	Desempleo	Bajas y accidentes	Formación profesional	FOGASA	Coste anual
Desarrollador Backend	49.961€	11.791€	2.748€	1.749€	300€	100€	66.648€
Desarrollador Frontend	45.493 €	10.736€	2.502€	1.592€	273€	91€	60.688€
Administrador de Bases de Datos	45.503 €	10.739€	2.503€	1.593€	273€	91€	60.701€
Ingeniero de inteligencia artificial	33.264 €	7.850€	1.830€	1.164€	200€	67€	44.374€
Gestor de Proyecto	46.986 €	11.089€	2.584€	1.645€	282€	94€	62.679€

Tabla 3.5 Coste de los RRHH para la empresa

A continuación se indica el coste total de los salarios de los trabajadores para la empresa.

Recurso	Coste anual para la empresa	Coste mensual para la empresa	Coste por hora para la empresa
Desarrollador Backend	66.648€	4761€	31.96€
Desarrollador Frontend	60.688€	4333€	29.18€
Administrador de Bases de Datos	60.701€	4336€	29.18€
Ingeniero de inteligencia artificial	44.374€	3170€	21.03€
Gestor de Proyecto	62.679€	4477€	30.13€

Tabla 3.6 Coste de los RRHH para la empresa con impuestos

Además, se deben considerar los costos de los dispositivos y el software. En la siguiente tabla no se especifica el entorno de desarrollo, ya que se utilizará Visual Studio Code, que es gratuito. La tabla a continuación detalla el costo por unidad de cada recurso, las unidades requeridas, su importe, así como el total de unidades y los importes totales.

Recurso	Coste unitario (€)	Unidades	Coste total (€)
Licencia de Windows 11 Pro	177.83€	5	889.15
Créditos de Openrouter	60,43 (basado en \$68 USD: 2M tokens de entrada a \$2/M y 8M tokens de salida a \$8/M)	1	60,43
Base de Datos (PostgreSQL - AWS)	81,96/mes	1 x 6 meses	491,77
Servicios en la Nube (AWS EC2, S3)	177,85/mes	1 x 6 meses	1.067,11
Entorno de desarrollo (Visual Studio Code)	0	5	0
Herramienta de Gestión de Proyectos (Jira)	7,25/usuario/mes	5 x 6 meses	217,50
Software de Diseño (Figma - Pro)	10,67/usuario/mes	1 x 6 meses	63,85
Total			2.789,61

Tabla 3.7 Coste de recursos software

A continuación se detalla también los costes del hardware necesario para que pueda trabajar el equipo, sin incluir agua luz o calefacción que se incluyen en los costes indirectos :

Recurso	Coste unitario (€)	Unidades	Coste total (€)
Estación de trabajo (PC de alta gama)	2.000,00	5	10.000,00
Monitor 27" 4K	350,00	5	1.750,00
Teclado y ratón ergonómicos	100,00	5	500,00
Total			12.250,00

Tabla 3.8 Coste de recursos hardware

3.3.2 ASIGNACIÓN DE RECURSOS

En esta sección se llevará a cabo la asignación de los recursos mencionados previamente para el proyecto, con el objetivo de estimar un presupuesto final.

En cuanto a los recursos humanos, trabajarán en jornadas laborales de 8 horas diarias. La tabla a continuación detalla las horas totales asignadas a cada recurso, junto con el costo total correspondiente a cada uno:

Recurso	Horas de trabajo	Coste por hora para la empresa (con impuestos)	Coste total
Desarrollador Backend	240	31.96€	7670.4 €
Desarrollador Frontend	224	29.18€	6536.32 €
Administrador de Bases de Datos	160	29.18€	4668.8 €
Ingeniero de inteligencia artificial	240	21.03€	5047.2 €
Gestor de Proyecto	104	30.13€	3133.52 €
Total			27.056,24

Tabla 3.9 Asignación de Recursos Humanos

Por tanto se alcanza un total de 968 horas, equivalentes a 121 jornadas laborales de 8 horas cada una. Estas jornadas abarcan las fases de diseño, desarrollo, implementación de la arquitectura y pruebas del proyecto. El costo total asociado a estas horas trabajadas para la empresa asciende a 27.056,24 EUR.

3.4 PRESUPUESTO

En cuanto al presupuesto asignado para llevar a cabo el proyecto, es necesario considerar los costos detallados en los apartados anteriores. También, se tendrá en cuenta el beneficio industrial que será del 6%, los costes indirectos y el IVA.

Tipo de recurso	Coste (€)
Humanos	27.056,24
Software	2.789,61
Hardware	12.250,00
Beneficio Industrial (6%)	2.533,24
Costes Indirectos (agua, luz...)	3.500,00
IVA (21%)	10.133,30
Total	58.262,09

Tabla 3.10 Costos totales del proyecto

El costo total para el desarrollo del proyecto asciende a **58.262,09** según los cálculos detallados que incluyen los costos de recursos humanos (27.056,24 EUR), software (2.789,61 EUR), hardware (12.250,00 EUR), costos indirectos (3.500,00 EUR), beneficio industrial (2.533,24 EUR) y el IVA al 21% (10.133,30 EUR). Para prever posibles gastos imprevistos o retrasos que generen costos adicionales, se establece un presupuesto final de **60.000,00 EUR**, proporcionando un margen de seguridad.

3.5 LEGISLACIÓN Y NORMATIVA

El proyecto está situado bajo el marco de las siguientes leyes:

- **Ley de Propiedad Intelectual (Real Decreto Legislativo 1/1996)**

El Real Decreto Legislativo 1/1996, de 12 de abril, que aprueba el texto refundido de la Ley de Propiedad Intelectual [\[9\]](#), establece en su artículo 1 que la propiedad intelectual de una obra, ya sea literaria, artística o científica, pertenece al autor por el mero hecho de su creación, incluyendo explícitamente los programas de ordenador en el artículo 10.1, sección “i”. En el contexto de mi aplicación web con integración de inteligencia artificial,

esta ley protege el código fuente, y el diseño de la interfaz como creaciones originales. Para evitar infracciones, se garantizará que todas las bibliotecas, frameworks y recursos de terceros (como los utilizados en el backend o frontend) cuenten con licencias adecuadas (por ejemplo, MIT, Apache). Además, dado que la aplicación procesa datos de usuarios, se implementarán medidas para respetar los derechos morales y de explotación de los autores, asegurando que cualquier contenido generado por la IA no vulnere derechos preexistentes.

- **Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD)**

En cumplimiento de la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales [\[10\]](#), se establece que la protección de los datos personales constituye un derecho fundamental reconocido en el artículo 18.4 de la Constitución Española. Esta ley regula el tratamiento de datos personales con el objetivo de garantizar la privacidad, la transparencia y los derechos digitales de los ciudadanos. En el contexto de esta aplicación, que recoge información como direcciones de correo electrónico, identificadores de GitHub, direcciones IP y registros de inicio de sesión, se aplican los principios del Reglamento General de Protección de Datos (RGPD) y de la LOPDGDD, asegurando el cumplimiento de los derechos ARSULIPO (Acceso, Rectificación, Supresión, Limitación, Portabilidad y Oposición). Todos los datos recopilados serán tratados con medidas técnicas y organizativas adecuadas para garantizar su seguridad y se limitarán exclusivamente al propósito para el que fueron recabados. Asimismo, se proporciona a los usuarios un canal de contacto visible desde la interfaz para el ejercicio de sus derechos o la gestión de incidencias relativas a sus datos personales.

- **Anteproyecto de Ley para el Buen Uso y la Gobernanza de la Inteligencia Artificial**

El Anteproyecto de Ley para el Buen Uso y la Gobernanza de la Inteligencia Artificial, impulsado por el Ministerio para la Transformación Digital [\[32\]](#), representa una iniciativa normativa nacional que busca complementar el Reglamento (UE) 2024/1689, de 13 de junio, sobre inteligencia artificial. Aunque aún no ha sido aprobado como ley, este

anteproyecto marca la dirección que seguirá el marco regulador español en el ámbito de la IA y ya debe ser tenido en cuenta por desarrolladores y operadores de sistemas de inteligencia artificial. Su estatus como anteproyecto implica que no tiene fuerza legal vigente, pero su contenido anticipa obligaciones futuras como la supervisión del uso de IA, la necesidad de trazabilidad y transparencia en los algoritmos, y la posible aplicación de un régimen sancionador en caso de infracciones. En el contexto de esta aplicación, que emplea sistemas de recomendación e IA generativa, es recomendable seguir sus principios para promover un desarrollo ético, seguro y centrado en los derechos fundamentales. Adoptar desde el inicio buenas prácticas alineadas con esta futura legislación no solo facilita la adaptación normativa, sino que contribuye a la confianza del usuario y a la sostenibilidad legal del proyecto.

4 SOLUCIÓN

Esta sección detalla las decisiones adoptadas para abordar el problema planteado en el capítulo 1. Se especificarán los requisitos funcionales y no funcionales, y se describirán en profundidad las fases de análisis, diseño, implementación y pruebas del proceso de desarrollo.

4.1 DESCRIPCIÓN DE LA SOLUCIÓN

La arquitectura desarrollada se basa en la implementación de un sistema integral compuesto por dos subsistemas principales, cada uno orientado a cubrir una función específica.

Subsistema de ingesta y procesamiento de datos

El subsistema de ingesta y procesamiento de datos automatiza el flujo completo desde la carga de datos crudos hasta su almacenamiento final, permitiendo que otras aplicaciones consuman la información sin preocuparse por su preparación previa. Este sistema está orquestado mediante un archivo docker-compose.yml que coordina servicios como Apache Kafka, Zookeeper y PostgreSQL. A continuación, se detallan las etapas principales del proceso:

1. Carga de Datos

El proceso comienza con la ingesta de un dataset (conjunto de datos) en formato JSON que contiene reseñas y metadatos de productos, obtenidos de [\[2\]](#). Un productor de Kafka lee estos archivos y los envía a un topic específico en Kafka. Este enfoque permite manejar grandes volúmenes de datos de manera eficiente y desacoplada, garantizando un flujo continuo desde la fuente externa.

2. Transmisión de Datos

Un consumidor de Kafka independiente recoge los datos del topic correspondiente y los redirige al backend mediante llamadas HTTP a una API REST. Esta etapa asegura que los

datos fluyan de manera ordenada hacia el sistema de procesamiento, manteniendo la integridad y disponibilidad de la información.

3. Procesamiento y Transformación

En el backend, desarrollado con Flask, los datos recibidos son procesados. Durante esta etapa:

- Se generan representaciones vectoriales (embeddings) a partir de los textos de las reseñas, habilitando análisis semánticos avanzados.
- Se realiza la validación de contenidos para garantizar la calidad y consistencia de los datos.
- Los datos se transforman y preparan para su almacenamiento en un formato optimizado.

4. Almacenamiento

El almacenamiento se realiza en una base de datos PostgreSQL con la extensión pgvector, que permite gestionar vectores de alta dimensionalidad. Esta extensión habilita búsquedas semánticas eficientes, ideales para aplicaciones que requieran análisis de similitud o recomendación basadas en los embeddings generados.

El subsistema de ingesta y procesamiento de datos está diseñado de manera desacoplada, lo que permite su reutilización en diversos contextos, como sistemas de recomendación o dashboards, aprovechando los datos ya procesados y almacenados sin necesidad de rehacer el flujo de ingesta y transformación. La estructura del subsistema, dividida en etapas lógicas (carga, transmisión, procesamiento y almacenamiento), garantiza claridad en la comprensión del proceso, resalta la modularidad de cada componente —que puede adaptarse de forma independiente— y ofrece escalabilidad para incorporar detalles técnicos, como la configuración de Kafka o los algoritmos de generación de embeddings, sin perder cohesión.

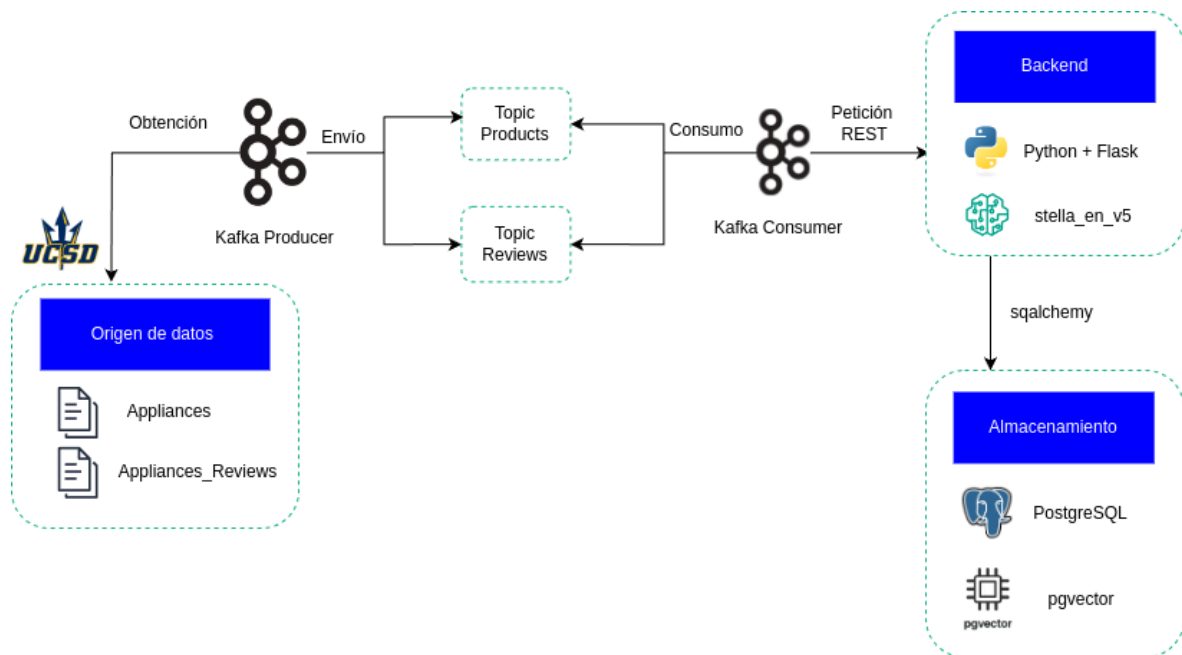


Figura 4.1 Subsistema de ingesta y procesamiento de datos

Aplicación web interactiva

El segundo componente del sistema corresponde a la interfaz de usuario, implementada como una aplicación web dinámica con React. Desde su inicio, el usuario es recibido con una página de bienvenida, a partir de la cual puede acceder a las secciones de inicio de sesión o registro. En función del rol que posea, el usuario será redirigido a diferentes áreas de la plataforma.

Los usuarios con rol estándar acceden a la vista principal de productos, donde pueden explorar el catálogo disponible, realizar búsquedas semánticas a través del chatbot integrado y marcar productos como favoritos. Cada producto cuenta con una página individual que incluye información detallada, enlace directo a Amazon y visualización de sus reseñas. En esta misma vista, el chatbot es capaz de generar respuestas contextualizadas sobre el producto concreto, sus valoraciones y sus reviews. Además, los usuarios disponen de una sección dedicada a gestionar sus productos favoritos, donde pueden seleccionar o deseleccionar elementos a su elección.

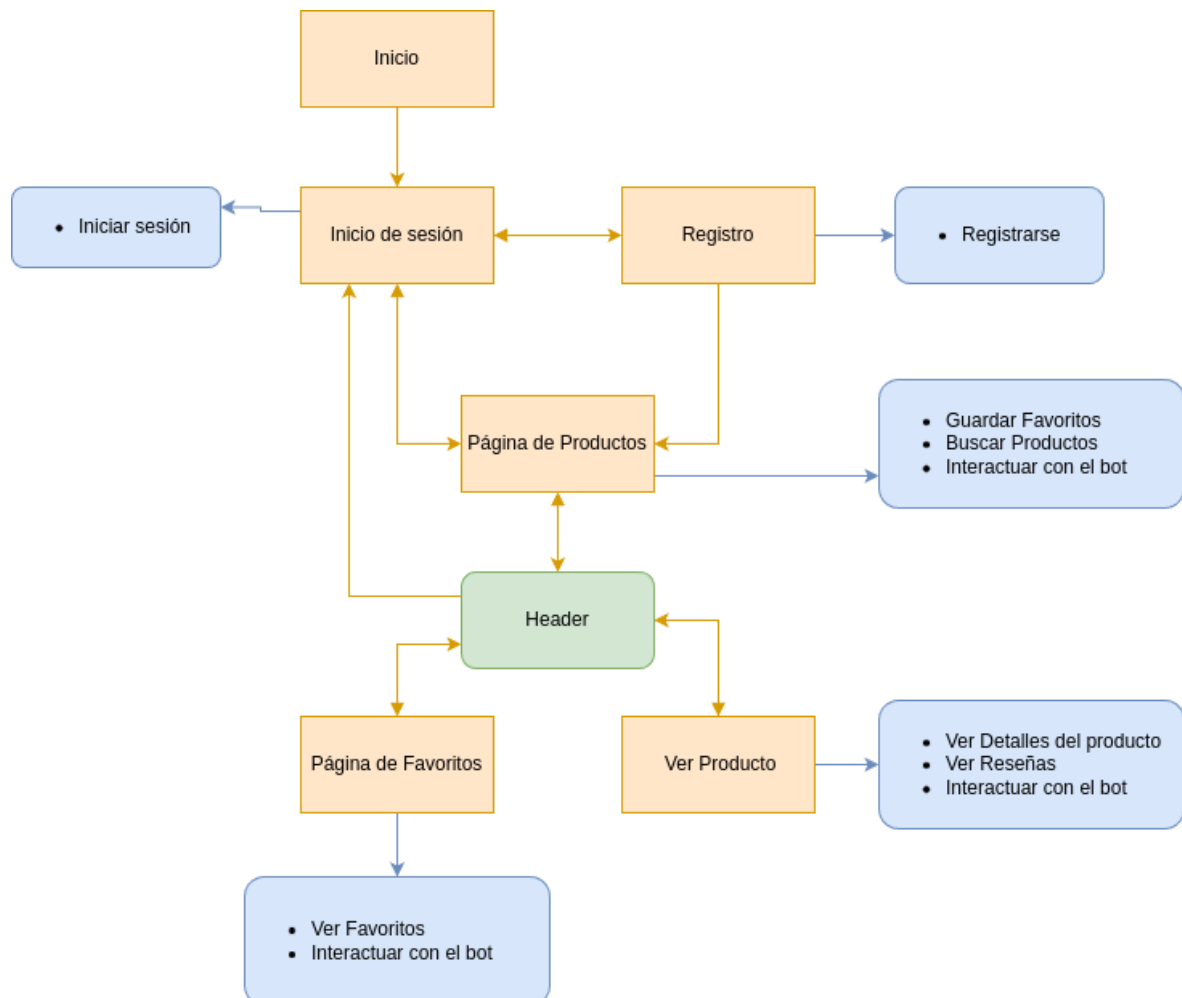


Figura 4.2 Vistas y acciones del usuario

Por su parte, los administradores acceden a un dashboard especializado tras el inicio de sesión. Desde este panel pueden consultar visualizaciones interactivas como gráficas y mapas de calor, acceder a estadísticas del sistema y explorar la pestaña de usuarios, donde se permite la eliminación o modificación del rol de los mismos. Además, el administrador puede generar reportes desde una sección dedicada, así como revisar el listado de productos y su propia lista de favoritos, al igual que un usuario convencional. Todas estas

funcionalidades están disponibles también mediante interacción con el chatbot, lo que permite una gestión avanzada y conversacional del sistema.

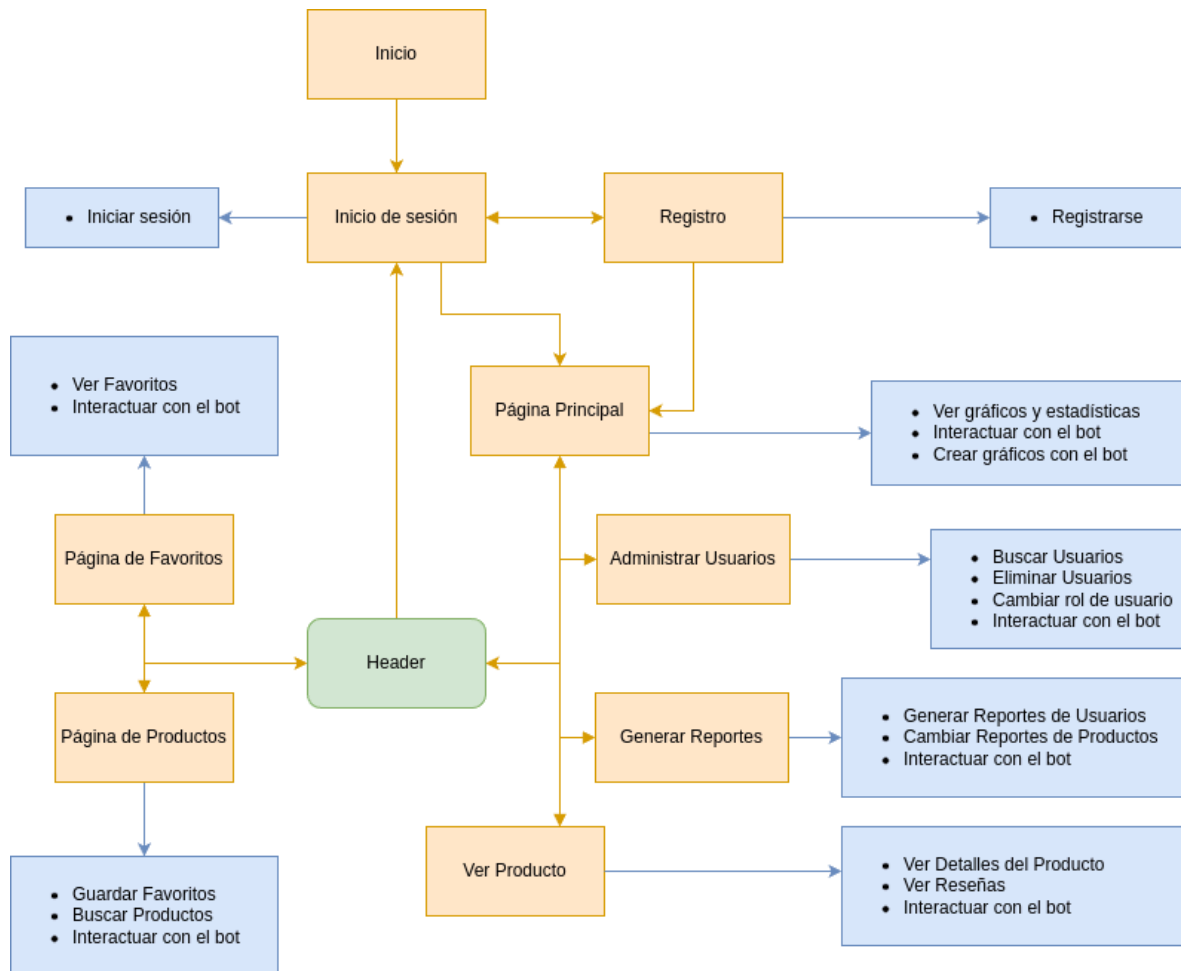


Figura 4.3 Vistas y acciones del Administrador

La navegación, tanto para usuarios estándar como administradores, se realiza mediante un header accesible en todas las vistas. La interfaz ha sido diseñada con un enfoque responsive, garantizando una experiencia fluida y adaptativa desde cualquier dispositivo, ya sea móvil, tablet o equipo de escritorio como podemos ver en el apartado [4.3.2](#).

4.2 ANÁLISIS DE REQUISITOS

La etapa de análisis de requisitos se organiza en dos partes: una dedicada a su definición y otra a su especificación, en las que se identifican y detallan tanto los requisitos funcionales como los no funcionales del sistema.

4.2.1 DEFINICIÓN DE REQUISITOS

Los requisitos del sistema se definieron durante la reunión inicial con el cliente. Estos se clasifican en dos grandes grupos: funcionales y no funcionales.

Los **requisitos funcionales** describen las acciones que el sistema debe ser capaz de realizar, es decir, las funcionalidades esperadas por el usuario.

En cambio, **los requisitos no funcionales** especifican las características generales que debe cumplir el sistema para garantizar su calidad, como el rendimiento, la seguridad, la usabilidad o la tolerancia a fallos. Cada uno de estos requisitos no funcionales se documenta indicando su identificador y nombre.

ID	CATEGORÍA	TÍTULO
RNF-01	Interfaz y Usabilidad	Adaptabilidad a distintos dispositivos (responsive design)
RNF-02	Interfaz y Usabilidad	Navegación clara mediante header persistente
RNF-03	Interfaz y Usabilidad	Integración no intrusiva del chatbot
RNF-04	Tecnologías y Lenguajes	Uso de React, Flask y PostgreSQL
RNF-05	Tecnologías y Lenguajes	Contenerización con Docker
RNF-06	Tecnologías y Lenguajes	Integración de Apache Kafka para ingesta de datos
RNF-07	Tecnologías y Lenguajes	Soporte del estándar HTTP
RNF-08	Rendimiento	Tiempo de respuesta del chatbot

RNF-09	Rendimiento	Tiempo de carga de productos
RNF-10	Rendimiento	Paginación en respuestas con grandes volúmenes de datos
RNF-11	Seguridad	Almacenamiento cifrado de credenciales
RNF-12	Seguridad	Acceso restringido al panel de administración
RNF-13	Seguridad	Cumplimiento con la normativa de protección de datos
RNF-14	Seguridad	Autenticación mediante token en la API
RNF-15	Tolerancia a Fallos	Continuidad de servicio ante fallos de Kafka
RNF-16	Tolerancia a Fallos	Registro y notificación de errores críticos
RNF-17	Portabilidad	Entorno de desarrollo reproducible con Docker Compose
RNF-18	Mantenibilidad	Código modular
RNF-19	Mantenibilidad	Mostrar logs para análisis posterior

Tabla 4.1 Requisitos no funcionales

A continuación se enumeran los requisitos funcionales de la aplicación. En el siguiente apartado se ampliarán con más detalle.

ID	CATEGORÍA	TÍTULO
RF-01	Gestión de usuarios	Registro con email/contraseña o mediante Google/GitHub.
RF-02	Gestión de usuarios	Inicio de sesión con credenciales o proveedores sociales.
RF-03	Gestión de usuarios	Cierre de sesión seguro.
RF-04	Gestión de usuarios	Visualización del listado de usuarios por parte del administrador.
RF-05	Gestión de usuarios	Eliminación de usuarios por el administrador.

RF-06	Gestión de usuarios	Cambio de rol de usuario por el administrador.
RF-07	Acceso por roles	Diferenciación de accesos para usuario y administrador.
RF-08	Acceso por roles	Bloqueo de funciones protegidas sin autenticación.
RF-09	Acceso por roles	Limitación de funciones del bot según rol.
RF-10	Exploración de productos	Mostrar productos tras iniciar sesión.
RF-11	Exploración de productos	Guardar las búsquedas del usuario
RF-12	Exploración de productos	Motor de búsqueda de productos
RF-13	Exploración de productos	Vista detallada del producto (info, reviews, enlace).
RF-14	Exploración de productos	Añadir y quitar favoritos.
RF-15	Exploración de productos	Ver favoritos del usuario.
RF-16	Interacción con IA	Acceso al chatbot desde cualquier vista.
RF-17	Interacción con IA	Consultas generales al bot sobre productos.
RF-18	Interacción con IA	Respuestas del bot contextualizadas por producto.
RF-19	Interacción con IA	Generación de contenido para administradores por el bot (gráficos, reportes).
RF-20	Interacción con IA	Selección del proveedor de IA por parte del usuario
RF-21	Dashboard de administración	Acceso a métricas y estadísticas.
RF-22	Dashboard de administración	Visualización de gráficos y mapa de calor de inicio de sesiones.
RF-23	Dashboard de administración	Creación de reportes mediante formulario
RF-24	Monitorización	Endpoint HTTP para verificar el estado del sistema.

RF-25	Procesamiento de datos	Lectura de archivos desde Kafka Producer
RF-26	Procesamiento de datos	Envío de datos a topic
RF-27	Procesamiento de datos	Consumo de datos y reenvío al backend
RF-28	Procesamiento de datos	Procesamiento y almacenamiento en backend

Tabla 4.2 Requisitos funcionales

4.2.2 ESPECIFICACIÓN DE REQUISITOS

En este apartado se procede a la especificación detallada de los requisitos funcionales y no funcionales previamente enumerados, describiendo de forma clara su propósito, comportamiento esperado y las condiciones que deben cumplir dentro del sistema.

REQUISITO NO FUNCIONAL	
ID	RNF-01
Título	Adaptabilidad a distintos dispositivos (responsive design)
Descripción	La aplicación debe adaptarse correctamente a diferentes tamaños de pantalla, garantizando una experiencia fluida en móviles, tablets y ordenadores.
Categoría	Interfaz y Usabilidad
Estado	Desarrollado

Tabla 4.3 RNF-01: Adaptabilidad a distintos dispositivos (responsive design)

REQUISITO NO FUNCIONAL	
ID	RNF-02
Título	Navegación clara mediante header persistente
Descripción	La navegación debe ser accesible desde un encabezado fijo visible en todas las vistas, facilitando el acceso a las funcionalidades principales.
Categoría	Interfaz y Usabilidad
Estado	Desarrollado

Tabla 4.4 RNF-02: Navegación clara mediante header persistente

REQUISITO NO FUNCIONAL	
ID	RNF-03
Título	Integración no intrusiva del chatbot
Descripción	El chatbot debe estar disponible desde cualquier vista de la aplicación mediante un botón flotante que no interfiera con el contenido principal.
Categoría	Interfaz y Usabilidad
Estado	Desarrollado

Tabla 4.5 RNF-03: Integración no intrusiva del chatbot

REQUISITO NO FUNCIONAL	
ID	RNF-04
Título	Uso de React, Flask y PostgreSQL
Descripción	El sistema debe desarrollarse utilizando React para el frontend, Flask para el backend y PostgreSQL como sistema de gestión de base de datos.
Categoría	Tecnologías y Lenguajes
Estado	Desarrollado

Tabla 4.6 RNF-04: Uso de React, Flask y PostgreSQL

REQUISITO NO FUNCIONAL	
ID	RNF-05
Título	Contenerización con Docker
Descripción	Todos los componentes del sistema deben estar empaquetados en contenedores Docker para asegurar la portabilidad y facilitar el despliegue.
Categoría	Tecnologías y Lenguajes
Estado	Desarrollado

Tabla 4.7 RNF-05: Contenerización con Docker

REQUISITO NO FUNCIONAL	
ID	RNF-06
Título	Integración de Apache Kafka para ingesta de datos
Descripción	Se debe utilizar Apache Kafka para la transmisión eficiente de datos desde los archivos fuente hacia el backend.
Categoría	Tecnologías y Lenguajes
Estado	Desarrollado

Tabla 4.8 RNF-06: Integración de Apache Kafka para ingesta de datos

REQUISITO NO FUNCIONAL	
ID	RNF-07
Título	Soporte del estándar HTTP
Descripción	El sistema debe cumplir con el protocolo HTTP para asegurar interoperabilidad con otros servicios y facilitar las integraciones.
Categoría	Tecnologías y Lenguajes
Estado	Desarrollado

Tabla 4.9 RNF-07: Soporte del estándar HTTP

REQUISITO NO FUNCIONAL	
ID	RNF-08
Título	Tiempo de respuesta del chatbot
Descripción	El chatbot debe responder a las consultas de los usuarios en un tiempo inferior a 2 segundos bajo condiciones normales de uso y siempre que no use una función, como por ejemplo buscar productos.
Categoría	Rendimiento
Estado	Desarrollado

Tabla 4.10 RNF-08: Tiempo de respuesta del chatbot

REQUISITO NO FUNCIONAL	
ID	RNF-09
Título	Tiempo de carga de productos
Descripción	La visualización inicial de productos debe completarse en menos de 5 segundos para datasets medianos. (< 5000 productos)
Categoría	Rendimiento
Estado	Desarrollado

Tabla 4.11 RNF-09: Tiempo de carga de productos

REQUISITO NO FUNCIONAL	
ID	RNF-10
Título	Paginación en respuestas con grandes volúmenes de datos
Descripción	El sistema debe implementar paginación en las respuestas de datos extensos para evitar sobrecargar el sistema o la red.
Categoría	Rendimiento
Estado	Desarrollado

Tabla 4.12 RNF-10: Paginación en respuestas con grandes volúmenes de datos

REQUISITO NO FUNCIONAL	
ID	RNF-11
Título	Almacenamiento cifrado de credenciales
Descripción	Las contraseñas y datos sensibles deben almacenarse de forma cifrada para garantizar la seguridad de los usuarios.
Categoría	Seguridad
Estado	Desarrollado

Tabla 4.13 RNF-11: Almacenamiento cifrado de credenciales

REQUISITO NO FUNCIONAL	
ID	RNF-12
Título	Acceso restringido al panel de administración
Descripción	Solo los usuarios con rol de administrador podrán acceder al panel y funciones de métricas, reportes y gestión.
Categoría	Seguridad
Estado	Desarrollado

Tabla 4.14 RNF-12: Acceso restringido al panel de administración

REQUISITO NO FUNCIONAL	
ID	RNF-13
Título	Cumplimiento con la normativa de protección de datos
Descripción	El sistema debe garantizar el tratamiento adecuado de los datos personales conforme a la legislación vigente.
Categoría	Seguridad
Estado	Desarrollado

Tabla 4.15 RNF-13: Cumplimiento con la normativa de protección de datos

REQUISITO NO FUNCIONAL	
ID	RNF-14
Título	Autenticación mediante token en la API
Descripción	Las peticiones a la API deben requerir autenticación mediante un token válido por seguridad.
Categoría	Seguridad
Estado	Desarrollado

Tabla 4.16 RNF-14: Autenticación mediante token en la API

REQUISITO NO FUNCIONAL	
ID	RNF-15
Título	Continuidad de servicio ante fallos de Kafka
Descripción	El sistema debe mantener su funcionalidad básica en caso de interrupciones del flujo de datos por errores en Kafka.
Categoría	Tolerancia a Fallos
Estado	Desarrollado

Tabla 4.17 RNF-15: Continuidad de servicio ante fallos de Kafka

REQUISITO NO FUNCIONAL	
ID	RNF-16
Título	Registro y notificación de errores críticos
Descripción	Los errores importantes deben registrarse y generar una alerta para el administrador del sistema.
Categoría	Tolerancia a Fallos
Estado	Desarrollado

Tabla 4.18 RNF-16: Registro y notificación de errores críticos

REQUISITO NO FUNCIONAL	
ID	RNF-17
Título	Entorno de desarrollo reproducible con Docker Compose
Descripción	El entorno de desarrollo debe poder reproducirse fácilmente mediante Docker Compose.
Categoría	Portabilidad
Estado	Desarrollado

Tabla 4.19 RNF-17: Entorno de desarrollo reproducible con Docker Compose

REQUISITO NO FUNCIONAL	
ID	RNF-18
Título	Código modular
Descripción	El código debe estar estructurado de forma modular para facilitar su mantenimiento y ampliación.
Categoría	Mantenibilidad
Estado	Desarrollado

Tabla 4.20 RNF-18: Código modular

REQUISITO NO FUNCIONAL	
ID	RNF-19
Título	Mostrar logs para análisis posterior
Descripción	El sistema debe almacenar logs que puedan ser analizados posteriormente para depuración y mejora continua.
Categoría	Mantenibilidad
Estado	Desarrollado

Tabla 4.21 RNF-19: Mostrar logs para análisis posterior

A continuación, se procede a la especificación individual de los requisitos funcionales definidos previamente. Para cada uno de ellos se indicará a qué vista o componente de la aplicación está asociado, así como posibles dependencias con otros requisitos o funcionalidades, con el fin de comprender mejor su contexto de implementación y su papel dentro del sistema.

REQUISITO FUNCIONAL	
ID	RF-01
Título	Registro con email/contraseña o mediante Google/GitHub.
Descripción	Permitir a los usuarios crear una cuenta utilizando su email y contraseña o mediante autenticación con proveedores sociales como Google o GitHub.
Categoría	Gestión de usuarios
Vista asociada	Registro
Dependencias	
Estado	Desarrollado

Tabla 4.22 RF-01: Registro con email/contraseña o mediante Google/GitHub.

REQUISITO FUNCIONAL	
ID	RF-02
Título	Inicio de sesión con credenciales o proveedores sociales.
Descripción	Autenticar a los usuarios mediante el ingreso de credenciales propias o acceso con servicios como Google o GitHub.
Categoría	Gestión de usuarios
Vista asociada	Inicio de sesión
Dependencias	RF-01
Estado	Desarrollado

Tabla 4.23 RF-02: Inicio de sesión con credenciales o proveedores sociales.

REQUISITO FUNCIONAL	
ID	RF-03
Título	Cierre de sesión seguro.
Descripción	Proporcionar al usuario la capacidad de cerrar sesión de forma segura desde cualquier parte del sistema.
Categoría	Gestión de usuarios
Vista asociada	Header
Dependencias	RF-02
Estado	Desarrollado

Tabla 4.24 RF-03: Cierre de sesión seguro.

REQUISITO FUNCIONAL	
ID	RF-04
Título	Visualización del listado de usuarios por parte del administrador.
Descripción	Permitir al administrador acceder a una lista con los usuarios registrados y su información básica.
Categoría	Gestión de usuarios
Vista asociada	Administrar Usuarios
Dependencias	RF-02, RF-07
Estado	Desarrollado

Tabla 4.25 RF-04: Visualización del listado de usuarios por parte del administrador.

REQUISITO FUNCIONAL	
ID	RF-05
Título	Eliminación de usuarios por el administrador.
Descripción	Brindar al administrador la posibilidad de eliminar cuentas de usuario desde el panel de gestión.
Categoría	Gestión de usuarios
Vista asociada	Administrar Usuarios
Dependencias	RF-04
Estado	Desarrollado

Tabla 4.26 RF-05: Eliminación de usuarios por el administrador.

REQUISITO FUNCIONAL	
ID	RF-06
Título	Cambio de rol de usuario por el administrador.
Descripción	Permitir al administrador modificar el rol de un usuario, asignando privilegios de administrador o revocándolos.
Categoría	Gestión de usuarios
Vista asociada	Administrar Usuarios
Dependencias	RF-04
Estado	Desarrollado

Tabla 4.27 RF-06: Cambio de rol de usuario por el administrador.

REQUISITO FUNCIONAL	
ID	RF-07
Título	Diferenciación de accesos para usuario y administrador.
Descripción	Gestionar la visibilidad y accesibilidad de funcionalidades en función del rol del usuario autenticado.
Categoría	Acceso por roles
Vista asociada	
Dependencias	RF-02
Estado	Desarrollado

Tabla 4.28 RF-07: Diferenciación de accesos para usuario y administrador.

REQUISITO FUNCIONAL	
ID	RF-08
Título	Bloqueo de funciones protegidas sin autenticación.
Descripción	Evitar el acceso a funciones restringidas por parte de usuarios no autenticados.
Categoría	Acceso por roles
Vista asociada	
Dependencias	RF-07
Estado	Desarrollado

Tabla 4.29 RF-08: Bloqueo de funciones protegidas sin autenticación.

REQUISITO FUNCIONAL	
ID	RF-09
Título	Limitación de funciones del bot según rol.
Descripción	Controlar el acceso a ciertas capacidades del chatbot dependiendo del tipo de usuario (normal o admin).
Categoría	Acceso por roles
Vista asociada	Chatbot
Dependencias	RF-07
Estado	Desarrollado

Tabla 4.30 RF-09: Limitación de funciones del bot según rol.

REQUISITO FUNCIONAL	
ID	RF-10
Título	Mostrar productos tras iniciar sesión.
Descripción	Desplegar una vista con productos disponibles al usuario tras autenticarse exitosamente.
Categoría	Exploración de productos
Vista asociada	Ver Productos
Dependencias	RF-02
Estado	Desarrollado

Tabla 4.31 RF-10: Mostrar productos tras iniciar sesión.

REQUISITO FUNCIONAL	
ID	RF-11
Título	Guardar las búsquedas del usuario
Descripción	Almacenar las búsquedas realizadas por el usuario para recuperación rápida.
Categoría	Exploración de productos
Vista asociada	Ver Productos
Dependencias	RF-10
Estado	Desarrollado

Tabla 4.32 RF-11: Guardar las búsquedas del usuario

REQUISITO FUNCIONAL	
ID	RF-12
Título	Motor de búsqueda de productos
Descripción	Proveer una barra de búsqueda para localizar productos mediante el sistema combinado de palabras clave y embeddings.
Categoría	Exploración de productos
Vista asociada	Ver Productos
Dependencias	RF-10
Estado	Desarrollado

Tabla 4.33 RF-12: Motor de búsqueda de productos

REQUISITO FUNCIONAL	
ID	RF-13
Título	Vista detallada del producto (info, reviews, enlace).
Descripción	Mostrar información detallada de cada producto, incluyendo su descripción, enlace a Amazon y opiniones de usuarios.
Categoría	Exploración de productos
Vista asociada	Página de Producto
Dependencias	RF-10
Estado	Desarrollado

Tabla 4.34 RF-13: Vista detallada del producto (info, reviews, enlace).

REQUISITO FUNCIONAL	
ID	RF-14
Título	Añadir y quitar favoritos.
Descripción	Permitir al usuario marcar y desmarcar productos como favoritos desde la lista o la vista individual del producto.
Categoría	Exploración de productos
Vista asociada	Ver Productos
Dependencias	RF-13
Estado	Desarrollado

Tabla 4.35 RF-14: Añadir y quitar favoritos.

REQUISITO FUNCIONAL	
ID	RF-15
Título	Ver favoritos del usuario.
Descripción	Acceder a una sección personalizada donde el usuario puede consultar los productos que ha marcado como favoritos.
Categoría	Exploración de productos
Vista asociada	Página de Favoritos
Dependencias	RF-14
Estado	Desarrollado

Tabla 4.36 RF-15: Ver favoritos del usuario.

REQUISITO FUNCIONAL	
ID	RF-16
Título	Acceso al chatbot desde cualquier vista.
Descripción	Ofrecer un acceso persistente al chatbot en todas las vistas para facilitar la interacción conversacional.
Categoría	Exploración de productos
Vista asociada	
Dependencias	RF-02
Estado	Desarrollado

Tabla 4.37 RF-16: Acceso al chatbot desde cualquier vista.

REQUISITO FUNCIONAL	
ID	RF-17
Título	Consultas generales al bot sobre productos.
Descripción	Habilitar al usuario para hacer preguntas abiertas al chatbot sobre productos.
Categoría	Interacción con IA
Vista asociada	Chatbot
Dependencias	RF-16
Estado	Desarrollado

Tabla 4.38 RF-17: Consultas generales al bot sobre productos.

REQUISITO FUNCIONAL	
ID	RF-18
Título	Respuestas del bot contextualizadas por producto.
Descripción	Permitir que el chatbot responda de manera específica sobre el producto que el usuario está visualizando, con información del mismo y de las reviews de este.
Categoría	Interacción con IA
Vista asociada	Página de Producto
Dependencias	RF-13, RF-16
Estado	Desarrollado

Tabla 4.39 RF-18: Respuestas del bot contextualizadas por producto.

REQUISITO FUNCIONAL	
ID	RF-19
Título	Generación de contenido para administradores por el bot (gráficos, reportes).
Descripción	Facilitar al bot la generación de visualizaciones o reportes a partir de datos del sistema si el usuario es administrador.
Categoría	Interacción con IA
Vista asociada	Página principal de administrador
Dependencias	RF-07, RF-16
Estado	Desarrollado

Tabla 4.40 RF-19: Generación de contenido para administradores por el bot (gráficos, reportes).

REQUISITO FUNCIONAL	
ID	RF-20
Título	Selección del proveedor de IA por parte del usuario
Descripción	Permitir al usuario elegir entre distintos proveedores de modelos de lenguaje integrados en el sistema.
Categoría	Interacción con IA
Vista asociada	Chatbot
Dependencias	RF-16
Estado	Desarrollado

Tabla 4.41 RF-20: Selección del proveedor de IA por parte del usuario

REQUISITO FUNCIONAL	
ID	RF-21
Título	Acceso a métricas y estadísticas.
Descripción	Habilitar al administrador a consultar estadísticas generales sobre el uso del sistema.
Categoría	Dashboard de administración
Vista asociada	Página principal de administrador
Dependencias	RF-07
Estado	Desarrollado

Tabla 4.42 RF-21: Acceso a métricas y estadísticas.

REQUISITO FUNCIONAL	
ID	RF-22
Título	Visualización de gráficos y mapa de calor de inicio de sesiones.
Descripción	Mostrar visualizaciones interactivas sobre el comportamiento de los usuarios en el sistema.
Categoría	Dashboard de administración
Vista asociada	Página principal de administrador
Dependencias	RF-21
Estado	Desarrollado

Tabla 4.43 RF-22: Visualización de gráficos y mapa de calor de inicio de sesiones.

REQUISITO FUNCIONAL	
ID	RF-23
Título	Creación de reportes mediante formulario
Descripción	Ofrecer al administrador un formulario para generar reportes personalizados del sistema.
Categoría	Dashboard de administración
Vista asociada	Generar reportes
Dependencias	RF-21
Estado	Desarrollado

Tabla 4.44 RF-23: Creación de reportes mediante formulario

REQUISITO FUNCIONAL	
ID	RF-24
Título	Endpoint HTTP para verificar el estado del sistema.
Descripción	Exponer un endpoint de tipo GET para consultar el estado de salud del sistema.
Categoría	Monitorización
Vista asociada	
Dependencias	
Estado	Desarrollado

Tabla 4.45 RF-24: Endpoint HTTP para verificar el estado del sistema.

REQUISITO FUNCIONAL	
ID	RF-25
Título	Lectura de archivos desde Kafka Producer
Descripción	Configurar el sistema para que un productor Kafka lea datos desde archivos externos.
Categoría	Procesamiento de datos
Vista asociada	
Dependencias	
Estado	Desarrollado

Tabla 4.46 RF-25: Lectura de archivos desde Kafka Producer

REQUISITO FUNCIONAL	
ID	RF-26
Título	Envío de datos a topic
Descripción	Permitir que el productor Kafka envíe los datos a un topic específico para su posterior procesamiento.
Categoría	Procesamiento de datos
Vista asociada	
Dependencias	RF-25
Estado	Desarrollado

Tabla 4.47 RF-26: Envío de datos a topic

REQUISITO FUNCIONAL	
ID	RF-27
Título	Consumo de datos y reenvío al backend
Descripción	El consumidor Kafka debe extraer los datos del topic y reenviarlos al backend mediante una API REST.
Categoría	Procesamiento de datos
Vista asociada	
Dependencias	RF-26
Estado	Desarrollado

Tabla 4.48 RF-27: Consumo de datos y reenvío al backend

REQUISITO FUNCIONAL	
ID	RF-28
Título	Procesamiento y almacenamiento en backend
Descripción	Procesar los datos recibidos desde Kafka y almacenarlos de forma estructurada en la base de datos.
Categoría	Procesamiento de datos
Vista asociada	
Dependencias	RF-27
Estado	Desarrollado

Tabla 4.49 RF-28: Procesamiento y almacenamiento en backend

4.3 DISEÑO

Este apartado presenta el diseño del sistema, cuyo objetivo es reflejar las decisiones técnicas y arquitectónicas tomadas para dar respuesta a los requisitos definidos

previamente. Se expone tanto la estructura general de la aplicación como los componentes clave, sus relaciones y los principales flujos de interacción, sentando así las bases para la fase de implementación.

4.3.1 DISEÑO DE LA ARQUITECTURA

El sistema ha sido organizado en diferentes submódulos, cada uno de ellos contenido en su propia carpeta dentro del repositorio principal. Cada carpeta agrupa el código y los recursos correspondientes a una parte específica del sistema: backend, frontend, orquestación con Docker y scripts para el flujo de datos con Kafka.

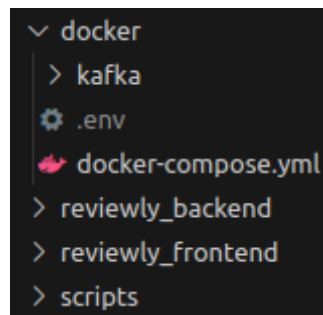


Figura 4.4 Sub-proyectos de la aplicación

Cada uno de los módulos del sistema ha sido diseñado para ser **independiente** en la medida de lo posible. Esto permite que, por ejemplo, el frontend actual podría sustituirse por una aplicación móvil o de escritorio sin necesidad de modificar la lógica del backend. De forma similar, el backend podría ser reimplementado en otro lenguaje o framework respetando la estructura de la API, y seguir funcionando con el frontend actual. Incluso el sistema de ingesta y procesamiento de datos basado en Kafka puede funcionar de forma aislada, permitiendo su reutilización en otros contextos o su sustitución por otro sistema de mensajería.

La arquitectura de la aplicación se basa en el estilo **cliente-servidor** [48], un modelo distribuido ampliamente utilizado que separa claramente las responsabilidades entre dos componentes: el cliente, que representa los datos e inicia las acciones, y el servidor, que

realiza el trabajo pesado y gestiona el acceso a los recursos. En este caso, el frontend actúa como cliente y se comunica exclusivamente con el backend, que funciona como servidor, a través de peticiones HTTP bajo el protocolo TCP/IP. Esta comunicación permite que el cliente envíe y reciba datos del servidor, pero no accede directamente ni a la base de datos ni a otros servicios como Kafka. Un ejemplo típico de interacción sería el siguiente:

- El usuario accede a la vista principal de productos desde el navegador.
- El frontend, que se ubica en el puerto 5173, envía una petición GET al endpoint `/api/v0/products` del backend en el puerto 5000.
- El backend, procesa la solicitud, consulta la base de datos (ubicada en el puerto 5432) y devuelve los resultados en formato JSON.
- El frontend renderiza los productos recibidos para mostrarlos al usuario.

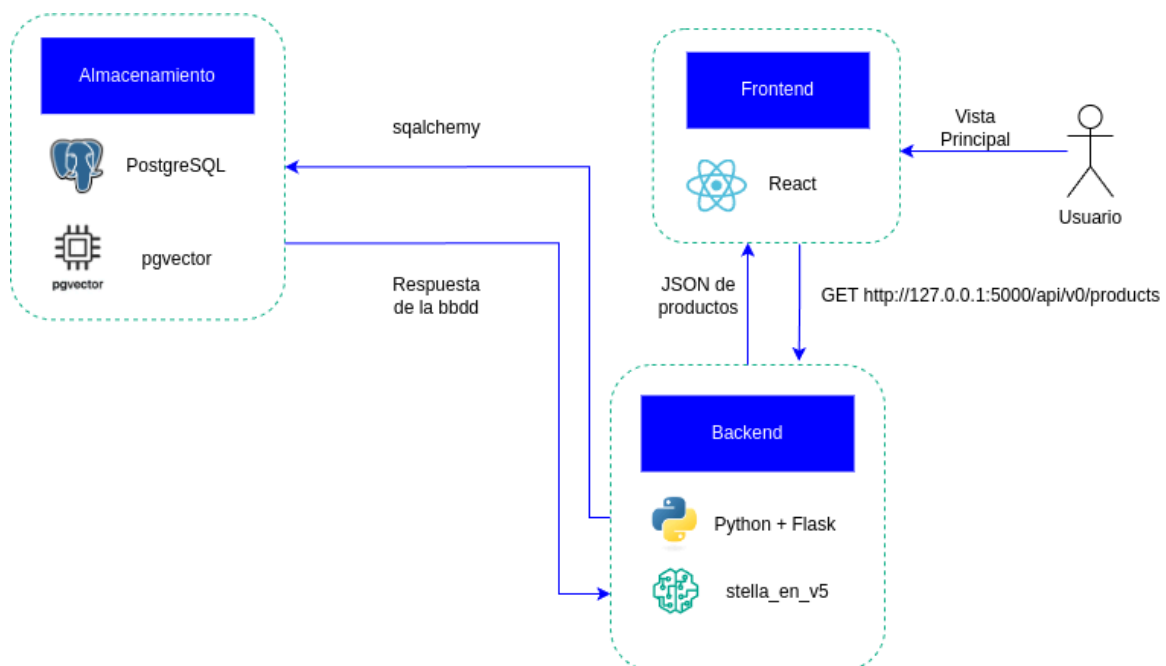


Figura 4.5 Ejemplo de interacción cliente-servidor

4.3.2 DISEÑO DEL FRONTEND

La interfaz de usuario del sistema se ha desarrollado utilizando **React** [\[47\]](#), una biblioteca de JavaScript centrada en la construcción de interfaces web dinámicas y eficientes. React permite crear aplicaciones de una sola página (Single Page Applications, SPA), en las cuales todo el contenido se carga inicialmente en un único documento HTML, y las diferentes vistas se renderizan dinámicamente desde el navegador sin necesidad de recargar la página. Este modelo ofrece una experiencia más fluida, rápida y coherente para el usuario.

Para gestionar la navegación dentro de la aplicación, se ha empleado **React Router**, una herramienta que permite definir rutas asociadas a diferentes componentes visuales. Gracias a ello, el sistema puede controlar el acceso a cada sección en función del rol y estado de autenticación del usuario, garantizando que solo se muestren las vistas apropiadas según el contexto.

El frontend del sistema se encuentra organizado en una serie de carpetas que reflejan la distribución modular de sus funcionalidades:

- **pages:** Contiene las vistas principales del sistema, como la lista de productos, la pantalla de inicio de sesión o el panel de administración. Cada archivo representa una pantalla completa de la aplicación.
- **components:** Incluye elementos visuales reutilizables, como botones, tarjetas, menús o el header, que se integran en múltiples páginas.
- **routes:** Define la estructura de navegación de la aplicación, indicando qué vistas se muestran en función de la URL. También aplica restricciones según el estado de autenticación del usuario.
- **context:** Gestiona el estado global de la aplicación, especialmente la autenticación, el rol del usuario y la persistencia de la sesión en todo el sistema.
- **assets:** Agrupa recursos estáticos como imágenes, logotipos e iconos utilizados en el diseño visual de la interfaz.

- **styles:** Contiene los estilos globales de la aplicación (archivos CSS), incluyendo colores, tipografías y clases comunes que definen la apariencia general.
- **services:** Centraliza las funciones que realizan llamadas al backend, actuando como intermediario entre la lógica del frontend y la API del servidor.
- **hooks:** Almacena funciones personalizadas de React que encapsulan lógica reutilizable relacionada con el estado, los efectos y las interacciones del usuario.

`main.tsx` y `App.tsx` (aunque no se describen en profundidad en esta sección): son los puntos de entrada de la aplicación y configuran el enrutador, el proveedor de contexto y otras integraciones esenciales.

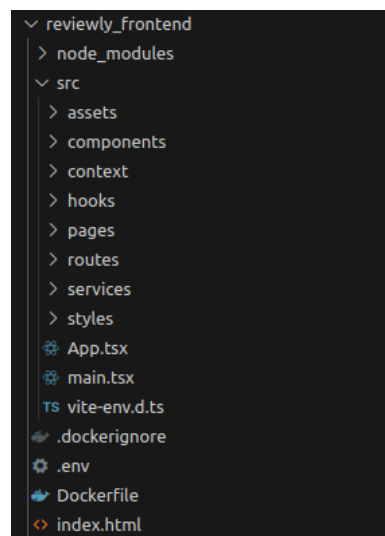


Figura 4.6 Estructura de carpetas del frontend

En términos de diseño visual, se estableció una paleta de colores personalizada y un logotipo representativo como parte del primer prototipo. Aunque estos elementos sirvieron como base para definir la identidad visual del sistema, han experimentado ligeras variaciones a lo largo del desarrollo.

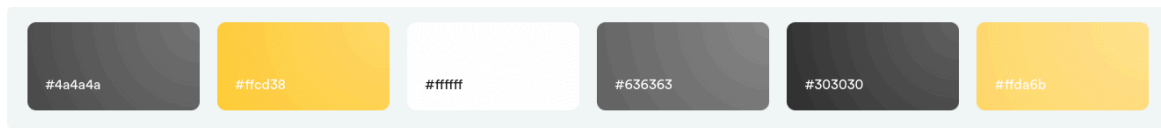


Figura 4.7 Prototipo de la paleta de colores de la aplicación



Figura 4.8 Prototipo del icono de la Aplicación

La [Figura 4.2](#) y la [Figura 4.3](#) muestran esquemas de navegación que recogen las vistas y acciones disponibles para los distintos perfiles de usuario. La primera se centra en el flujo de navegación del usuario estándar, mientras que la segunda ilustra las funcionalidades adicionales accesibles para los administradores. En ambos esquemas se representa también el header común de la aplicación, que actúa como elemento de navegación principal y conecta las distintas pantallas mediante enlaces indicados con flechas.

A continuación, se incluyen dos bocetos que representan el diseño inicial de las vistas de producto y de detalle de producto. Estas pantallas fueron las primeras en ser prototipadas y sentaron las bases del proyecto.

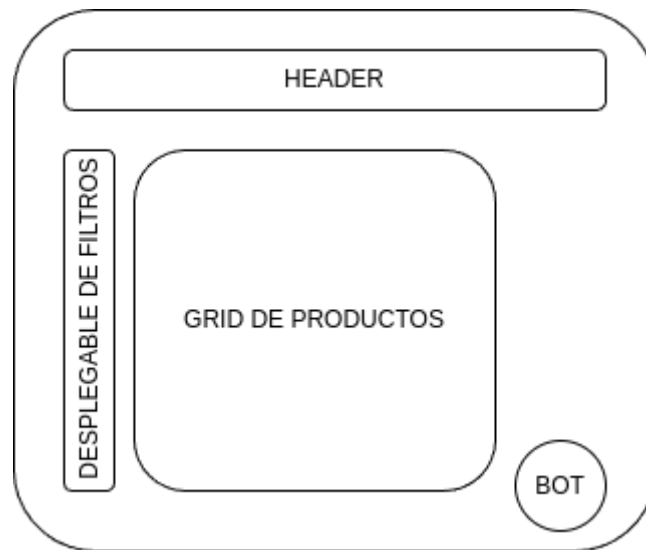


Figura 4.8 Boceto Página de Productos

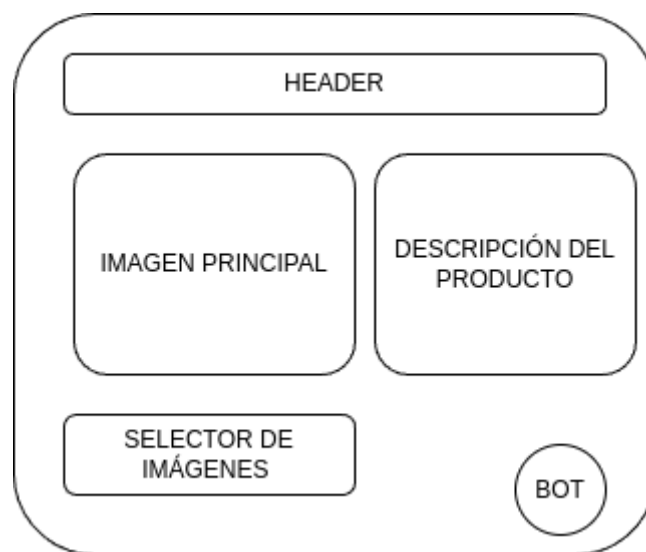


Figura 4.9 Boceto Página Ver Producto

Este diseño modular y escalable permite mantener una clara separación de responsabilidades entre los diferentes elementos del frontend, facilitando su mantenimiento, evolución y coherencia con los requisitos definidos en fases anteriores del proyecto.

4.3.3 DISEÑO DEL BACKEND

El backend de la aplicación se ha desarrollado con un enfoque modular utilizando Flask, un *microframework* ligero para Python que permite construir aplicaciones web y APIs REST de forma rápida y flexible.

Para la gestión de la base de datos se ha empleado **SQLAlchemy**, una herramienta de mapeo objeto-relacional, comúnmente conocida como ORM (Object-Relational Mapping). Esta tecnología permite definir las entidades del sistema como clases de Python, las cuales se traducen automáticamente en tablas dentro de una base de datos. De esta manera, se facilita el acceso, manipulación y consulta de los datos sin necesidad de escribir directamente sentencias SQL.

La documentación y organización de la API se ha implementado mediante **Flask-RESTX**, una extensión que permite estructurar los distintos módulos del backend en namespaces y generar documentación interactiva, así como el testeo de los diferentes endpoints desde el navegador.

El sistema de autenticación y control de acceso se basa en **JWT** (JSON Web Tokens). Esta tecnología permite emitir tokens cifrados al usuario tras autenticarse, los cuales son utilizados en cada petición posterior para verificar su identidad y permisos, sin necesidad de mantener sesiones activas en el servidor.

A nivel estructural, el proyecto se organiza en distintas carpetas según la responsabilidad de cada módulo. A continuación se detallan sus propósitos:

- **models:** Define las entidades principales del sistema, como usuarios, productos o reviews, mediante clases que representan directamente las tablas de la base de datos.
- **routes:** Contiene los controladores de la API organizados en namespaces. Cada archivo define un conjunto de rutas relacionadas con una funcionalidad específica (productos, autenticación, chatbot, etc.).

- **services:** Implementa la lógica de negocio de cada módulo, separando esta capa del manejo directo de rutas. Esta división favorece la reutilización del código y mejora su legibilidad.
- **config.py:** Centraliza la configuración del sistema, como parámetros de conexión, claves secretas o duraciones de tokens. Esta configuración se aplica globalmente durante la inicialización de la aplicación.
- **utils:** Incluye funciones auxiliares de propósito general, utilizadas en distintos módulos del sistema, como validaciones, tratamiento de errores o procesamiento de texto.
- **scripts:** Carpeta destinada a operaciones específicas o automatizadas, como los scripts de producción y consumo de datos mediante Kafka.
- **data:** Contiene archivos iniciales de productos y reviews que sirven de soporte para el funcionamiento del sistema, especialmente en entornos de desarrollo o pruebas.
- **__init__.py:** Módulo principal del backend donde se inicializa la aplicación Flask, se configuran los servicios externos (ORM, CORS, JWT) y se registran los namespaces de la API.

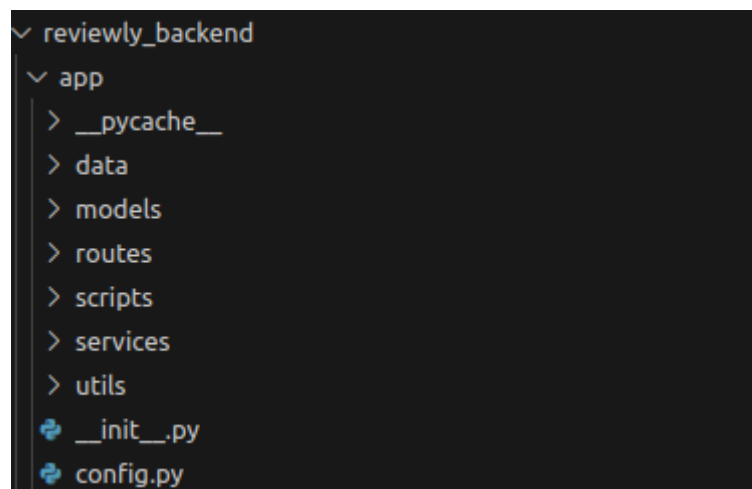


Figura 4.10 Estructura de carpetas del backend

Este diseño garantiza una organización coherente del sistema, una separación clara entre capas (datos, lógica y exposición de endpoints). La modularidad facilita la evolución del proyecto, así como su mantenimiento y ampliación a futuro.

4.3.4 CONTENERIZACIÓN Y PORTABILIDAD DE LA SOLUCIÓN

Con el objetivo de facilitar el despliegue y asegurar la portabilidad del sistema, se ha diseñado una solución contenerizada mediante Docker. En la raíz del proyecto se encuentra un archivo `docker-compose.yml` que orquesta el lanzamiento de los principales servicios de la aplicación: **base de datos**, **backend** y **frontend**. La base de datos se ejecuta como un contenedor independiente basado en una imagen de PostgreSQL con soporte para extensiones vectoriales, y se inicializa automáticamente mediante un volcado de datos. El backend y el frontend se construyen a partir de sus respectivos Dockerfile, ubicados en las carpetas `reviewly_backend` y `reviewly_frontend`.

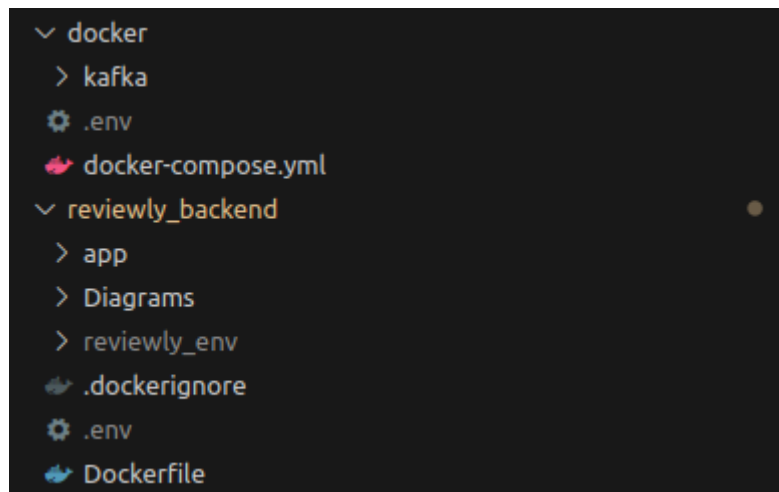


Figura 4.11 Estructura de la parte Docker

4.3.5 DISEÑO DEL SISTEMA KAFKA

El entorno de Kafka se despliega mediante contenedores Docker, definidos en un archivo docker-compose localizado dentro de la carpeta docker/kafka. Este entorno incluye tres servicios principales:

- **Zookeeper**: coordina y gestiona los nodos de Kafka. Es un componente requerido para la operación del clúster.
- **Kafka**: actúa como el broker de mensajes. Permite la publicación y suscripción a flujos de datos a través de tópicos.
- **Kafka UI**: una interfaz gráfica de administración que facilita la inspección de tópicos, mensajes y consumidores activos. Se accede a través del puerto 8080 y permite visualizar el estado del sistema en tiempo real.



Figura 4.12 Servicios de Kafka desplegados

Además del entorno de mensajería, la lógica de producción y consumo de datos se encuentra en la carpeta scripts. Allí se ubican dos scripts principales:

- `producer.py`: responsable de leer datos desde archivos y enviarlos a un tópico de Kafka.
- `consumer.py`: suscrito al tópico correspondiente, recibe los datos, los transforma según un mapeo definido y los reenvía al backend para su almacenamiento.

Ambos scripts utilizan archivos auxiliares para su funcionamiento. El archivo `electronics.db` contiene un mapeo necesario para procesar correctamente los datos, mientras que `last_sent.json` permite llevar un seguimiento de los datos ya procesados, evitando duplicidades. Los archivos que sirven como fuente de datos para el sistema se almacenan en la carpeta `data`, previamente mencionada en la estructura general del proyecto. El funcionamiento detallado de estos componentes y la lógica interna del productor y consumidor se explicará en profundidad en el apartado correspondiente a la implementación.

4.3.6 DISEÑO DE LA BASE DE DATOS

Para la base de datos se ha utilizado **PostgreSQL**, un sistema de gestión de bases de datos relacional robusto, extensible y ampliamente adoptado en entornos de producción. Además de su estructura relacional tradicional, la base de datos incorpora extensiones como **pgvector** para el almacenamiento de embeddings vectoriales, utilizados en las funcionalidades relacionadas con IA, y **pg_trgm** para búsquedas aproximadas sobre texto. A continuación, se describen las tablas principales del sistema:

- **users**: guarda la información de los usuarios registrados, como correo electrónico, contraseñas cifradas, fecha de registro, identificadores sociales (por ejemplo, GitHub) y el rol asignado (usuario o administrador). Además, se relaciona con otras tablas como **user_favorites**, que registra los productos marcados como favoritos, y **user_queries**, que almacena el historial de consultas realizadas al sistema conversacional junto con sus tiempos de ejecución.

- **products:** contiene los productos indexados desde Amazon, incluyendo atributos como título, categoría principal, precio, valoración media, descripciones, enlaces a Amazon y contenido multimedia (imágenes y vídeos). También se vincula con otras tablas como **product_features** y **product_details**, que almacenan características específicas y detalles adicionales de los productos, junto con sus embeddings vectoriales para facilitar análisis avanzados. Las reseñas asociadas a los productos están representadas en la tabla reviews, y cada reseña está vinculada a un autor externo a través de la tabla auxiliar **amazon_users**, que identifica a los usuarios de Amazon que emitieron dichas valoraciones.
- **reviews:** almacena las reseñas de productos realizadas por usuarios de Amazon, incluyendo título, contenido, puntuación, sentimiento (automáticamente detectado), votos útiles, estado de compra verificada, fecha y embeddings para procesamiento semántico. Cada reseña se relaciona con un producto y con un autor registrado en la tabla **amazon_users**.
- **login_logs:** lleva un registro detallado de los accesos de los usuarios a la plataforma, capturando la dirección IP y el momento de inicio de sesión, lo cual permite realizar análisis de uso y la creación de gráficos para administradores.

Estas tablas están relacionadas mediante claves foráneas que garantizan la integridad referencial. La mayoría de las relaciones siguen el modelo de uno-a-muchos o muchos-a-muchos, como ocurre en el caso de productos y reseñas, o usuarios y favoritos.

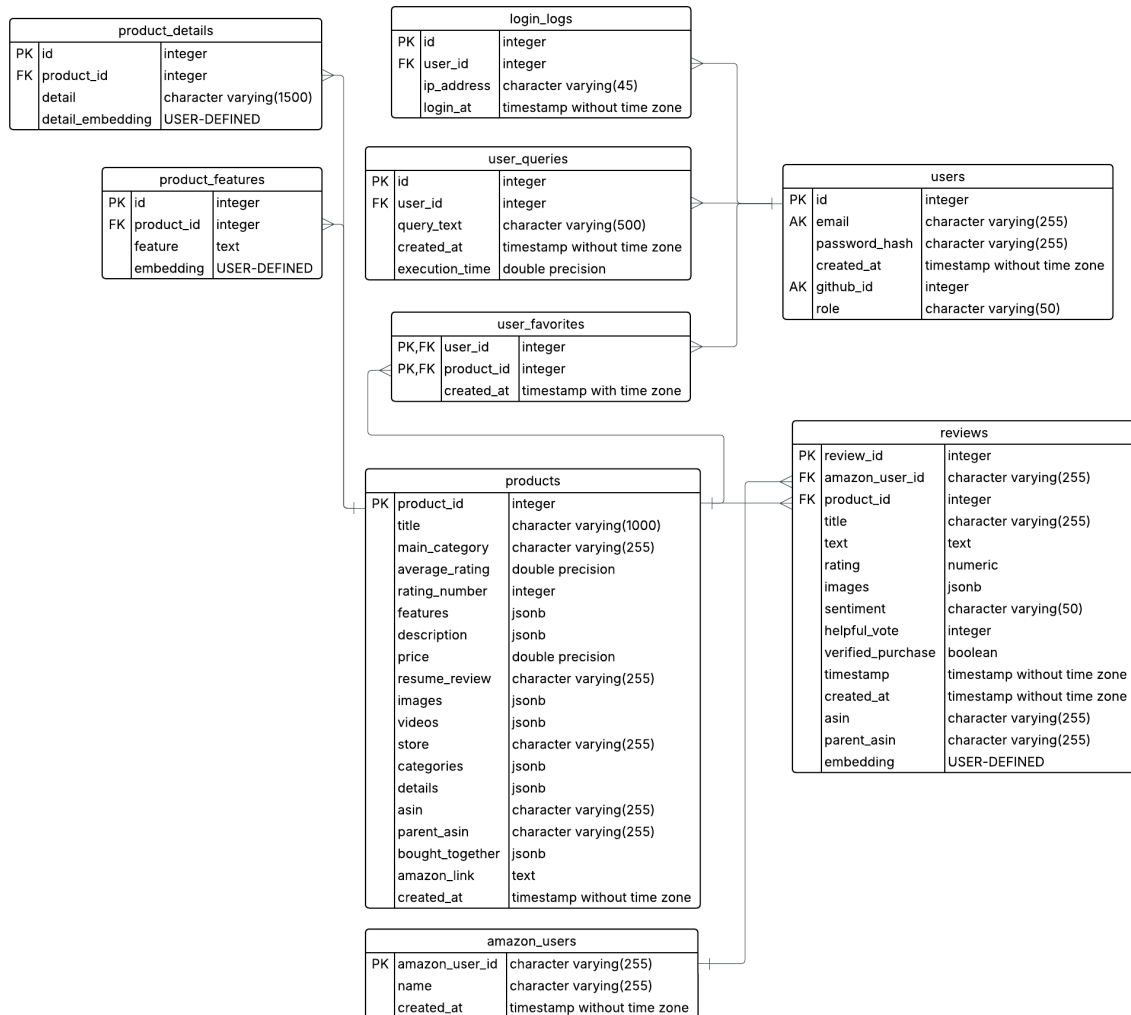


Figura 4.13 Esquema de la base de datos

Esta base de datos actúa como fuente única de verdad para el backend de la aplicación, siendo accedida exclusivamente mediante la API definida en Flask.

4.4 IMPLEMENTACIÓN

Tras haber descrito el diseño de los distintos componentes del proyecto, es momento de abordar los elementos clave de su implementación. En esta sección se detallará cómo se han alcanzado los objetivos propuestos, así como las herramientas utilizadas y las particularidades del proceso de desarrollo.

4.4.1 IMPLEMENTACIÓN DEL FRONTEND

En este apartado se detalla la implementación técnica del frontend de la aplicación. A diferencia del apartado anterior centrado en el diseño, aquí se explican las decisiones prácticas adoptadas durante el desarrollo, incluyendo cómo se estructura la navegación, cómo se gestionan las autenticaciones, cómo se comunican los componentes con el backend y cómo se construyen vistas clave como el menú de productos, el detalle del producto, el panel de administración y el chatbot integrado.

A continuación, se desglosa el proceso de implementación en diferentes secciones temáticas.

1. Inicialización y configuración del proyecto

La implementación del frontend comienza con la creación del proyecto mediante **Vite**, una herramienta de desarrollo diseñada para ofrecer una experiencia rápida y ligera. Vite está especialmente optimizada para frameworks como React y destaca por su tiempo de arranque casi instantáneo y su recarga en caliente eficiente. Su uso facilita un entorno de desarrollo ágil, al tiempo que simplifica la configuración inicial en comparación con herramientas tradicionales como Webpack.

El archivo `main.tsx` actúa como punto de entrada principal de la aplicación. En él se utiliza `ReactDOM.createRoot()` para renderizar el componente principal `<App />`, que se encapsula dentro de `StrictMode`, una herramienta de desarrollo que ayuda a detectar errores potenciales y prácticas obsoletas durante la fase de desarrollo.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.tsx'
import React from 'react'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Figura 4.14 Fragmento de código de inicialización en main.tsx

Por otro lado, el componente **App.tsx** es el encargado de orquestar los distintos proveedores de contexto global de la aplicación. En primer lugar, se integra **GoogleOAuthProvider**, que habilita el inicio de sesión mediante cuentas de Google, utilizando el `clientId` proporcionado por las variables de entorno. A continuación, se encapsula la lógica de autenticación general mediante el **AuthProvider**, encargado de gestionar el estado de sesión del usuario, incluyendo su token, rol y persistencia de autenticación.

Finalmente, se configura la navegación de la aplicación utilizando **BrowserRouter**, un componente de React Router que gestiona el historial de navegación mediante la API del navegador. Dentro del enrutador, se renderiza el componente **AppRoutes**, que centraliza toda la lógica de enrutamiento declarando qué vistas deben mostrarse según la URL actual y el estado del usuario. Esta estructura permite aplicar filtros como rutas protegidas (**ProtectedRoute**) o rutas exclusivas para administradores (**AdminRoute**), garantizando que solo se acceda a las secciones autorizadas según el rol y autenticación del usuario. Gracias a este enfoque, la navegación es fluida, segura y coherente con la lógica de negocio del sistema.

En conjunto, esta estructura inicial proporciona el esqueleto fundamental sobre el que se construyen el resto de funcionalidades de la aplicación, asegurando una separación clara de responsabilidades y una inicialización limpia y extensible.

```
import React from 'react';
import { BrowserRouter as Router } from 'react-router-dom';
import AppRoutes from './routes/AppRoutes';
import { AuthProvider } from './context/AuthContext';
import { GoogleOAuthProvider } from '@react-oauth/google';

const clientId = import.meta.env.VITE_GOOGLE_CLIENT_ID;

const App = () => {
  return (
    <div className="app">
      <GoogleOAuthProvider clientId={clientId}>
        <AuthProvider>
          <Router>
            <AppRoutes />
          </Router>
        </AuthProvider>
      </GoogleOAuthProvider>
    </div>
  );
};

export default App;
```

Figura 4.15 Fragmento de código de inicialización en main.tsx

2. Gestión de rutas y control de acceso

La gestión de rutas en el frontend se realiza mediante la librería **React Router**, que permite definir y controlar la navegación entre las distintas vistas del sistema de manera declarativa. El componente `AppRoutes`, ubicado en `src/routes/AppRoutes.tsx`, centraliza toda la configuración de rutas de la aplicación y aplica mecanismos de protección según el estado de autenticación del usuario.

En primer lugar, se define una función `RootRedirect` que redirige al usuario cuando carga la página a la ruta correspondiente en función de su rol: si es administrador, se le dirige al panel de administración; si es usuario estándar, a la página de productos; y si no ha iniciado sesión, se muestra la pantalla de bienvenida (`SplashScreen`).

```
const RootRedirect = () => {  
  if (isLoading) {  
    return <div>Loading...</div>;  
  }  
  if (user && isAdmin()) {  
    return <Navigate to="/admin/dashboard" replace />;  
  }  
  if (user) {  
    return <Navigate to="/products" replace />;  
  }  
  return <Splash />;  
};
```

Figura 4.16 Función para RootRedirect

A continuación, se definen las rutas mediante el componente `<Routes>`:

- Las rutas públicas, como `/login`, `/register` o `/introduction`, están accesibles para cualquier visitante.
- Las rutas que requieren autenticación están anidadas dentro del componente `<ProtectedRoute />`, que se encarga de verificar si el usuario ha iniciado sesión.
- Del mismo modo, las rutas exclusivas para administradores están contenidas dentro de `<AdminRoute />`, que valida que el usuario posea privilegios de administración.

```
return (  
  <Routes>  
    <Route path="/" element={<RootRedirect />} />  
    <Route path="/introduction" element={<Introduction />} />  
  
    <Route element={<ProtectedRoute />>  
      <Route path="/products/:id" element={<ProductDetails />} />  
      <Route path="/products" element={<ProductsMenu />} />  
      <Route path="/favourites" element={<Favourites />} />  
    </Route>  
  
    <Route element={<AdminRoute />>  
      <Route path="/admin/dashboard" element={<AdminDashboard />} />  
      <Route path="/admin/users" element={<UserManagementPage />} />  
      <Route path="/admin/reports" element={<ReportGeneratorPage />} />  
    </Route>  
  
    <Route path="/login" element={<Login />} />  
    <Route path="/register" element={<Register />} />  
    <Route path="*" element={<NotFound />} />  
  </Routes>  
)
```

Figura 4.17 Declaración de las rutas

El componente `ProtectedRoute` (ubicado en `src/components/ProtectedRoute.tsx`) se encarga de comprobar si existe un usuario autenticado antes de permitir el acceso a determinadas rutas. Si el usuario no está autenticado, se le redirige a la vista de inicio de sesión, almacenando la ruta original desde la que accedió para redirigirlo correctamente tras iniciar sesión.

Además, este componente también incluye una verificación adicional para impedir que un usuario sin privilegios intente acceder manualmente a rutas administrativas.

```
const ProtectedRoute = () => {
  const { user, isLoading, isAdmin } = useAuth();
  const location = useLocation();

  if (isLoading) {
    return <div>Loading...</div>;
  }

  if (!user) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  const isAdminRoute = location.pathname.startsWith('/admin');

  if (isAdminRoute && !isAdmin()) {
    return <Navigate to="/products" replace />;
  }

  return <Outlet />;
};

export default ProtectedRoute;
```

Figura 4.18 Lógica del componente ProtectedRoute

Por su parte, el componente AdminRoute (src/components/AdminRoute.tsx) actúa como un filtro exclusivo para usuarios con rol de administrador. Si el usuario no posee el rol adecuado, se redirige automáticamente a la sección general de productos. En caso contrario, se renderiza el contenido mediante el componente <Outlet />, que actúa como contenedor para las rutas hijas.

```
const AdminRoute = () => {
  const { isAdmin } = useAuth();

  if (!isAdmin()) {
    return <Navigate to="/products" replace />;
  }

  return <Outlet />;
};

export default AdminRoute;
```

Figura 4.19 Lógica del componente AdminRoute

Esta estructura permite controlar de manera sencilla y centralizada el acceso a las distintas partes del sistema, garantizando una experiencia segura y adaptada al perfil del usuario. Al utilizar componentes de orden superior para la protección de rutas, el código se mantiene limpio, reutilizable y coherente con los principios de diseño modular adoptados en el proyecto.

3. Gestión de autenticación y sesión

La autenticación de usuarios en la aplicación se gestiona mediante un sistema centralizado basado en un contexto global (AuthContext) y un conjunto de servicios que interactúan con la API del backend. El objetivo de este sistema es ofrecer una experiencia persistente para el usuario a lo largo de su navegación por la plataforma.

Contexto de autenticación

El componente **AuthContext** encapsula toda la lógica de autenticación. A través del AuthProvider, se ofrece a toda la aplicación acceso a:

- La información del usuario autenticado (user).
- Un estado de carga (isLoading) para controlar comportamientos mientras se resuelve la sesión.
- Funciones como login, logout e isAdmin, que permiten establecer la sesión, cerrarla y comprobar si el usuario tiene permisos administrativos.

Este contexto se inicializa al arrancar la aplicación e intenta recuperar automáticamente el token JWT desde localStorage, decodificándolo con jwt-decode para establecer el estado de sesión. Esto asegura que la sesión persista incluso tras recargar la página.

```
export const AuthProvider = ({ children }: { children: React.ReactNode }) => {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  const login = (token: string): User => { ...
  };

  const logout = async () => { ...
  };

  const isAdmin = () => { ...
  };

  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) {
      try {
        const decoded = jwtDecode<User>(token);
        setUser(decoded);
      } catch (error) {
        console.error('Error decoding token:', error);
        localStorage.removeItem('token');
      }
    }
    setIsLoading(false);
  }, []);

  return ( ...
  );
};
```

Figura 4.20 Funciones e inicialización del archivo Authcontext

Autenticación con email y contraseña

En la vista de login, el usuario puede autenticarse introduciendo su correo y contraseña. El componente Login.tsx captura el formulario, realiza una validación básica y envía los datos a través de AuthService.login. Si el backend responde correctamente, se almacena el token devuelto y se decodifica para redirigir al usuario según su rol.

```
const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();

  if (!email || !password) {
    setError('Please enter both email and password.');
    return;
  }

  try {
    const response = await AuthService.login({ email, password });
    console.log('Login successful:', response);

    const user = login(response.access_token);

    if (user?.role === 'admin') {
      navigate('/admin/dashboard');
    } else {
      navigate('/products');
    }
  } catch (err) {
    if (err.response && err.response.status === 401) {
      setError('Invalid email or password.');
```

Figura 4.21 Lógica de autenticación tradicional

Autenticación con Google y GitHub

La aplicación también ofrece autenticación mediante OAuth con Google y GitHub:

En el caso de Google, se utiliza el componente `GoogleLogin`, que devuelve un token que se envía al backend mediante `AuthService.googleAuth`.

Para GitHub, se construye una URL de autorización que redirige al usuario a la plataforma de GitHub. Una vez que el usuario concede los permisos, GitHub redirige de nuevo a la aplicación con un código, que es capturado y procesado por `handleGitHubCallback`.

Ambas opciones siguen el mismo patrón: tras recibir un token del backend, este se almacena localmente y se actualiza el estado del contexto de autenticación.

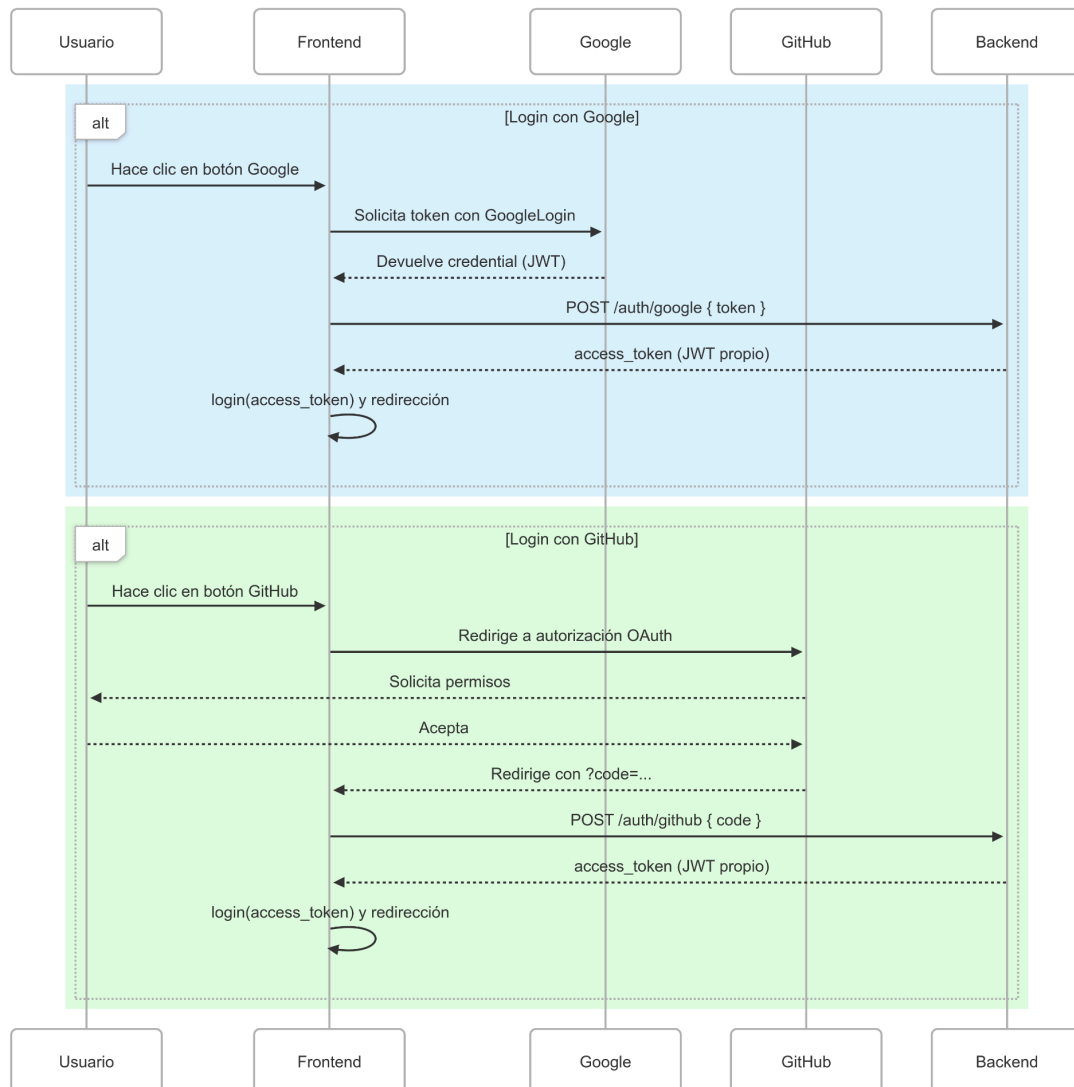


Figura 4.22 Proceso de autenticación con proveedores externos

Registro de nuevos usuarios

La vista Register.tsx permite al usuario crear una cuenta introduciendo su correo y una contraseña. Además de validaciones sobre el formato de email y la fortaleza de la contraseña, se incluye un checkbox para aceptar la política de privacidad. El formulario utiliza AuthService.register para enviar los datos al servidor. Si el proceso es exitoso, el sistema procede automáticamente a iniciar sesión.

También se puede registrar con Google o GitHub, de forma similar al login. En todos los casos, una vez autenticado, el usuario es redirigido a la vista correspondiente según su rol.

4. Comunicación con la API

La comunicación entre el frontend y el backend se centraliza mediante el módulo `apiClient`, construido con la biblioteca **Axios** [8], una herramienta que facilita la realización de solicitudes HTTP asincrónicas de forma simple y configurable. Este cliente define una URL base, añade automáticamente el token JWT en las cabeceras si está disponible, y registra los datos enviados y recibidos para facilitar la depuración. Todos los servicios del frontend que se encuentran en la carpeta `services` (Figura 4.6), como `authService` o `chatService`, utilizan este cliente para interactuar con los distintos endpoints del backend. De este modo, se garantiza una gestión unificada, segura y escalable de las peticiones HTTP en toda la aplicación.

```
import axios from 'axios';

const apiClient = axios.create({
  baseURL: `${import.meta.env.VITE_API_BASE_URL}`,
  timeout: 30000,
});

apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  console.log('Realizando solicitud:', config.method?.toUpperCase(), config.url);
  console.log('Datos de la solicitud:', config.data);
  return config;
}, (error) => {
  console.error('Error en la solicitud:', error);
  return Promise.reject(error);
});

apiClient.interceptors.response.use((response) => {
  console.log('Respuesta recibida:', response.status, response.data);
  return response;
}, (error) => {
  console.error('Error en la respuesta:', error.response?.status, error.message);
  return Promise.reject(error);
});

export default apiClient;
```

Figura 4.23 Archivo `apiClient.tsx`

5. Página de productos

La página de productos es una de las vistas principales del sistema ([Figura 4.74](#)), y permite a los usuarios explorar, filtrar y buscar entre los productos. Su implementación combina lógica de negocio, interacciones del usuario y comunicación con el backend, y se apoya en componentes como `ProductsDisplay`, `Header`, `ChatBubble` y `FilterButton`.

La lógica principal se encuentra en el archivo `ProductsMenu.tsx`, donde se definen múltiples estados locales para gestionar la búsqueda, los filtros activos (por categoría, tienda y precio), las sugerencias de autocompletado y las búsquedas recientes del usuario. Para ello, se utilizan hooks personalizados como `useProductsMenu`, para la carga inicial, así como funciones auxiliares del servicio `productService`.

A nivel de interacción, la búsqueda de productos se activa al introducir una consulta en el input correspondiente. Si el usuario hace clic en la barra de búsqueda del header, se activa el modo búsqueda. Cuando pulsa Enter, se envía una petición POST al backend utilizando el servicio `searchProducts` (que usa el motor de búsqueda en el backend), y se almacenan los resultados junto con el tiempo de ejecución en la base de datos para trazabilidad. En la siguiente figura se muestra el bloque de código encargado de realizar esta operación.

```
const handleSearchSubmit = async (e: React.KeyboardEvent<HTMLInputElement>) => {
  if (e.key === 'Enter' && searchQuery.trim()) {
    try {
      setSearchLoading(true);
      setSuggestions([]);
      setRecentQueries([]);
      setGridMinHeight();

      const startTime = performance.now();
      const searchResults = await searchProducts(searchQuery.trim());
      const endTime = performance.now();
      const executionTime = (endTime - startTime) / 1000;

      setProducts(searchResults.top_products || []);
      await userService.saveQuery(searchQuery.trim(), executionTime);
    } catch (error) {
      console.error('Error in search:', error);
      setProducts([]);
    } finally {
      setSearchLoading(false);
    }
  }
};
```

Figura 4.24 Búsqueda manual con medición del tiempo de respuesta

En paralelo, se ofrecen sugerencias dinámicas de autocompletado, obtenidas mediante la función `autocompleteProducts`, que realiza llamadas GET a un endpoint específico. El siguiente fragmento muestra cómo se implementa esta lógica.

```
const handleSearchChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const query = e.target.value;
  setSearchQuery(query);

  if (searchTimeoutRef.current) {
    clearTimeout(searchTimeoutRef.current);
  }

  if (query.trim().length >= 2) {
    searchTimeoutRef.current = setTimeout(() => {
      handleAutocomplete(query);
    }, 500);
  } else {
    setSuggestions([]);
  }
};
```

Figura 4.25 Mecanismo de autocompletado al escribir en el campo de búsqueda

Además, esta lógica de búsqueda no se limita al input manual: cuando el usuario realiza una consulta al chatbot (componente `ChatBubble`), el sistema procesa su mensaje y

devuelve también una lista de productos sugeridos cuando el usuario lo solicita. Estos se cargan directamente en la vista utilizando el mismo mecanismo que en la búsqueda manual, lo que permite aplicar posteriormente los mismos filtros sobre resultados provenientes del chat o del input tradicional.

Los resultados —ya sean por búsqueda o navegación por categorías— se visualizan mediante el componente `ProductsDisplay`, el cual recibe un listado de productos y los organiza en una rejilla. Internamente, `ProductsDisplay` utiliza el componente `ProductCard` para representar individualmente cada producto, mostrando su nombre, imagen, tienda, precio, valoración y un botón de favorito. Esta estructura facilita la navegación visual y permite interacciones como el marcado o desmarcado de favoritos.

Cuando no hay una búsqueda activa, los productos se agrupan por categorías populares como `Appliances`, `Videogames` o `Clothes`. Además, si el usuario aplica algún filtro sin haber buscado previamente, se lanza automáticamente una petición `getProducts`, que acepta parámetros como el rango de precio, la tienda o la categoría.

Este enfoque modular y unificado garantiza que todas las formas de explorar productos —ya sea por búsqueda, filtrado o interacción conversacional— compartan una experiencia visual coherente y dinámica.

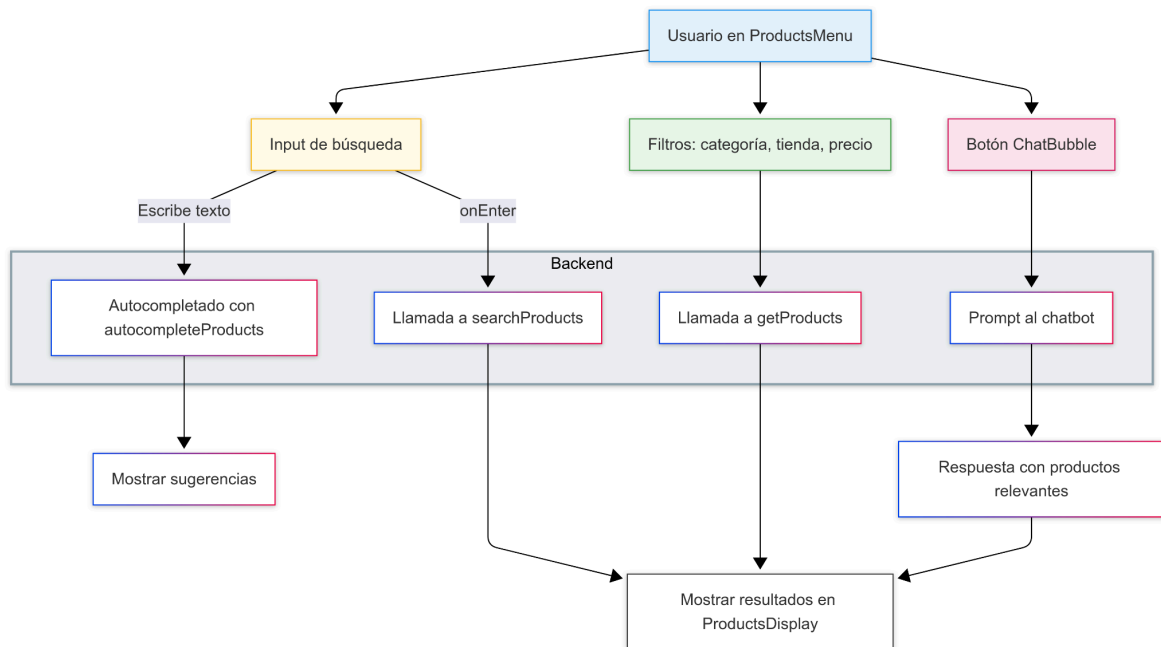


Figura 4.26 Funcionamiento de la pestaña principal de Productos

La obtención de datos del backend se realiza mediante funciones del servicio productService, el cual se comunica con la API a través del cliente apiClient (basado en Axios). Este cliente añade automáticamente el token JWT a cada solicitud si existe en el almacenamiento local, asegurando que todas las operaciones estén autenticadas. Entre las funciones expuestas por productService se incluyen: getProducts, getCategories, searchProducts, autocompleteProducts, getProductById y getProductReviews.

```
export const getProductById = async (productId: string) => {--
};

export const getProductReviews = async (productId: string, page: number = 1) => {--
};

export const searchProducts = async (query: string, top_n?: number, category?: string, min_price?: number, max_price?: number) => {--
};

export const autocompleteProducts = async (searchTerm: string, limit: number = 3) => {--
};

export const getCategories = async () => {--
};

export const getProductCount = async (): Promise<number> => {--
};

export const getMostFavoritedProducts = async (startDate: string, endDate: string, limit: number = 10) => {--
};
```

Figura 4.27 Funciones del servicio de productos

Este diseño modular permite una experiencia rica para el usuario, que puede buscar por texto libre, filtrar por tienda, categoría o precio, y consultar recomendaciones inteligentes basadas en lenguaje natural, todo desde una única interfaz coherente.

6. Detalles de producto

La vista de detalles de producto permite al usuario explorar en profundidad las características, imágenes y reseñas asociadas a un producto concreto. Esta pantalla se carga dinámicamente a partir del identificador recibido en la URL, a continuación, se ejecuta un **useEffect** —un hook de React que permite realizar efectos secundarios como llamadas a APIs tras el renderizado del componente— para realizar las peticiones necesarias al backend, como se muestra en la siguiente figura:

```
useEffect(() => {  
  const fetchProductDetails = async () => {  
    const productData = await getProductById(id!);  
    setProduct(productData);  
    setSelectedImage(productData.images[0]);  
  };  
  
  const fetchProductReviews = async (page: number) => {  
    const reviewsData = await getProductReviews(id!, page);  
    setReviews(reviewsData.reviews || []);  
    setTotalPages(reviewsData.total_pages || 1);  
  };  
  
  fetchProductDetails();  
  fetchProductReviews(currentPage);  
}, [id, currentPage]);
```

Figura 4.28 Carga inicial de detalles y reseñas

A partir de este, se ejecutan dos peticiones al backend: una para recuperar los detalles del producto (`getProductById`) y otra para obtener sus reseñas paginadas (`getProductReviews`) estos están definidos en el servicio `productsService` ([Figura 4.25](#)).

La interfaz está compuesta por dos columnas principales ([Figura 4.78](#)). En el lateral izquierdo, se presenta un carrusel de imágenes con miniaturas navegables mediante flechas y clic. Al seleccionar una imagen, esta se amplía en el área principal. En el lateral derecho se muestran el título, precio, valoración media, número de reseñas y descripción, además de una lista con características destacadas y un botón que redirige a la página del producto en Amazon.

Debajo del contenido principal se encuentra la sección de reseñas, dispuestas en un contenedor horizontal desplazable, implementada como se muestra en la [Figura 4.27](#), permitiendo una navegación fluida con interacción directa.

```
<div
  className="reviews-container"
  ref={reviewsContainerRef}
  onMouseDown={handleMouseDown}
  onMouseMove={handleMouseMove}
  onMouseUp={handleMouseUp}
  onMouseLeave={handleMouseUp}
  style={{ cursor: isDragging ? 'grabbing' : 'grab' }}
>
  {reviewsLoading ? (
    <div>Loading reviews...</div>
  ) : reviews.length > 0 ? (
    <div className="reviews-carousel">
      {reviews.map((review, index) => (
        <div
          id={`review-${review.review_id}`}
          key={index}
          className={`review-card ${
            highlightedReviewIds.includes(review.review_id) ? 'highlighted' : ''
          }}
        >
          <h3>{review.title}</h3>
          <div className="review-text" dangerouslySetInnerHTML={{ __html: review.text }} />
          <div style={{ display: 'flex', justifyContent: 'center', alignItems: 'center' }}>
            <Rating
              name={`review-rating-${index}`}
              value={review.rating}
              precision={0.1}
              readOnly
              size="small"
            />
          </div>
        </div>
      )
    )}
    </div>
  ) : null}
</div>
```

Figura 4.29 Componente de reseñas con scroll

El usuario puede navegar entre páginas y desplazar lateralmente las reseñas mediante interacción directa con el ratón. Si el usuario ha utilizado el asistente conversacional (componente ChatBubble), algunas reseñas pueden aparecer resaltadas para indicar que fueron relevantes según su consulta. Para facilitar la navegación, el sistema incluye una funcionalidad de scroll automático para reseñas específicas destacadas.

El componente ChatBubble también está presente en esta vista, permitiendo realizar preguntas contextuales sobre el producto y obtener respuestas generadas por IA. Si el chatbot considera que ciertas reseñas son relevantes para la consulta, puede resaltarlas

dinámicamente y desplazar la vista hacia ellas, reforzando la integración entre los datos extraídos del backend y las capacidades conversacionales del sistema [Figura 4.83](#).

Gracias a este diseño, la vista de detalle proporciona una experiencia rica e interactiva que combina información estructurada con inteligencia artificial, permitiendo al usuario tomar decisiones informadas de forma visual e intuitiva.

7. Páginas de administración

La aplicación incluye un conjunto de interfaces dedicadas exclusivamente a la administración del sistema, accesibles únicamente por usuarios con rol de administrador. Estas páginas permiten monitorizar la actividad de los usuarios, gestionar sus permisos, analizar patrones de uso y generar informes descargables.

Implementación general

Las páginas de administración ofrecen al usuario con rol de administrador un conjunto de funcionalidades orientadas a la monitorización del sistema y a la gestión de usuarios y productos ([Figura 4.82](#)). Estas páginas presentan datos agregados a partir del backend y permiten visualizar patrones de comportamiento mediante gráficos dinámicos y mapas interactivos. Entre sus características principales se encuentra un mapa de calor que muestra las ubicaciones desde las que los usuarios acceden a la aplicación, gráficos que analizan el rendimiento de las consultas y la distribución de productos más guardados, así como un generador de informes en formato PDF configurable por parámetros como rango de fechas, roles o categorías. Además, se incluye una interfaz para la gestión directa de usuarios, que permite modificar su rol o eliminarlos del sistema.

Heatmap de accesos

Uno de los elementos más representativos de esta sección es el heatmap, que permite visualizar geográficamente los puntos de acceso a la aplicación. Su implementación se basa en Leaflet, una biblioteca JavaScript ligera y de código abierto para la creación de mapas interactivos en aplicaciones web [\[23\]](#). Leaflet permite renderizar mapas mediante

tiles (fragmentos de mapa) obtenidos de proveedores como OpenStreetMap, que actúa como mapa base mundial sobre el que se representa la información. Encima de este mapa base se superpone una capa de calor (heatLayer) que indica las zonas con mayor actividad. Esta capa se genera a partir de coordenadas geográficas y un valor de peso asociado que representa la intensidad o frecuencia de los accesos, y se alimenta con los datos proporcionados por el backend en tiempo real. Los datos pueden filtrarse por rango temporal utilizando un deslizador o botones preconfigurados. El mapa incluye también soporte para pantalla completa y ajuste automático del zoom y encuadre, de modo que se enfoca en las regiones con mayor concentración de actividad. El siguiente fragmento de código ([Figura 4.28](#)) muestra cómo se implementa esta funcionalidad dentro del componente de administración, incluyendo la inicialización del mapa, la carga dinámica de la capa de calor, el ajuste automático de la vista y la limpieza del recurso cuando el componente se desmonta para evitar fugas de memoria.

```
useEffect(() => {
  if (!mapRef.current) {
    const map = L.map('map', {
      fullscreenControl: true,
      fullscreenControlOptions: {
        position: 'topleft',
        title: {
          'false': 'View in fullscreen',
          'true': 'Exit fullscreen'
        }
      }
    });
    map.setView([20, 0], 2);
    mapRef.current = map;
    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
      attribution: '© <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors',
      noWrap: true
    }).addTo(map);
  }

  if (mapRef.current && heatmapData.length > 0) {
    if (heatLayerRef.current) {
      mapRef.current.removeLayer(heatLayerRef.current);
    }
    const heatPoints = heatmapData.map(point => [point.lat, point.lng, point.weight]);
    heatLayerRef.current = (L as any).heatLayer(heatPoints, {
      radius: 25,
      blur: 15,
      maxZoom: 17,
      minOpacity: 0.5
    }).addTo(mapRef.current);
    const bounds = L.latLngBounds(heatmapData.map(point => [point.lat, point.lng]));
    mapRef.current.fitBounds(bounds, { padding: [50, 50] });
  }

  return () => {
    if (mapRef.current) {
      mapRef.current.remove();
      mapRef.current = null;
      heatLayerRef.current = null;
    }
  };
}, [heatmapData]);
```

Figura 4.30 Inicialización del mapa de calor

Gráficos estadísticos

El panel de administración también permite representar métricas mediante gráficos interactivos que se actualizan dinámicamente en función de los datos obtenidos desde el backend. Se destacan dos visualizaciones principales:

- Evolución temporal del rendimiento: un gráfico de líneas muestra el tiempo de ejecución de las consultas realizadas por los usuarios, con líneas adicionales que

representan los percentiles 10, 50 y 90, lo que permite contextualizar la variabilidad del rendimiento del sistema.

- Distribución por categoría: se emplea un gráfico circular para representar la proporción de productos más guardados por los usuarios según su categoría principal, con datos agregados y filtrables por rango de fechas.

Además de las visualizaciones predefinidas, el panel permite generar nuevos gráficos de forma dinámica a partir de las consultas realizadas en el chatbot integrado ([Figura 4.88](#)). Mediante lenguaje natural, el administrador puede solicitar representaciones personalizadas —por ejemplo, diagramas de barras, líneas o sectores sobre datos específicos—, que se renderizan automáticamente en la interfaz y pueden eliminarse individualmente según convenga. Esta funcionalidad se apoya en una estructura flexible basada en un array de gráficos (`chartData`), donde cada entrada contiene el tipo de gráfico, los datos a representar y las opciones de configuración. Durante el renderizado, el sistema recorre esta lista y utiliza una función (`renderChart`) que selecciona el componente gráfico correspondiente de acuerdo con el tipo solicitado, renderizándolo en el cliente con la biblioteca `Chart.js` [\[11\]](#), a través de su integración con `react-chartjs-2`. Esta aproximación permite una experiencia de análisis altamente interactiva y adaptada a las necesidades del administrador.

El siguiente fragmento de código ilustra el proceso de renderizado de estos gráficos dinámicos, incluyendo la lógica condicional para elegir el tipo de gráfico adecuado y la interfaz para eliminar individualmente cada visualización:

```
const renderChart = (chart: { chartType: string; data: any; options: any }) => {
  switch (chart.chartType) {
    case 'bar':
      return <Bar data={chart.data} options={chart.options} />;
    case 'line':
      return <Line data={chart.data} options={chart.options} />;
    case 'pie':
      return <Pie data={chart.data} options={chart.options} />;
    case 'doughnut':
      return <Pie data={chart.data} options={chart.options} />;
    default:
      return <Pie data={chart.data} options={chart.options} />;
  }
};

const removeChart = (id: string) => {
  setChartData(prev => prev.filter((chart) => chart.id !== id));
};

const stats = [
  { title: 'Total Users', value: userCount ?? 'Loading...', change: userCount !== null ? '+12%' : '', trend: 'up' },
  { title: 'Products', value: productCount ?? 'Loading...', change: productCount !== null ? '+5%' : '', trend: 'up' },
  {
    title: 'Products saved today',
    value: productsSavedToday ?? 'Loading...',
    change: productsSavedToday !== null ? '-2%' : '',
    trend: 'down'
  },
  { title: 'Income', value: '$12,345', change: '+18%', trend: 'up' }
];
```

Figura 4.31 Renderizado de gráficos personalizados a partir de consultas del chatbot

Generación de informes

Otra funcionalidad destacada es la generación de informes personalizados en formato PDF ([Figura 4.87](#)). Esta característica está implementada con la biblioteca jsPDF [\[21\]](#), junto a autoTable [\[50\]](#), permitiendo crear documentos estructurados que incluyen encabezados, tablas con datos filtrados y pie de página automático. El sistema permite generar dos tipos principales de informes:

- Informe de actividad de usuarios: recoge información como el número de productos guardados, consultas realizadas, rol, fecha de creación y último acceso de cada usuario. Puede filtrarse por rango de fechas y por tipo de rol, lo que facilita el análisis de comportamiento de distintos perfiles dentro del sistema.
- Informe de popularidad de productos: muestra una tabla con los productos más relevantes según distintos criterios como categoría, precio, valoración, número de veces guardado, número de reseñas y ordenación seleccionada. Este informe es útil para identificar tendencias de interés entre los usuarios.

Los informes pueden configurarse mediante formularios en la interfaz administrativa o generarse directamente a través del chatbot utilizando lenguaje natural, lo que aporta una vía alternativa más rápida y flexible. Una vez definidos los parámetros, el informe se genera y descarga automáticamente en el navegador. En este proceso, se estructura el contenido del documento PDF utilizando las bibliotecas jsPDF y autoTable. Se añaden manualmente los parámetros del informe como texto explicativo y se genera una tabla que resume los datos relevantes por usuario, como su rol, fecha de creación, número de favoritos, consultas realizadas y último acceso. Además, se aplican estilos visuales personalizados en cabeceras, celdas y espaciado, lo que mejora la claridad y presentación del documento final.

El fragmento mostrado en la en la siguiente figura ilustra este proceso de construcción del informe de actividad de los usuarios, incluyendo la definición del contenido textual y tabular junto con los estilos aplicados.

```
addHeaderFooter(doc, 'User Activity Report', pageNumber);

doc.setFontSize(12);
doc.setTextColor(40);
doc.text('Report Parameters', 20, 45);
doc.setFontSize(10);
doc.setTextColor(60);
doc.text('Login Date Range: ${startDate || 'All'} to ${endDate || 'All'}', 20, 55);
doc.text('Role: ${role || 'All'}', 20, 65);

autoTable(doc, {
  startY: 75,
  head: [['ID', 'Email', 'Role', 'Created At', 'Favorites', 'Queries', 'Last Login']],
  body: userDetails.map(user => [
    user.id,
    user.email || 'N/A',
    user.role,
    user.created_at.split('T')[0],
    user.favoriteCount,
    user.queryCount,
    user.lastLogin.split('T')[0] || 'N/A',
  ]),
  styles: { fontSize: 10, cellPadding: 3, textColor: [60], fillColor: [245, 245, 245] },
  headStyles: { fillColor: [40, 74, 112], textColor: [255], fontStyle: 'bold' },
  alternateRowStyles: { fillColor: [230, 230, 230] },
  margin: { top: 75, left: 20, right: 20 },
  didDrawPage: (data: any) => {
    pageNumber++;
    addHeaderFooter(doc, 'User Activity Report', pageNumber);
  }
});
```

Figura 4.32 Construcción del informe de actividad de usuarios en PDF

Gestión de usuarios

Por último, se incluye una página específica para la gestión de usuarios, donde el administrador puede buscar por correo o ID, cambiar el rol (entre user y admin) y eliminar usuarios si es necesario. Todos los cambios se sincronizan con la base de datos a través de peticiones API.

8. Chatbot

El componente ChatBubble implementa el chatbot conversacional del sistema, funcionando como una interfaz flotante que permite a los usuarios interactuar mediante lenguaje natural. Está diseñado para integrarse con modelos de lenguaje (LLMs) desplegados en el backend, y es capaz de gestionar respuestas en tiempo real, contenido multimedia, gráficos y navegación hacia secciones de la interfaz como reseñas o productos recomendados.

Estructura general

Internamente, ChatBubble mantiene el estado de la conversación a través de un array de mensajes (messages), donde cada entrada incluye el remitente (user o bot), el contenido del mensaje y metadatos adicionales como la hora, posibles IDs de reseñas destacadas o productos sugeridos.

Además, el componente maneja el estado de carga (isLoading), el mensaje actual en el input (currentMessage) y una bandera para mostrar el menú inicial (showMenuPage), definido en el componente ChatMenuPage.

Comunicación con el backend y manejo de respuestas en chunks

Uno de los aspectos más destacados de este componente es el tratamiento de respuestas por streaming, que permite que el bot envíe su respuesta en bloques incrementales (chunks) en lugar de esperar al mensaje completo. Esto mejora la fluidez de la interacción y proporciona retroalimentación más inmediata al usuario. El flujo es el siguiente:

```
try {  
  const reader = await chatService.queryChat(currentMessage, productId);  
  const decoder = new TextDecoder();
```

Figura 4.33 Llamada al backend para usar el servicio del chat

- Se obtiene un **ReadableStreamDefaultReader** desde chatService, que conecta con el backend mediante fetch y permite leer progresivamente los datos de la respuesta del servidor sin esperar a que llegue completa. Este lector es parte de la API Streams de JavaScript, diseñada para manejar grandes cantidades de datos de forma eficiente, y en nuestro caso nos permite el stream de las respuestas.
- Se utiliza un TextDecoder, que transforma los datos binarios (**Uint8Array**) en texto legible, ya que el backend envía los mensajes codificados como cadenas JSON dentro de un stream binario.

A continuación, se entra en un bucle donde se procesan los chunks:

```
while (true) {  
  const { done, value } = await reader.read();  
  if (done) break;  
  
  const chunk = decoder.decode(value, { stream: true });  
  
  if (chunk) {  
    try {  
      const parsedChunk = JSON.parse(chunk);
```

Figura 4.34 Procesamiento de los chunks recibidos desde el backend

Cada bloque chunk puede representar:

- Un texto parcial de la respuesta del bot.

- Un mensaje especial con type: 'additional_data' que contiene productos, reviews o gráficos.
- Un estado (type: 'status') que indica que el mensaje es informativo (por ejemplo, "generating chart...").

```
{
  "type": "additional_data",
  "data": {
    "review_ids": [102, 105],
    "products": [
      { "product_id": 12, "title": "Zapatillas Trail X", "price": 79.99 },
      { "product_id": 27, "title": "Mochila Trek Pro", "price": 45.50 }
    ]
  }
}
```

Figura 4.35 Ejemplo de un chunk de additional data

```
{
  "type": "status",
  "message": "Buscando productos similares..."
}
```

Figura 4.36 Ejemplo de un chunk de status

Fragmentación y ensamblado del mensaje del bot

Uno de los elementos clave de esta lógica es el tratamiento de texto incompleto en los chunks. Por ejemplo, si el backend va generando progresivamente una respuesta como:

- "Este producto es muy"
- " cómodo y resistente. Lo usé durante"
- " una caminata de 10 km y funcionó perfectamente."

El sistema irá concatenando estos trozos en el mismo mensaje del bot, actualizando dinámicamente en pantalla para ofrecer una experiencia fluida.

```
botMessage.text += chunk;  
setMessages((prev) => [...prev.slice(0, -1), { ...botMessage }]);
```

Figura 4.37 Impresión de los chunks recibidos desde el backend

Esto se gestiona a través del estado `botMessage.text`, que se va extendiendo con cada bloque recibido antes de actualizar la lista de mensajes con `setMessages`.

Reacciones a mensajes especiales

Cuando se detectan datos adicionales (`additional_data`), el componente no los maneja directamente, sino que delega la gestión a métodos definidos en `AdminPage`. Estos métodos se activan mediante los callbacks `onResponse`, `onReportGenerate` o `onChartData`, y permiten accionar funcionalidades avanzadas como:

- Generación de gráficos: como se muestra en la [Figura 4.29](#), el método que interpreta la configuración del gráfico (recibida como `chart_config`) y lo renderiza en el área correspondiente se implementa dentro de `AdminPage`.
- Generación de informes: tal como se ilustra en la [Figura 4.30](#), el procesamiento del `report_type` y sus parámetros también se realiza desde `AdminPage`, permitiendo generar documentos o visualizaciones específicas según el tipo de informe solicitado.
- Inserción de productos sugeridos: cuando el LLM devuelve productos relevantes como respuesta, se integran visualmente en la interfaz. En la [Figura 4.25](#) se muestra cómo añadirían los productos sugeridos al usuario.

UI y usabilidad

Visualmente, el chatbot se organiza en dos fases:

1. Menú inicial (ChatMenuPage): se presenta un selector de proveedor (OpenAI, Claude, Mistral, etc.), almacenado en localStorage.
2. Interfaz de chat: una vez iniciada la conversación, se muestra el historial de mensajes y un campo de entrada. Se pueden enviar mensajes con Enter o botón, y el sistema gestiona indicadores de carga (<ThreeDots />).

Además, si el mensaje del bot incluye referencias a reseñas (reviewIds), se generan botones para hacer scroll directamente a esas secciones, integrando así el chatbot con el resto de la experiencia de usuario

4.4.2 IMPLEMENTACIÓN DEL BACKEND

El backend del sistema se ha desarrollado en Flask, estructurado en rutas y servicios que gestionan la lógica del chatbot, los datos de usuarios y productos, así como funcionalidades avanzadas como el heatmap y la búsqueda semántica. A continuación, se describen los componentes más relevantes.

1. Arquitectura general del backend

La arquitectura del backend de Reviewly se basa en el framework **Flask**, con una estructura modular organizada en rutas, servicios y modelos, lo que facilita la mantenibilidad y escalabilidad del sistema .

El punto de entrada principal es el archivo run.py, que simplemente lanza la aplicación Flask mediante la función create_app() definida en __init__.py. Este archivo sigue el patrón de factory method, permitiendo construir dinámicamente una instancia de la aplicación con su configuración, middleware y extensiones.

Dentro de create_app(), se configuran elementos clave como:

- CORS: se habilita para permitir peticiones desde el cliente (http://localhost:5173), incluyendo cabeceras personalizadas y autenticación.
- JWTManager: se inicializa para gestionar la autenticación basada en tokens.

- SQLAlchemy: se utiliza como ORM para la persistencia de datos.
- Carga de modelos y verificación de conexión: se instancian las tablas del sistema y se valida la conexión con la base de datos.

Una parte importante del `__init__.py` es la asociación de namespaces REST a rutas específicas mediante `flask_restx`, lo que permite estructurar la API de forma modular y documentada. La siguiente figura muestra cómo se realiza este proceso:

```
from app.routes import product_routes, review_routes, chat_routes, health_routes, auth_routes, user_queries, user_routes, heatmap_routes

api.add_namespace(product_routes.api, path=f"{API_PREFIX}/products")
api.add_namespace(review_routes.api, path=f"{API_PREFIX}/reviews")
api.add_namespace(chat_routes.api, path=f"{API_PREFIX}/chat")
api.add_namespace(health_routes.api, path="/health")
api.add_namespace(auth_routes.api, path=f"{API_PREFIX}/auth")
api.add_namespace(user_queries.api, path=f"{API_PREFIX}/user")
api.add_namespace(user_routes.api, path=f"{API_PREFIX}/user")
api.add_namespace(heatmap_routes.api, path=f"{API_PREFIX}/heatmap")

return app
```

Figura 4.38 Asociación de namespaces a las rutas

La documentación de la API se genera automáticamente y puede consultarse visualmente a través de Swagger en la ruta raíz (`/`), como se muestra a continuación:

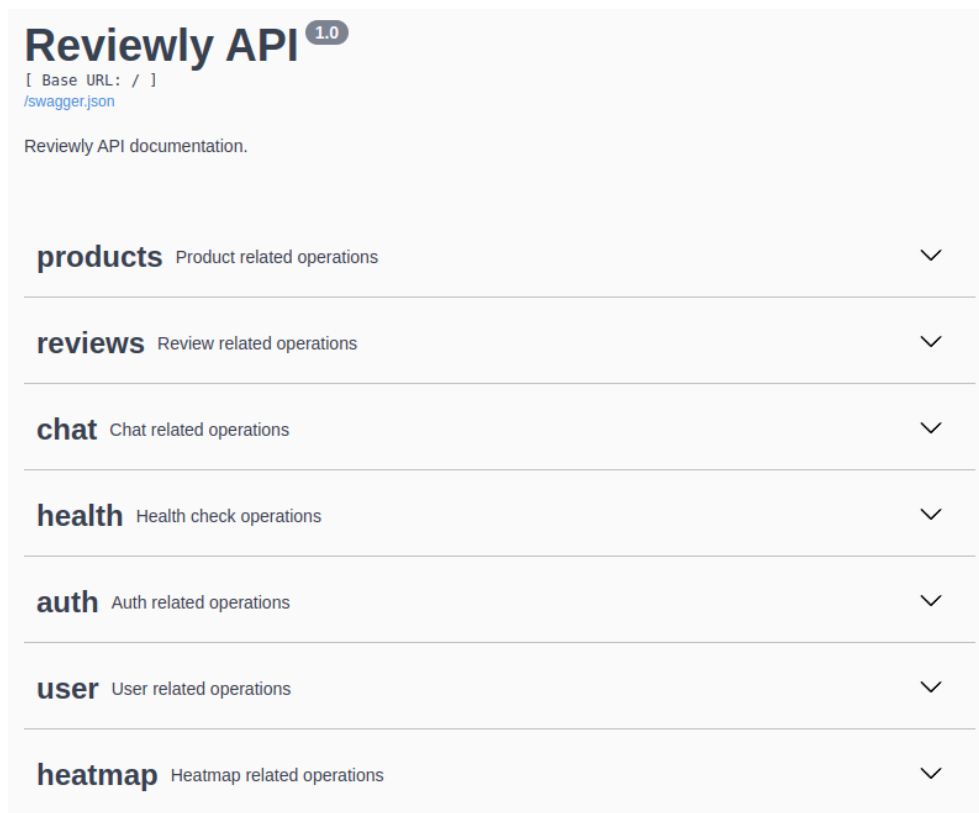


Figura 4.39 Documentación de la API de la Aplicación

Este enfoque permite a los desarrolladores y usuarios explorar fácilmente todos los endpoints disponibles, sus parámetros y respuestas, facilitando la integración con otros servicios o aplicaciones cliente.

2. Autenticación

El sistema de autenticación de Reviewly proporciona una capa segura de acceso mediante tokens JWT, permitiendo a los usuarios iniciar sesión, registrarse o autenticarse usando proveedores externos como Google y GitHub. Esta funcionalidad está centralizada en el módulo `auth_routes.py`, donde se definen las rutas relacionadas con el inicio de sesión, registro, cierre de sesión y autenticación OAuth.

Arquitectura general

A nivel de arquitectura, el sistema combina:

- Flask-JWT-Extended para la gestión de tokens JWT, necesarios para rutas protegidas.
- werkzeug.security para el hash seguro de contraseñas.
- Servicios externos de OAuth (Google y GitHub) mediante google-auth y llamadas HTTP a la API de GitHub.
- LoginLog, una tabla de la base de datos que registra cada intento de autenticación con IP y marca temporal, lo que mejora la trazabilidad del sistema.

La siguiente figura muestra el conjunto de endpoints de autenticación documentados automáticamente por Swagger:

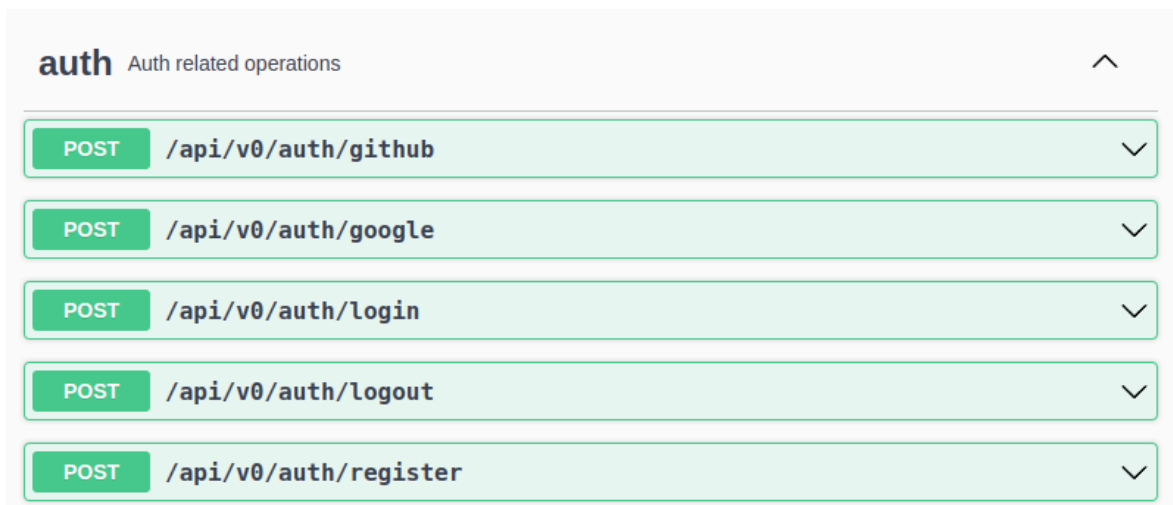


Figura 4.40 Endpoints de autenticación disponibles (Swagger)

Funcionalidad clave

/register: permite crear una nueva cuenta en el sistema. El correo se normaliza y se aplica hash a la contraseña antes de almacenar el usuario en la base de datos.

/login: valida el usuario y contraseña, y devuelve un `access_token` si son correctos. También se guarda un log del inicio de sesión.

/logout: invalida la sesión activa, además de eliminar la instancia de conversación activa del usuario en el servicio de chat.

/google y /github: gestionan la autenticación mediante tokens externos, generando un usuario nuevo si no existe o devolviendo uno existente, y entregando un JWT válido.

La lógica de negocio se encuentra en el servicio `AuthService`, desde donde se implementan las funciones `register_user`, `login_user`, `google_auth` y `github_auth`. Cada una de ellas abstrae la complejidad del flujo y encapsula el acceso a la base de datos.

Por ejemplo, en el registro local (`register_user`), el servicio:

```
def register_user(email, password):
    normalized_email = email.lower()
    if User.query.filter_by(email=normalized_email).first():
        return None, "User already exists", 409

    password_hash = generate_password_hash(password)

    new_user = User(email=normalized_email, password_hash=password_hash)
    db.session.add(new_user)
    db.session.commit()

    return new_user, None, 201
```

Figura 4.41 Registro seguro de usuario con hash de contraseña

Para la autenticación OAuth con **Google**, se recibe un token de ID en el frontend tras la autenticación del usuario. Este token se valida en el backend mediante la biblioteca `google-auth`, que comprueba su firma, emisor (`iss`) y si ha sido generado para el `GOOGLE_CLIENT_ID` correcto. Una vez verificado, se extrae la información del usuario, y si el usuario no existe en la base de datos, se crea un nuevo registro con ese email.

En el caso de GitHub, el proceso consta de dos pasos: primero, el backend envía una petición POST al endpoint oficial de GitHub (https://github.com/login/oauth/access_token) junto con el `client_id`, `client_secret` y el código recibido desde el frontend. GitHub responde con un access token que permite hacer una segunda llamada a su API pública (<https://api.github.com/user>) para obtener los datos del perfil del usuario (como id y email). Si el usuario aún no está registrado en la base de datos, se guarda como nuevo.

Una vez identificado o creado el usuario en ambos casos, se genera un access token JWT propio de la aplicación usando la utilidad `generate_access_token(user)`. Este token es el que se devuelve al frontend para autorizar las futuras peticiones del usuario dentro del ecosistema de Reviewly. De esta forma, aunque el proceso de autenticación se realice con un proveedor externo, el control de sesión y autorización se gestiona de forma interna y segura por el backend.

Este enfoque híbrido permite a Reviewly ofrecer una experiencia de acceso moderna, flexible y segura, adaptada a diferentes tipos de usuarios y proveedores externos.

3. Gestión de usuarios

La gestión de usuarios en el backend se estructura a través del namespace `user`, el cual expone múltiples endpoints RESTful para operaciones CRUD sobre usuarios, gestión de favoritos y consulta de logs de acceso. Todos los endpoints están protegidos mediante autenticación JWT, y su definición puede visualizarse en la siguiente Figura, donde se muestra el conjunto de rutas disponibles a través de la interfaz Swagger del sistema.

user		User related operations	^
GET	/api/v0/user	List all users	▼
POST	/api/v0/user	Create a new user	▼
GET	/api/v0/user/count	Get the total number of users	▼
GET	/api/v0/user/execution-times	Get execution times for user queries between two timestamps	▼
GET	/api/v0/user/favorites	Get list of user's favorite product IDs	▼
POST	/api/v0/user/favorites	Add a product to user's favorites	▼
DELETE	/api/v0/user/favorites	Remove a product from user's favorites	▼ 
GET	/api/v0/user/queries	Get the last 5 user queries	▼
POST	/api/v0/user/queries	Save a user query to their history (keeps last 5 queries)	▼
GET	/api/v0/user/{user_id}	Get a specific user	▼
DELETE	/api/v0/user/{user_id}	Delete a user	▼
PUT	/api/v0/user/{user_id}	Update a user	▼
GET	/api/v0/user/{user_id}/favorites		▼
GET	/api/v0/user/{user_id}/login-logs	Get all login logs for a specific user	▼
GET	/api/v0/user/{user_id}/queries	Get the last 5 queries for a specific user	▼

Figura 4.42 Endpoints de usuario disponibles en Swagger

A nivel de implementación, la gestión de usuarios está dividida principalmente en dos módulos: `user_routes.py`, donde se definen los endpoints y modelos asociados, y `user_service.py`, donde se encapsula la lógica de negocio.

Estructura de rutas y operaciones

El archivo `user_routes.py` define un namespace llamado `user`, bajo el cual se agrupan todas las operaciones relacionadas. Las rutas disponibles abarcan tanto la gestión de la entidad `User` como funcionalidades adicionales como la gestión de favoritos y la consulta de logs de acceso. A continuación, se detalla el propósito de cada endpoint:

`/user/favorites`

- GET: Devuelve un array con los identificadores (`product_id`) de los productos marcados como favoritos por el usuario autenticado.

- POST: Permite añadir un producto a la lista de favoritos. Requiere un JSON con la clave `product_id`.
- DELETE: Elimina un producto específico de la lista de favoritos del usuario. También requiere el campo `product_id`.

Estos métodos utilizan internamente la función `get_jwt_identity()` para identificar al usuario actual a partir del token JWT, sin tener que pasarlo como parámetro.

/user

- GET: Lista todos los usuarios del sistema. Incluye filtros configurables desde el servicio, como por rol, correo electrónico o si tienen asociada una cuenta de GitHub.
- POST: Permite crear un nuevo usuario. El cuerpo de la petición debe incluir los campos `email`, `role` y, opcionalmente, `github_id`.

/user/<user_id>

- GET: Recupera los datos de un usuario específico identificado por su `user_id`.
- PUT: Permite actualizar los datos del usuario especificado. Se puede modificar su correo, rol o GitHub ID.
- DELETE: Elimina al usuario del sistema, si existe.

Cada una de estas operaciones está protegida mediante JWT y realiza comprobaciones internas desde el servicio `UserService`.

/user/<user_id>/favorites

- GET: Recupera los productos favoritos de un usuario específico, indicado por su `user_id`. Esta ruta está pensada para usuarios con rol `admin`, ya que permite consultar información de terceros.

/user/<user_id>/login-logs

- GET: Devuelve el historial de accesos del usuario indicado, ordenado de forma descendente por fecha. Cada entrada contiene la dirección IP y el instante de inicio de sesión. Esta funcionalidad es clave para auditoría y trazabilidad.

/user/count

- GET: Devuelve el número total de usuarios registrados en el sistema. Útil para métricas y análisis administrativos.

/user/execution-times

- GET: Devuelve estadísticas de tiempo de ejecución de las consultas realizadas por los usuarios, entre dos timestamps proporcionados como parámetros de consulta.

Servicios definidos en user_service.py

La clase UserService concentra la lógica de negocio relacionada con la gestión de usuarios. Entre sus responsabilidades se encuentran la creación, modificación, eliminación y recuperación de usuarios, así como la gestión de favoritos y el filtrado por atributos.

Una de las funciones más relevantes es `create_user(email, role, github_id=None)`, que permite registrar nuevos usuarios en el sistema. Esta función valida que el rol sea válido ("user" o "admin") y que el correo no esté registrado previamente. Tras validar los datos, crea una instancia de usuario, la añade a la base de datos y confirma la operación con `commit()`:

```
@staticmethod
def create_user(email, role, github_id=None):
    """Create a new user"""
    app = create_app()
    with app.app_context():
        session = Session(db.engine)
        try:
            if role.lower() not in UserService.VALID_ROLES:
                return None, f"Invalid role. Must be one of: {'', '}.join(UserService.VALID_ROLES)}"

            if UserService.get_user_by_email(email):
                return None, "Email already exists"

            user = User(
                email=email.lower(),
                role=role.lower(),
                github_id=github_id
            )
            session.add(user)
            session.commit()
            session.refresh(user)
            return user, None
```

Figura 4.43 Método `create_user` en `UserService`

El método `get_users(...)` permite obtener un listado filtrado y ordenado de usuarios. Este método acepta parámetros opcionales como `email`, `role`, `github_id` o `has_github_id`. Además, permite aplicar filtros parciales (por ejemplo, usando `ilike` para búsqueda parcial por correo) y establecer el criterio de ordenación. Esta funcionalidad resulta fundamental para implementar vistas administrativas que requieren una gestión dinámica de usuarios.

```
def get_users(
    limit=5,
    email=None,
    email_contains=None,
    role=None,
    github_id=None,
    has_github_id=None,
    order_by='created_at',
    descending=True
):
    app = create_app()
    with app.app_context():
        query = User.query

        if email:
            query = query.filter_by(email=email.lower())

        if email_contains:
            query = query.filter(User.email.ilike(f"%{email_contains.lower()}%"))

        if role:
            if role.lower() not in UserService.VALID_ROLES:
                raise ValueError(f"Invalid role. Must be one of: {'', '.join(UserService.VALID_ROLES)}")
            query = query.filter_by(role=role.lower())

        if github_id is not None:
            query = query.filter_by(github_id=github_id)

        if has_github_id is not None:
```

Figura 4.44 Filtrado de usuarios por parámetros

Por otro lado, `add_to_favorites(user_id, product_id)` gestiona la lógica para añadir productos a la lista de favoritos del usuario. El método verifica que ambas entidades existan y que el producto no se encuentre ya en la lista de favoritos antes de proceder. Esta validación previene duplicados y mantiene la integridad del sistema:

```
@staticmethod
def add_to_favorites(user_id, product_id):
    try:
        user = User.query.get(user_id)
        product = Product.query.get(product_id)

        if not user:
            return False, "User not found"
        if not product:
            return False, "Product not found"
        if product in user.favorites:
            return False, "Product already in favorites"

        user.favorites.append(product)
        db.session.commit()
        return True, "Product added to favorites"
```

Figura 4.45 Añadir un producto a favoritos

Los métodos `remove_from_favorites(...)` y `get_user_favorite_ids(...)` complementan esta funcionalidad, permitiendo la eliminación de favoritos y la recuperación de sus IDs, respectivamente. Del mismo modo, `update_user(...)` y `delete_user(...)` proporcionan capacidades completas de mantenimiento de perfiles, incluyendo validaciones para cambios de correo y eliminación controlada mediante sesiones de base de datos.

En conjunto, esta arquitectura modular proporciona una capa sólida, segura y extensible para la administración de usuarios, facilitando tanto operaciones básicas como futuras integraciones avanzadas (por ejemplo, sistemas de roles jerárquicos, restricciones personalizadas o estadísticas de uso).

4. Manejo de consultas recientes del usuario

El sistema permite almacenar y recuperar las últimas consultas realizadas por cada usuario, facilitando así el análisis de su historial de interacción y la optimización de futuras respuestas. Esta funcionalidad se implementa principalmente en el archivo `user_queries.py` y se apoya en el modelo `UserQuery`, que representa cada consulta realizada con sus metadatos asociados.

En primer lugar, el endpoint `POST /user/queries` permite almacenar una nueva consulta. Este endpoint recibe como cuerpo un objeto JSON con los campos `query_text` (texto de la consulta) y opcionalmente `execution_time` (tiempo de ejecución en segundos). El servidor asocia esta consulta al usuario autenticado y la guarda en la base de datos. Para evitar un crecimiento descontrolado del historial, el sistema conserva únicamente las últimas cinco consultas de cada usuario, eliminando las más antiguas si es necesario:

```
try:
    new_query = UserQuery(
        user_id=current_user_id,
        query_text=query_text,
        execution_time=execution_time if execution_time is not None else None
    )
    db.session.add(new_query)

    query_count = UserQuery.query.filter_by(user_id=current_user_id).count()

    if query_count > 5:
        oldest_queries = UserQuery.query.filter_by(user_id=current_user_id)\
            .order_by(UserQuery.created_at.asc())\
            .limit(query_count - 5)\
            .all()
        for query in oldest_queries:
            db.session.delete(query)

    db.session.commit()

    return {"status": "success", "message": "Query saved"}, 200
```

Figura 4.46 Guardado de búsquedas del usuario

Complementariamente, el endpoint GET /user/queries devuelve las cinco últimas consultas del usuario autenticado, ordenadas por fecha de creación. Esta funcionalidad es útil para el usuario, el cual obtiene su historial cuando va a buscar un producto.

Adicionalmente, se implementa el endpoint GET /user/execution-times, que permite recuperar todas las consultas realizadas por cualquier usuario dentro de un intervalo de tiempo especificado por los parámetros start_time y end_time. Esto resulta especialmente útil para propósitos analíticos o de evaluación del rendimiento del sistema. El backend se encarga de validar los formatos de fecha y asegurar que el intervalo temporal sea válido antes de ejecutar la consulta.

Por último, el endpoint GET /user/count proporciona el número total de usuarios registrados en el sistema, permitiendo así monitorizar el crecimiento de la base de usuarios de la aplicación.

En conjunto, esta sección permite no solo mejorar la experiencia del usuario mediante el recuerdo de interacciones previas, sino también ofrece herramientas básicas de análisis de uso y rendimiento orientadas a la administración del sistema.

5. Gestión de productos

Esta sección describe el módulo de gestión de productos y reseñas de la aplicación, un componente esencial que permite la creación, actualización, eliminación y búsqueda de productos y reseñas asociadas. Este módulo es crucial para la interacción de los usuarios con el catálogo de productos, ofreciendo funcionalidades avanzadas como búsqueda semántica, gestión de categorías, autocompletado de términos de búsqueda y manejo de reseñas. A continuación, se detallan los endpoints de la API, el funcionamiento del motor de búsqueda `searchProduct`, el proceso de creación de productos y reseñas, la generación de embeddings, la función de autocompletado, la búsqueda de reseñas por embedding y otros aspectos relevantes del código.

Se utiliza el espacio de nombres `products` para operaciones relacionadas con productos. Los endpoints permiten listar productos con filtros, crear, actualizar, eliminar, buscar productos, obtener categorías, contar favoritos, entre otras funcionalidades. Los endpoints disponibles son los siguientes:

`/api/products/`

- GET: Lista productos con filtros opcionales como categoría, nombre, rango de precios, tienda y número mínimo de favoritos. Soporta paginación mediante parámetros `page` y `limit`.
- POST: Crea un nuevo producto con los datos proporcionados, validando que no exista un producto con el mismo título.

`/api/products/{id}`

- GET: Obtiene los detalles de un producto específico según su ID.
- POST: No implementado (se usa PUT para actualizaciones).

- PUT: Actualiza los datos de un producto existente identificado por su ID.
- DELETE: Elimina un producto por su ID, incluyendo sus detalles, características y reseñas asociadas.

/api/products/search

- POST: Realiza una búsqueda semántica de productos basada en una consulta de texto, con filtros opcionales de categoría y precios, utilizando embeddings y similitud textual.

/api/products/categories

- GET: Devuelve una lista de todas las categorías de productos disponibles.

/api/products//favorite-count

- GET: Obtiene el número de usuarios que han marcado un producto como favorito.

/api/products//reviews

- GET: Lista las reseñas asociadas a un producto, con soporte para paginación.

/api/products/autocomplete

- GET: Proporciona sugerencias de búsqueda basadas en coincidencias parciales en los títulos de los productos.

/api/products/count

- GET: Devuelve el número total de productos en la base de datos.

/api/products/most-favorited

- POST: Obtiene los productos más añadidos a favoritos en un rango de fechas especificado, con un límite configurable.

products		Product related operations	^
POST	/api/v0/products/	Create a new product	▼
GET	/api/v0/products/	List products with filters	▼
GET	/api/v0/products/autocomplete	Autocomplete product search	▼
GET	/api/v0/products/categories	Get all product categories	▼
GET	/api/v0/products/count	Get the total number of products	▼
POST	/api/v0/products/most-favorited	Gets the IDs of the most favorited products within a date range	▼
POST	/api/v0/products/search	Search products by query	▼
PUT	/api/v0/products/{id}	Update a product by ID	▼
GET	/api/v0/products/{id}	Get a product by ID	▼
DELETE	/api/v0/products/{id}	Delete a product by ID	▼
GET	/api/v0/products/{id}/reviews	Get reviews for a product	▼
GET	/api/v0/products/{product_id}/favorite-count	Get the number of users who favorited a product	▼

Figura 4.47 Endpoints de productos

Creación de Productos

El proceso de creación de productos está implementado en la función `create_product` del servicio de productos. Esta función recibe un diccionario con los datos del producto, que puede representar un solo producto o una lista de productos. Los pasos principales son:

1. Validación de duplicados: Se verifica si ya existe un producto con el mismo título en la base de datos para evitar duplicados.

2. Creación del objeto: Se instancia un objeto `Product` con atributos como `title`, `main_category`, `price`, `features`, `description`, `images`, `videos`, `store`, `categories`, `details`, `parent_asin`, `bought_together` y `amazon_link`.
3. Generación del enlace de Amazon: Se genera automáticamente un enlace de Amazon utilizando el método `generate_amazon_link`. Este se obtiene a partir de su ASIN.
4. Generación de embeddings: Para los detalles (`details`) y características (`features`) del producto, se generan embeddings utilizando el modelo **stella_en_v5**. Estos embeddings se almacenan en las tablas `ProductDetail` y `ProductFeature`, respectivamente.
5. Confirmación en la base de datos: Los datos se persisten en la base de datos con una operación de `commit`. En caso de error, se realiza un `rollback`.

```
db.session.add(product)
db.session.flush()

if product.details:
    for key, value in product.details.items():
        detail_text = f"{key}: {value}"

        try:
            embedding = model.encode([detail_text])[0]

            new_detail = ProductDetail(
                product_id=product.product_id,
                detail=detail_text,
                detail_embedding=embedding.tolist()
            )
            db.session.add(new_detail)
            print(f"Detalle añadido para producto ID: {product.product_id}")

        except Exception as embed_error:
            print(f"Error al generar embedding para producto ID: {product.product_id}, detalle: {detail_text}")
            print(f"Detalles del error: {embed_error}")
            continue
```

Figura 4.48 Creación de producto y generación de embeddings para los detalles

Funcionamiento del Motor de Búsqueda searchProduct

El método searchProduct implementa una búsqueda semántica avanzada que combina similitud textual y embeddings para devolver resultados relevantes. A continuación, se detalla su funcionamiento:

1. Generación del embedding de la consulta: La consulta del usuario (query) se convierte en un embedding utilizando el modelo de lenguaje preentrenado (model.encode([query])).
2. Construcción de filtros: Se aplican filtros opcionales como categoría (category), precio mínimo (min_price) y precio máximo (max_price) para limitar los resultados. Estos filtros son generados automáticamente por el bot cuando lo considera necesario.
3. Consulta SQL combinada: La búsqueda utiliza una consulta SQL compleja que combina tres fuentes de información
4. Similitud de título y descripción: Utiliza la función SIMILARITY de PostgreSQL para calcular la similitud textual entre la consulta y los campos title y description de los productos.

```
WITH title_matches AS (  
  SELECT product_id, title, description, main_category, price,  
         (SIMILARITY(title, :query) * 70 +  
          SIMILARITY(COALESCE(CAST(description AS TEXT), ''), :query) * 30) AS title_score  
  FROM products  
  WHERE (SIMILARITY(title, :query) > 0.05  
         OR SIMILARITY(COALESCE(CAST(description AS TEXT), ''), :query) > 0.05)  
         {filters}  
  ORDER BY title_score DESC  
  LIMIT 30  
)
```

Figura 4.49 Cálculo de la similitud textual entre la consulta y los campos title y description

5. Similitud de detalles: Calcula la distancia de los embeddings de los detalles (ProductDetail.detail_embedding) respecto al embedding de la consulta utilizando el operador <=> (distancia coseno).

6. Similitud de características: Calcula la distancia de los embeddings de las características (ProductFeature.embedding) respecto al embedding de la consulta.

```
feature_scores AS (  
  SELECT pf.product_id,  
         SUM(1 - (pf.embedding <=> CAST(:query_embedding AS vector))) AS feature_score  
  FROM product_features pf  
 WHERE pf.product_id IN (SELECT product_id FROM title_matches)  
 GROUP BY pf.product_id  
) ,  
product_features AS (  
  SELECT pf.product_id,  
         JSON_AGG(pf.feature) AS features  
  FROM product_features pf  
 WHERE pf.product_id IN (SELECT product_id FROM title_matches)  
 GROUP BY pf.product_id  
)
```

Figura 4.50 Distancia de los embeddings de detalles y características respecto de la consulta

7. Puntuación total: Se calcula una puntuación total combinada (total_score) con pesos:
- 70% para la similitud del título (title_score).
 - 20% para la similitud de los detalles (detail_score).
 - 10% para la similitud de las características (feature_score).
8. Resultados ordenados: Los productos se ordenan por total_score en orden descendente y se limitan a top_n resultados.

```
    ],  
    "scores": {  
      "title_score": 5.000000223517418,  
      "detail_score": 4.268623062066869,  
      "feature_score": 1.7176147263855683,  
      "total_score": 4.525486241514122  
    }  
  },  
  {  
    "product_id": 133050,  
    "title": "Dr. Chaos",  
    "description": [  
      "NINTENDO NES GAME"  
    ],  
  },  
]
```

Figura 4.51 Formato devuelto por el motor de búsqueda

Formato de respuesta: Se devuelven los productos con sus detalles, imágenes, características y las puntuaciones calculadas.

El motor de búsqueda `searchProduct` proporciona una solución avanzada mediante búsqueda semántica que captura el significado de la consulta más allá de coincidencias exactas, utilizando embeddings para detalles y características, y permite personalizar resultados con filtros opcionales como categoría o rango de precios, garantizando relevancia y precisión gracias a una ponderación balanceada que combina similitud textual (70% título, 20% detalles, 10% características). No obstante, la consulta inicial está limitada a 30 resultados basados en similitud textual (`title_matches`), lo que podría excluir productos relevantes con títulos menos similares, y la generación de embeddings puede ser computacionalmente costosa para bases de datos grandes, lo que sugiere la necesidad de optimización mediante índices más avanzados, como HNSW (Hierarchical Navigable Small World), que acelera las búsquedas de vecinos más cercanos en espacios de alta dimensionalidad al organizar los embeddings en un grafo jerárquico eficiente.

Análisis de Productos Más Favoritos

El endpoint `/api/products/most-favorited` (POST) permite obtener los productos más añadidos a favoritos por los usuarios en un rango de fechas especificado, con un límite configurable (`limit`). Este endpoint es relevante para analizar tendencias de popularidad y

generar recomendaciones personalizadas. La función subyacente, `get_most_favorited_products`, recibe como entrada las fechas de inicio y fin (`start_date`, `end_date`) en formato YYYY-MM-DD y el número máximo de resultados (`limit`). Ejecuta una consulta SQL que agrupa los registros de la tabla `user_favorites` por `product_id`, cuenta las ocurrencias dentro del rango de fechas y ordena los resultados por número de favoritos en orden descendente.

```
SELECT product_id, COUNT(*) as favorite_count
FROM user_favorites
WHERE created_at BETWEEN :start_date AND :end_date
GROUP BY product_id
ORDER BY favorite_count DESC
LIMIT :limit
```

Figura 4.52 Lógica detrás de `/products/most-favorited`

Análisis de Autocompletado de Búsqueda

El endpoint `/api/products/autocomplete` (GET) proporciona sugerencias de búsqueda basadas en coincidencias parciales en los títulos de los productos, optimizando la experiencia del usuario al agilizar la interacción con el catálogo. Este endpoint es relevante para mejorar la usabilidad en búsquedas rápidas. La función subyacente, `autocomplete_products`, recibe como entrada un término de búsqueda (`search_term`) y un límite configurable (`limit`, por defecto 3). Ejecuta una consulta SQL que filtra los productos en la tabla `products` usando el operador `ilike` en el campo `title`, ordena los resultados por popularidad (`rating_number`) en orden descendente y limita la respuesta al número especificado.

```
if not search_term or len(search_term.strip()) < 2:
    return []

products = Product.query.filter(
    Product.title.ilike(f'%{search_term}%')
).order_by(
    Product.rating_number.desc()
).limit(limit).all()

suggestions = []
for product in products:
    suggestions.append({
        "product_id": product.product_id,
        "title": product.title,
    })

return suggestions
```

Figura 4.53 Lógica detrás de /api/products/autocomplete

6. Gestión de reseñas

Esta sección describe el módulo de gestión de reseñas de la aplicación, un componente esencial que permite la creación, actualización, eliminación y búsqueda de reseñas asociadas a los productos. Este módulo es crucial para enriquecer la interacción de los usuarios con el catálogo, ofreciendo funcionalidades avanzadas como la búsqueda semántica de reseñas y la vinculación de opiniones de usuarios a productos específicos. A continuación, se detallan los endpoints de la API para el espacio de nombres reviews, el proceso de creación de reseñas, la generación de embeddings, la búsqueda de reseñas por embedding mediante la función `get_reviews_by_embedding`, y otros aspectos relevantes del código.

Descripción de los Endpoints del Namespace reviews

Los endpoints permiten crear, actualizar y eliminar reseñas, asegurando que cada reseña esté vinculada a un producto mediante su `parent_asin`. Los endpoints disponibles son los siguientes:

/api/reviews/

POST: Crea una nueva reseña para un producto, vinculada mediante su parent_asin.

/api/reviews/{review_id}

PUT: Actualiza una reseña existente identificada por su ID.

DELETE: Elimina una reseña por su ID.

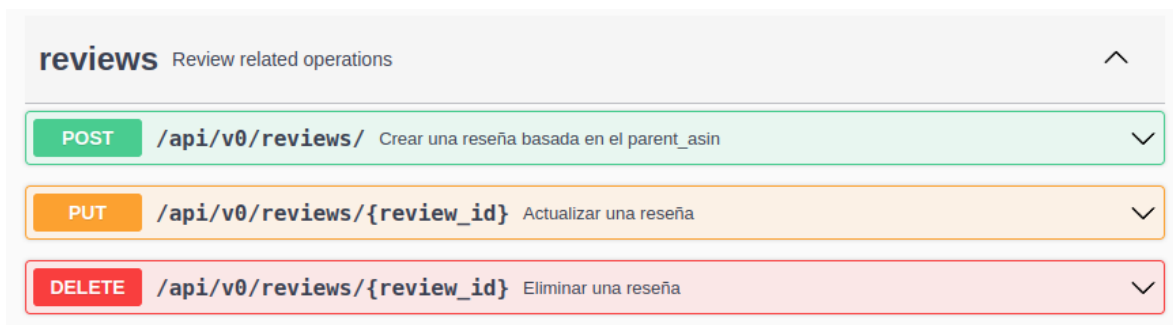


Figura 4.54 Endpoints de reseñas

Creación de Reseñas

El proceso de creación de reseñas está implementado en la función `create_review_for_product` del servicio de reseñas. Esta función recibe un diccionario con los datos de la reseña, que incluye el texto, la calificación, el usuario y el `parent_asin` del producto asociado. Los pasos principales son:

1. Validación del producto: Se verifica la existencia de un producto con el `parent_asin` proporcionado en la base de datos, devolviendo un error 404 si no se encuentra.
2. Verificación del usuario: Se comprueba si el usuario (`AmazonUser`) existe en la base de datos; si no, se crea un nuevo usuario con los datos proporcionados.
3. Procesamiento del timestamp: El timestamp de la reseña se convierte de milisegundos a un formato ISO compatible con la base de datos.

4. Generación de embeddings: Se genera un embedding para el texto de la reseña utilizando el modelo de lenguaje preentrenado `stella_en_v5`. Este embedding se almacena en el campo `embedding` de la reseña para habilitar búsquedas semánticas.

```
try:
    print("Generando embedding para la reseña")
    review_embedding = model.encode([data['text']]).tolist()[0]
    review.embedding = review_embedding
except Exception as e:
    print(f"Error generando el embedding: {e}")
    return {"error": "Error generando el embedding para la reseña"}, 500
```

Figura 4.55 Generación del embedding para la reseña

5. Confirmación en la base de datos: Los datos de la reseña se persisten en la base de datos con una operación de `commit`. En caso de error, se realiza un `rollback`.

Búsqueda de Reseñas por Embedding

La búsqueda de reseñas por embedding está implementada en la función `get_reviews_by_embedding`, que permite identificar reseñas relevantes para una consulta específica, utilizando embeddings generados por el modelo `stella_en_v5` para capturar el significado semántico del texto. Este método es fundamental para que el chatbot pueda responder preguntas de usuarios basadas en el contenido de las reseñas de un producto, aunque no está asociado a ningún endpoint de la API, ya que se utiliza exclusivamente desde el chatbot cuando necesita buscar información en las reseñas. Los pasos principales son:

1. Generación del embedding de la consulta: La consulta del usuario (`query_text`) se convierte en un embedding utilizando el modelo `stella_en_v5`.
2. Formato del embedding: El embedding se convierte en un array compatible con PostgreSQL para su uso en la consulta SQL.

3. Consulta SQL: Se ejecuta una consulta que calcula la distancia coseno entre el embedding de la consulta y el campo embedding de las reseñas, filtrando por product_id y ordenando por similitud (menor distancia coseno).

```
SELECT * FROM reviews
WHERE product_id = :product_id
ORDER BY embedding <=> :query_embedding
LIMIT :top_k
```

Figura 4.56 Consulta que devuelve la distancia coseno entre la pregunta y una reseña

La consulta selecciona todas las columnas de la tabla reviews, filtra las reseñas por el product_id especificado, y utiliza el operador <=> de PostgreSQL para calcular la distancia coseno entre el campo embedding de cada reseña (un vector almacenado en la base de datos) y el embedding de la consulta (:query_embedding). El operador <=> mide la similitud semántica entre dos vectores, retornando un valor entre 0 y 2, donde un valor menor indica mayor similitud (ángulo más pequeño entre los vectores). Los resultados se ordenan de menor a mayor distancia coseno, priorizando las reseñas más relevantes, y se limitan al número especificado por top_k.

La función get_reviews_by_embedding limita los resultados a un número configurable (top_k, por defecto 3) y devuelve una lista de objetos Review con los datos de las reseñas más relevantes, identificadas mediante la similitud semántica de sus embeddings con la consulta del usuario.

4.4.3 IMPLEMENTACIÓN DEL MÓDULO DEL CHATBOT

El módulo del chatbot es un componente central de la aplicación, diseñado para proporcionar una interfaz conversacional que permite a los usuarios interactuar con el sistema de manera natural, ya sea para buscar productos, obtener información de reseñas, realizar recomendaciones o, en el caso de administradores, gestionar operaciones de la tienda en línea. Conectado a un modelo de lenguaje a través de la API de OpenRouter, el

chatbot soporta streaming de respuestas, ejecución de funciones personalizadas y personalización según el rol del usuario (cliente o administrador). Este apartado detalla los endpoints de la API, la gestión de mensajes, el streaming de respuestas, los prompts del sistema, las funciones disponibles, la ejecución de herramientas y el flujo general de interacción, acompañado de un diagrama ilustrativo.

Endpoints de la API para el Chatbot

El módulo del chatbot se gestiona bajo el espacio de nombres chat en la API RESTful, definido en el archivo `chat_routes.py`. Los endpoints disponibles permiten realizar consultas conversacionales y gestionar sesiones de chat. La siguiente figura resume los endpoints, que se describen a continuación:

- POST `/api/chat/query`: Recibe una consulta del usuario (prompt), un identificador opcional de producto (`product_id`) y el modelo de lenguaje a utilizar (`model`, por defecto `openai/gpt-4o-mini`). Requiere autenticación mediante JWT y devuelve una respuesta en streaming en formato de texto plano. Si no se proporciona una consulta, retorna un error 400.
- POST `/api/chat/clear`: Elimina la sesión de chat del usuario actual, identificado por su `user_id`, reiniciando el historial de mensajes. Requiere autenticación JWT y devuelve un mensaje de éxito.

chat Chat related operations	
POST	<code>/api/v0/chat/clear</code>
POST	<code>/api/v0/chat/query</code>

Figura 4.57 Endpoints del chatbot

Gestión de Mensajes

La gestión de mensajes se realiza en la clase ChatService (chat_service.py), que mantiene un historial de mensajes (self.messages) para cada usuario, identificado por su user_id. Este historial asegura la coherencia conversacional al enviar el contexto completo al modelo de lenguaje en cada interacción. Los mensajes se estructuran en tres roles:

1. System: Define el comportamiento del chatbot mediante prompts iniciales. Para clientes, el prompt instruye al chatbot a responder preguntas sobre productos, proporcionar recomendaciones y asistir en compras, mientras que para administradores añade capacidades de gestión de usuarios, generación de informes y visualización de datos. También se incluye la fecha actual y las categorías de productos disponibles.
2. User: Contiene las consultas enviadas por el usuario (prompt).
3. Assistant: Almacena las respuestas generadas por el modelo o los resultados de las funciones ejecutadas.

Cada instancia de ChatService es única por usuario, almacenada en un diccionario estático (_instances), y se inicializa con un prompt de sistema personalizado según el rol del usuario, determinado al consultar la base de datos con UserService.get_user_by_id. El método remove_instance permite limpiar la sesión al invocar el endpoint /api/chat/clear.

```
self.messages = [{  
    "role": "system",  
    "content": f"You are an assistant for an online store. Today's date is {current_date}. You can  
    answer questions about product information, provide recommendations, and help customers with their  
    purchases. You cannot make direct purchases. Never show images in your responses."  
}]
```

Figura 4.58 Instrucciones iniciales del chatbot

Streaming de Respuestas

El chatbot utiliza streaming para enviar respuestas al cliente en tiempo real, mejorando la experiencia de usuario al mostrar el texto conforme se genera. Esto se implementa en el método `generate_stream` de la clase `GeneralQuery` (`chat_routes.py`), que procesa la respuesta del modelo de lenguaje como un flujo de datos mediante la API de OpenRouter con el parámetro `stream=True`. Los pasos principales son:

1. Envío de la solicitud: El método `ask_question` de `ChatService` realiza una solicitud POST a `https://openrouter.ai/api/v1/chat/completions`, incluyendo el historial de mensajes, el modelo seleccionado y las herramientas disponibles.
2. Procesamiento del flujo: La respuesta se recibe en fragmentos (chunks) codificados en UTF-8, procesados línea por línea. Cada fragmento contiene un objeto JSON con un campo `choices`, que puede incluir:
 - Contenido de texto (`delta.content`): Texto generado por el modelo, enviado directamente al cliente.
 - Llamadas a herramientas (`delta.tool_calls`): Indicaciones de funciones a ejecutar, procesadas posteriormente.
3. Bufferización: Un buffer acumula los fragmentos hasta encontrar un salto de línea, asegurando que los datos se procesen correctamente.
4. Finalización: Cuando se recibe `[DONE]`, el historial de mensajes se actualiza con la respuesta completa.

El siguiente fragmento de código ilustra la implementación del streaming:

```
def generate_stream(self, chat_service, question, product_id=None, model=None):
    """Genera el streaming de respuestas conforme se reciben."""
    try:
        for response_text in chat_service.ask_question(
            prompt=question,
            product_id=product_id,
            model=model
        ):
            yield response_text
    except Exception as e:
        yield f"data: Error en el streaming: {str(e)}\n\n"
```

Figura 4.59 Implementación del streaming de respuestas en el método generate_stream

Este código, parte de la clase GeneralQuery, utiliza un generador Python (yield) para enviar fragmentos de texto al frontend en tiempo real, provenientes del método ask_question. En caso de error, se envía un mensaje dentro del flujo, asegurando robustez. El streaming permite respuestas fluidas, reduciendo la percepción de latencia en consultas complejas.

Parámetros de configuración del modelo

Los parámetros de configuración del modelo permiten controlar y personalizar el comportamiento del modelo de lenguaje al hacer una solicitud a través de la API de OpenRouter. Estos parámetros se definen dentro del diccionario payload y afectan aspectos como la creatividad de la respuesta, la repetición de información, la forma de generación del texto (fluida o en tiempo real), y el uso de herramientas externas. A continuación, se describen los más relevantes:

- **model:** Especifica el modelo de lenguaje utilizado (por defecto, openai/gpt-4o-mini). Puede cambiarse como se ve en la [Figura 4.81](#).
- **messages:** Contiene el historial de la conversación (mensajes del sistema, usuario y asistente) para dar contexto al modelo.

- **top_p**: Controla la diversidad de las respuestas. El modelo ordena las posibles palabras según su probabilidad y elige solo entre las más probables hasta alcanzar un cierto umbral acumulado. Por ejemplo, con $\text{top_p} = 0.9$ (el valor usado en la aplicación), el modelo elegirá solo entre las palabras más seguras hasta que su suma de probabilidades llegue al 90%. Si $\text{top_p} = 1.0$, se consideran todas las posibles salidas sin ningún filtro, permitiendo respuestas más variadas pero menos controladas.
- **temperature**: Regula la aleatoriedad de las respuestas. Un valor bajo (se establece 0.2 en la aplicación) hace que las respuestas sean más precisas, consistentes y menos creativas.
- **frequency_penalty**: Penaliza la repetición de palabras ya utilizadas. Con valor 0, no hay penalización.
- **presence_penalty**: Penaliza la repetición de temas ya mencionados. También está en 0, por lo que no afecta la generación.
- **repetition_penalty**: Penaliza la repetición directa de tokens. Un valor de 1 indica que no se aplica penalización.
- **top_k**: Limita las opciones de salida a las k palabras más probables. Al estar en 0, se desactiva esta restricción.
- **stream**: Si está en True, permite que las respuestas se generen en tiempo real (streaming), mostrando el texto a medida que se produce.
- **tools**: Lista de herramientas que el modelo puede invocar dinámicamente, como funciones de búsqueda, generación de informes, entre otras.

Prompts y Mensajes del Sistema

Los prompts del sistema son fundamentales para definir el comportamiento del chatbot y se configuran en la inicialización de ChatService. Los prompts varían según el rol del usuario:

- **Cientes:** El prompt instruye al chatbot a responder preguntas sobre productos, ofrecer recomendaciones y asistir en compras sin realizar transacciones directas. También se prohíbe mostrar imágenes y se proporcionan las categorías de productos disponibles.
- **Administradores:** Además de las capacidades de cliente, el prompt habilita la gestión de usuarios, generación de informes y creación de gráficos dinámicos usando la función `generate_dynamic_chart`. Se especifica que las funciones `get_users`, `generate_user_activity_report` y `generate_product_popularity_report` son exclusivas para informes en PDF y no para gráficos.

```
"You are an admin assistant for an online store. Today's date is {current_date}. You can manage user information, generate reports, answer questions about products, provide recommendations, assist with store operations, and support data visualization through charts. Never show images in your responses. Do not call 'get_users', 'generate_user_activity_report', or 'generate_product_popularity_report' to obtain data for charts, as they are for PDF reports only."
```

Figura 4.60 Ejemplo de prompt para administradores

Ejecución de Funciones

El chatbot soporta la ejecución de funciones personalizadas (herramientas) mediante el mecanismo de tool calls proporcionado por la API de OpenRouter. Las funciones se definen en el atributo `tool_functions` de `ChatService` y se dividen en dos categorías:

1. **Funciones para clientes:**
 - `search_product`: Realiza una búsqueda semántica de productos basada en una consulta de texto, con filtros opcionales de categoría, precio mínimo, precio máximo y número de resultados (`top_n`). Devuelve un texto con los productos encontrados y `additional_data` con detalles completos.
 - `get_reviews_by_embedding`: Busca reseñas relevantes para una consulta específica, utilizando embeddings generados por `stella_en_v5` y filtrando por `product_id` si está disponible. No está asociada a un endpoint, ya que se

usa exclusivamente por el chatbot. Devuelve el texto de las reseñas y `additional_data` con los identificadores de las reseñas.

2. Funciones para administradores:

- `get_users`: Lista usuarios con filtros opcionales (correo, rol, GitHub ID). Devuelve un texto descriptivo y `additional_data` con la lista de usuarios.
- `get_user_by_id`: Obtiene detalles de un usuario específico por su ID, con texto y `additional_data`.
- `set_user_role`: Cambia el rol de un usuario (usuario o administrador), con texto y `additional_data`.
- `delete_user`: Elimina un usuario del sistema, con un mensaje de confirmación.
- `generate_user_activity_report`: Genera un informe en PDF de actividad de usuarios, con filtros de fecha y rol. Devuelve un texto de estado y `additional_data` con parámetros del informe.
- `generate_product_popularity_report`: Genera un informe en PDF de popularidad de productos, con filtros de categoría, precio, calificación y favoritos. Devuelve un texto de estado y `additional_data`.
- `generate_dynamic_chart`: Crea gráficos dinámicos (barras, líneas, torta, dona) usando Chart.js, con datos agregados desde la base de datos. Devuelve un texto de estado y `additional_data` con la configuración del gráfico.

El proceso de ejecución de funciones se maneja en el método `_handle_tool_calls` de `ChatService`:

1. **Detección de tool calls**: Durante el streaming, si se detecta un `tool_call`, los argumentos se acumulan en un buffer (`tool_calls_buffer`).

2. **Ejecución:** Los argumentos JSON se parsean, y la función correspondiente se ejecuta. El resultado incluye un campo `response_text` (enviado al chatbot para generar la respuesta final) y, en algunos casos, `additional_data` (enviado al cliente, e.g., configuraciones de gráficos o datos de informes).
3. **Feedback al chatbot:** El `response_text` se agrega al historial de mensajes como un mensaje de tipo `tool`, permitiendo al modelo de lenguaje generar una respuesta textual basada en el resultado.
4. **Envío de `additional_data`:** Para funciones como `generate_dynamic_chart`, `generate_user_activity_report` o `search_product`, los datos adicionales se envían al cliente en el flujo de streaming como un evento JSON.
5. **Iteración:** El proceso de `ask_question` se reinicia con el contexto actualizado para continuar la conversación.

```
{
  "response_text": "[{"title": "Smartphone XYZ", "price": 699.99, ...}, ...]",
  "additional_data": {
    "products": [{"product_id": 1, "title": "Smartphone XYZ", "images": [...], ...}, ...]
  }
}
```

Figura 4.61 Ejemplo de tool call para `search_product`

Funcionamiento General de la Interacción con el Chatbot

La interacción con el chatbot comienza cuando un usuario autenticado envía una consulta al endpoint `/api/chat/query`. El flujo general es el siguiente:

1. **Procesamiento de la consulta:** La consulta (prompt) y el `product_id` opcional se procesan en `ChatService`, agregándose al historial de mensajes y configurando herramientas según el rol.
2. **Solicitud al modelo:** La consulta se envía a `OpenRouter` con streaming activado.
3. **Manejo del stream:** Los fragmentos de respuesta se dividen en:
 - **Texto:** Enviado al cliente como respuesta directa.

- **Tool calls:** Ejecutan funciones, cuyos resultados (`response_text`) se envían al chatbot para generar la respuesta final, mientras que `additional_data` (e.g., gráficos, informes) se envía al cliente.

4. **Actualización y finalización:** El historial de mensajes se actualiza, y la interacción continúa hasta completar la respuesta.

El chatbot utiliza el modelo `openai/gpt-4o-mini` por defecto, configurado con parámetros estandar como `temperature=0.2` para respuestas precisas y `stream=True` para streaming. La interacción es personalizada según el contexto (e.g., producto en visualización) y el rol del usuario, garantizando respuestas relevantes y funcionales.

PASO	ACCIÓN	RESULTADO
1	Recibir consulta en <code>/api/chat/query</code>	Consulta y <code>product_id</code> opcional registrados
2	Procesar <code>prompt</code> y configurar herramientas	Historial de mensajes actualizado con <code>prompt</code> y herramientas por rol
3	Enviar solicitud a OpenRouter con streaming	Flujo de datos iniciado
4	Procesar fragmentos: texto o tool call	Texto enviado al cliente; tool call ejecuta función
5	Ejecutar función y enviar <code>response_text</code> al chatbot	Chatbot genera respuesta textual basada en <code>response_text</code>
6	Enviar <code>additional_data</code> al cliente	Datos de gráficos o informes enviados al cliente
7	Finalizar interacción	Historial actualizado y respuesta completa

Tabla 4.50 Flujo de interacción con el chatbot

Generación y visualización de mapas de calor y gráficos

La aplicación incorpora funcionalidades avanzadas para la visualización de datos mediante mapas de calor y gráficos, diseñadas principalmente para administradores con el propósito de analizar patrones de uso y actividad en la tienda en línea. Los mapas de calor representan geográficamente la distribución de inicios de sesión de usuarios a partir de sus direcciones IP, mientras que los gráficos, generados dinámicamente, permiten visualizar datos agregados de la base de datos, como favoritos por categoría o actividad de usuarios. Estas funcionalidades se integran con el módulo del chatbot para gráficos e informes, y se implementan mediante endpoints específicos para mapas de calor. Este apartado describe la generación de mapas de calor, el proceso de creación de gráficos e informes a través de tool calls, y otros aspectos relevantes, como la validación de datos y la gestión de errores.

Generación de Mapas de Calor

Los mapas de calor se generan a partir de los registros de inicios de sesión almacenados en la tabla LoginLog, utilizando las direcciones IP para obtener coordenadas geográficas que se visualizan en un mapa interactivo. La funcionalidad está implementada en los archivos HeatmapRoutes.py y HeatmapService.py, y se accede mediante el endpoint `/api/heatmap/heatmap_data`.

Endpoint y Filtrado

El endpoint **GET `/api/heatmap/heatmap_data`** (HeatmapRoutes.py) permite a los administradores autenticados (mediante JWT) obtener datos para un mapa de calor, con filtros opcionales de fecha de inicio (`start_date`) y fin (`end_date`) en formato YYYY-MM-DD. Si no se proporcionan fechas, se utiliza un rango predeterminado de los últimos 30 días hasta la fecha actual. Las fechas se validan utilizando `datetime.fromisoformat`, retornando un error 400 si el formato es inválido.

```
GET /api/heatmap/heatmap_data?start_date=2025-05-01&end_date=2025-06-01
```

Figura 4.62 Ejemplo de solicitud de los datos del heatmap

Proceso de Geolocalización

La clase HeatmapService (HeatmapService.py) maneja la lógica de generación de datos para el mapa de calor, con un enfoque particular en la geolocalización de direcciones IP. Este proceso utiliza la base de datos GeoLite2 y la biblioteca maxminddb para transformar direcciones IP en coordenadas geográficas [\[55\]](#). Los pasos principales son:

1. Consulta a la base de datos: Se ejecuta una consulta SQL en la tabla LoginLog, filtrando registros con direcciones IP no nulas y excluyendo IPs locales (127.0.0.1, ::1). Los filtros de fecha se aplican si se especifican start_date o end_date.
2. Geolocalización de IPs:
 - Base de datos GeoLite2: La base de datos GeoLite2-City.mmdb, proporcionada por MaxMind, es un archivo binario que contiene información geográfica para direcciones IP, incluyendo latitud, longitud, ciudad, país y otros datos. Es una versión gratuita y actualizable, diseñada para aplicaciones que requieren geolocalización sin alta precisión [\[56\]](#). En la aplicación, se almacena en app/data/GeoLite2-City.mmdb.
 - Biblioteca maxminddb: Esta biblioteca de Python permite leer el archivo GeoLite2-City.mmdb de manera eficiente [\[55\]](#). La función HeatmapService.geolocate_ip utiliza el método get del objeto maxminddb.Reader para buscar una dirección IP y extraer su información geográfica. Si la IP es válida y está en la base de datos, se obtienen las coordenadas (latitude y longitude) desde el campo location. Las coordenadas se redondean a dos decimales para reducir la granularidad y optimizar la agregación posterior.
 - Manejo de errores: Si la base de datos no está disponible (e.g., archivo no encontrado) o la IP no se puede geolocalizar (e.g., IP privada o no registrada), la función retorna None, y la IP se descarta con un mensaje de depuración.

3. Agregación de datos: Se utiliza un objeto Counter para contar la frecuencia de cada par de coordenadas (latitud, longitud). Este enfoque asegura que ubicaciones cercanas se agrupen, reduciendo la complejidad del mapa de calor.
4. Formato de salida: Los datos se devuelven como una lista de objetos JSON con los campos lat, lng y weight (frecuencia), compatibles con bibliotecas de visualización como Leaflet o Google Maps.

Este código consulta registros de inicio de sesión, geolocaliza IPs usando GeoLite2 y maxminddb, y agrega coordenadas para generar datos de un mapa de calor en formato JSON, permitiendo visualizar la densidad de inicios de sesión en un mapa interactivo; sin embargo, la precisión de GeoLite2 puede ser aproximada para IPs de redes móviles o proxies, la geolocalización falla si el archivo GeoLite2-City.mmdb está ausente o corrupto (manejado con un lector nulo), y el rendimiento puede ser lento con grandes volúmenes de registros, aunque el redondeo de coordenadas a dos decimales reduce la carga computacional. El resultado del mapa de calor generado se muestra en la [Figura 4.84](#).

```
@staticmethod
def get_heatmap_data(start_date=None, end_date=None):
    query = db.session.query(LoginLog).filter(
        LoginLog.ip_address.isnot(None),
        LoginLog.ip_address.notin(['127.0.0.1', '::1'])
    )
    if start_date:
        query = query.filter(LoginLog.login_at >= start_date)
    if end_date:
        query = query.filter(LoginLog.login_at <= end_date)
    logs = query.all()
    geo_counts = Counter()
    for log in logs:
        geo_data = HeatmapService.geolocate_ip(log.ip_address)
        if geo_data and geo_data['latitude'] and geo_data['longitude']:
            lat = round(geo_data['latitude'], 2)
            lng = round(geo_data['longitude'], 2)
            geo_counts[(lat, lng)] += 1
    heat_data = [
        {'lat': lat, 'lng': lng, 'weight': count}
        for (lat, lng), count in geo_counts.items()
    ]
    return heat_data
```

Figura 4.63 Implementación del metodo get_heatmap_data

Generación de Gráficos e Informes

Los gráficos e informes se generan mediante tool calls en el módulo del chatbot, implementados en AdminHandlers.py. Estas funciones permiten a los administradores visualizar datos agregados o generar informes en PDF, integrándose con el flujo conversacional del chatbot.

Generación de Gráficos Dinámicos

La función generate_dynamic_chart (AdminHandlers.handle_generate_dynamic_chart) crea gráficos dinámicos utilizando Chart.js, basándose en datos agregados de la base de datos. Los pasos son:

1. **Validación de parámetros:** Se verifican parámetros obligatorios (chart_type, data_source, x_axis, y_axis, title, aggregation_field, group_by_field) y opcionales

(time_field, time_range, join_tables, filter_conditions). Los tipos de gráficos soportados son bar, line, pie y doughnut. Las fuentes de datos de las que el chatbot puede seleccionar incluyen products, users, reviews, login_logs y user_favorites.

2. **Construcción de la consulta SQL:** Se genera una consulta dinámica utilizando SQLAlchemy, seleccionando el campo de agrupación (group_by_field) y el campo de agregación (aggregation_field, e.g., count para conteos o price para sumas). Se aplican uniones (join_tables), filtros temporales (time_range, e.g., last_month) y condiciones adicionales (filter_conditions).
3. **Procesamiento de datos:** Los resultados se mapean a etiquetas (labels) y valores (values), asegurando que todas las categorías posibles (e.g., todas las categorías de productos) estén incluidas, con valores 0 para las ausentes.
4. **Configuración del gráfico:** Se genera una configuración JSON para Chart.js, incluyendo colores modernos, opciones de escalas, leyendas y animaciones. La configuración se devuelve en additional_data, mientras que response_text indica el estado de la operación.
5. **Integración con el chatbot:** El response_text se envía al modelo de lenguaje para generar una respuesta textual, y additional_data se envía al cliente para renderizar el gráfico en la interfaz. Este proceso y su resultado se ilustra en la [Figura 4.91](#). A continuación, se muestra la salida del método en formato JSON, que contiene tanto el texto del chatbot como los parámetros utilizados para renderizar el gráfico.

```
{
  "response_text": "Preparando gráfico de barras para datos de user_favorites con título: Favoritos por Categoría",
  "additional_data": {
    "chart_type": "bar",
    "data_source": "user_favorites",
    "chart_config": {
      "type": "bar",
      "data": {
        "labels": ["Electronics", "Clothing", ...],
        "datasets": [{
          "label": "count",
          "data": [50, 30, ...],
          "backgroundColor": ["rgba(30, 136, 229, 0.4)", ...],
          "borderColor": ["rgba(30, 136, 229, 1)", ...]
        }]
      }
    },
    "options": {...}
  }
}
```

Figura 4.64 Salida de generate_dynamic_chart

Generación de Informes

Las funciones generate_user_activity_report y generate_product_popularity_report generan informes en PDF, con los siguientes pasos:

1. **Validación de parámetros:** Se aceptan filtros como start_date, end_date, role para actividad de usuarios, o category, min_price, max_price, min_rating, min_favorites para popularidad de productos.
2. **Procesamiento:** Los datos se procesan, y se genera un mensaje de estado en response_text. Los parámetros se incluyen en additional_data para que el cliente gestione la descarga del informe.
3. **Integración con el chatbot:** Similar a los gráficos, response_text se envía al modelo para la respuesta textual, y additional_data al cliente.

PASO	ACCIÓN	RESULTADO
1	Detectar tool call en el stream	Identificar función (generate_dynamic_chart, generate_user_activity_report, etc.)
2	Validar parámetros de entrada	Parámetros correctos o error retornado
3	Ejecutar función y consultar datos	Datos agregados para gráficos o parámetros para informes
4	Generar response_text	Texto enviado al chatbot para respuesta textual
5	Enviar additional_data al cliente	Configuración de gráficos o parámetros de informes enviados
6	Renderizar en la interfaz	Gráfico o enlace de descarga mostrado al usuario

Tabla 4.51 Flujo de generación de gráficos e informes

4.4.4 IMPLEMENTACIÓN DEL SISTEMA KAFKA

A continuación, se presenta una explicación detallada del funcionamiento del sistema, incluyendo el rol de los componentes Producer y Consumer, el flujo de datos y la interacción con una base de datos SQLite y una API RESTful. Se incluye un esquema visual del flujo de datos y fragmentos clave del código para ilustrar los procesos fundamentales.

Descripción General del Sistema

El sistema de ingesta de datos se compone de dos elementos principales: el Producer y el Consumer, ambos implementados en Python utilizando la biblioteca kafka-python. El *Producer* lee datos de productos y reseñas desde archivos JSONL, aplica filtros para garantizar la calidad de los datos y envía los registros válidos a dos tópicos de Kafka: products y reviews. El *Consumer* consume estos mensajes desde los tópicos y los envía al

backend para su almacenamiento en una base de datos remota. Además, se utiliza una base de datos SQLite local para indexar las reseñas y optimizar su recuperación.

El flujo de datos puede resumirse en los siguientes pasos:

1. **Lectura y Filtrado (Producer):** Lee productos y reseñas desde archivos JSONL, aplica filtros basados en criterios como el número de reseñas y la validez del precio, e indexa las reseñas en SQLite.
2. **Envío a Kafka (Producer):** Envía los datos filtrados a los tópicos products y reviews.
3. **Consumo y Almacenamiento (Consumer):** Lee los mensajes de Kafka y los envía al backend para su almacenamiento en la base de datos.
4. **Seguimiento y Persistencia:** Utiliza un archivo de seguimiento (last_sent.json) para garantizar la continuidad del procesamiento en caso de interrupciones.

Componentes del Sistema

1. Producer

El *Producer* es responsable de leer los datos de los archivos JSONL (meta_Electronics.jsonl para productos y Electronics.jsonl para reseñas), aplicar filtros y enviar los datos válidos a Kafka. Los pasos clave son:

- **Indexación de Reseñas:** Se crea una base de datos SQLite (electronics.db) para almacenar un índice de reseñas con los campos review_id, parent_asin y file_offset. El campo file_offset indica la posición en bytes de una reseña en el archivo en el que se almacenan las reviews, permitiendo su acceso directo mediante file.seek(offset). Esto optimiza la recuperación de reseñas asociadas a un producto específico, evitando recorrer secuencialmente el archivo completo.

```
def create_reviews_table(db_path):
    """Crea la tabla de reseñas en la base de datos SQLite."""
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS reviews (
            review_id TEXT PRIMARY KEY,
            parent_asin TEXT,
            file_offset INTEGER
        )
    ''')
    cursor.execute('CREATE INDEX IF NOT EXISTS idx_parent_asin ON reviews(parent_asin)')
    conn.commit()
    conn.close()
```

Figura 4.65 Creación de base de datos SQLite auxiliar

- **Filtrado de Productos:** Los productos se filtran según la presencia de una descripción válida y un precio positivo. La función `apply_filters` implementa estas reglas.
- **Envío a Kafka:** Los productos y reseñas aprobados se envían a los tópicos `products` y `reviews` utilizando el *Producer* de Kafka, configurado con `acks='all'` para garantizar la entrega de mensajes.

```
print(f"Procesando producto {parent_asin}...")
product_result, reviews = process_product(product, db_path, reviews_file)
if product_result:
    print(f"Producto {parent_asin} aprobado. Preparando para enviar...")
    # Enviar producto a Kafka
    if send_to_kafka('products', product_result, parent_asin):
        print(f"Producto {parent_asin} enviado exitosamente.")
        if reviews:
            success_count = 0
            for review in reviews:
                transformed_review = transform_review(review)
                if send_to_kafka('reviews', transformed_review, parent_asin):
                    success_count += 1
            print(f"Se enviaron {success_count} reviews para el producto {parent_asin}.")
```

Figura 4.66 Envío de datos al topic

- **Seguimiento de Progreso:** Un archivo JSON (`last_sent.json`) almacena el último `parent_asin` procesado, permitiendo reanudar el procesamiento desde el punto de interrupción.

2. Consumer

El *Consumer* lee los mensajes de los tópicos `products` y `reviews` y los envía a el backend. Los pasos clave son:

- **Procesamiento de Productos:** El *Consumer* se suscribe al tópico `products` y envía cada producto a la API mediante una solicitud POST. Una vez procesados todos los productos, se pasa al procesamiento de reseñas.
- **Procesamiento de Reseñas:** El *Consumer* se suscribe al tópico `reviews`, obtiene el `product_id` correspondiente al `parent_asin` de cada reseña mediante una solicitud GET a la API, y envía la reseña con el `product_id` a través de una solicitud POST.

```
def process_reviews(consumer):
    """Process reviews for any product with a valid product_id"""
    print("=== STARTING REVIEWS PROCESSING PHASE ===")
    consumer.subscribe(['reviews'])

    for message in consumer:
        if message.topic != 'reviews':
            continue
        record = message.value
        parent_asin = record.get('parent_asin')
        print(f"Processing review for product: {parent_asin}")

        product_id = get_product_id_by_parent_asin(parent_asin)
        if not product_id:
            print(f"Could not find product_id for {parent_asin}, skipping review")
            continue

        review_payload = record.copy()
        review_payload['product_id'] = product_id
        response = requests.post(REVIEWS_ENDPOINT, json=review_payload, headers=headers)
```

Figura 4.67 Procesamiento de reseñas en el consumer

Funcionamiento Detallado

1. Inicialización del Producer:

- Se inicializa el *Producer* de Kafka con los servidores de bootstrap (localhost:29092) y un serializador JSON.
- Se carga el archivo de seguimiento (last_sent.json) para determinar el último producto procesado.
- Si no existe la base de datos SQLite, se crea la tabla de reseñas y se construye el índice.

2. Procesamiento de Productos:

- Se lee el archivo meta_Electronics.jsonl línea por línea, donde cada línea representa un producto en formato JSON.
- Para cada producto, se consulta la base de datos SQLite para contar las reseñas asociadas al parent_asin.
- Se aplican los filtros definidos en apply_filters. Si el producto pasa los filtros, se envía al tópico products y sus reseñas asociadas al tópico reviews.

3. Procesamiento de Reseñas:

- Las reseñas se recuperan del archivo Electronics.jsonl utilizando los offsets almacenados en SQLite, lo que optimiza la lectura.
- Cada reseña se transforma al formato esperado mediante la función transform_review antes de ser enviada a Kafka.

4. Consumo y Almacenamiento:

- El *Consumer* procesa primero todos los mensajes del tópico products, enviándolos a la API RESTful.
- Luego, se suscribe al tópico reviews, obtiene el product_id correspondiente y envía las reseñas a la API.
- Se utiliza un token JWT para autenticar las solicitudes a la API.

Conclusión

El sistema implementado utiliza Apache Kafka como una solución robusta para la ingesta de datos, integrando un Producer para el filtrado y envío de datos desde archivos JSONL

(meta_Electronics.jsonl para productos y Electronics.jsonl para reseñas) y un Consumer para su procesamiento y almacenamiento en una base de datos remota a través de una API RESTful, autenticada mediante un token JWT para garantizar que solo usuarios autorizados puedan almacenar datos. La combinación de una base de datos SQLite (electronics.db) para indexar reseñas con campos como review_id, parent_asin y file_offset acelera las consultas por parent_asin, optimizando la recuperación de reseñas asociadas a un producto específico y reduciendo el tiempo de procesamiento al evitar la lectura secuencial del archivo completo. Kafka asegura escalabilidad mediante particiones y réplicas, garantizando alta disponibilidad, mientras que la configuración acks='all' en el Producer y el archivo de seguimiento last_sent.json proporcionan tolerancia a fallos, asegurando que no se pierdan datos en caso de interrupciones. Sin embargo, se ha identificado un cuello de botella en la ingesta de productos debido a la generación de embeddings mediante el modelo local stella_en_v5, que requiere recursos computacionales significativos; este problema podría solventarse transitando a una API externa de embeddings, como OpenAI Embeddings, asumiendo el coste adicional para mejorar la eficiencia.

4.5 PRUEBAS

En cualquier proyecto de software, es fundamental llevar a cabo diversas pruebas para verificar que el sistema desarrollado cumple con los requisitos establecidos. Se han realizado pruebas de sistema y pruebas de validación alfa en la página web.

La siguiente tabla detalla cada prueba de sistema efectuada sobre la arquitectura de datos, incluyendo un identificador, una descripción detallada y el resultado obtenido, verificando también el estado de los tópicos products y reviews desde la interfaz de usuario de Kafka.

ID	DESCRIPCIÓN	SALIDA
SYS-KAFKA-01	Verificar que el Producer inicializa correctamente la conexión con los brokers de Kafka utilizando los	El Producer se conecta exitosamente a Kafka.

	servidores de bootstrap (localhost:29092).	
SYS-K AFKA -02	Comprobar que el Producer lee y parsea correctamente los datos de productos y reviews.	El Producer carga todas las líneas del archivo JSONL.
SYS-K AFKA -03	Validar que el Producer aplica filtros de calidad (descripción válida y precio positivo) antes de enviar productos al tópico products.	Solo los productos con descripción no vacía y precio > 0 se envían al tópico products.
SYS-K AFKA -04	Asegurar que el Producer crea y actualiza correctamente la tabla de reseñas en la base de datos SQLite (electronics.db) con los campos review_id, parent_asin y file_offset.	La tabla reviews en SQLite se crea con los índices correspondientes y contiene los datos indexados.
SYS-K AFKA -05	Confirmar que el Producer envía mensajes al tópico products	Todos los mensajes de productos se envían al tópico products y la UI de Kafka confirma su recepción.
SYS-K AFKA -06	Verificar que el Producer envía reseñas al tópico reviews utilizando los offsets almacenados en SQLite para optimizar la recuperación.	Las reseñas asociadas a cada parent_asin se envían al tópico reviews.
SYS-K AFKA -07	Comprobar que el Consumer se suscribe correctamente al tópico products y procesa todos los mensajes recibidos.	El Consumer lee todos los mensajes del tópico products.
SYS-K AFKA -08	Asegurar que el Consumer envía reseñas al backend mediante solicitudes POST autenticadas con un token JWT	Las reseñas se almacenan en la base de datos, con un estado HTTP 200 para cada solicitud POST exitosa.
SYS-K AFKA -09	Verificar que el archivo last_sent.json registra correctamente el último parent_asin procesado por el Producer para tolerancia a fallos.	El archivo last_sent.json contiene el parent_asin del último producto procesado, y el Producer reanuda desde este punto tras una interrupción.
SYS-K AFKA -10	Evaluar la tolerancia a fallos reiniciando el Consumer durante el procesamiento de mensajes del tópico reviews.	El Consumer reanuda el procesamiento desde el último mensaje consumido sin duplicar datos.

SYS-K AFKA -11	Comprobar la integridad de los datos transferidos desde el Producer al Consumer y almacenados en la base de datos remota.	Los datos de productos y reseñas en la base de datos remota coinciden con los enviados por el Producer, verificado mediante una comparación de registros.
----------------------	---	---

Tabla 4.52 Pruebas de sistema para el sistema de datos

La [Tabla 4.53](#) detalla cada prueba de validación efectuada en la aplicación web, incluyendo un identificador, una descripción detallada y los resultados obtenidos.

ID	DESCRIPCIÓN	SALIDA
WEB-VAL-01	Verificar que la página de bienvenida del frontend muestra un botón para ir al formulario de login.	La UI muestra la página de bienvenida un botón al login.
WEB-VAL-02	Comprobar que desde la página de login del frontend se puede navegar al formulario de registro mediante un enlace.	La UI del login muestra un enlace al registro, y al hacer clic, carga la página de registro.
WEB-VAL-03	Validar que desde la página de registro del frontend se puede regresar al formulario de login mediante un enlace.	La UI del registro muestra un enlace al login, y al hacer clic, carga la página de login.
WEB-VAL-04	Asegurar que un usuario estándar puede registrarse completando el formulario de registro en el frontend con correo, contraseña y nombre válidos.	El formulario se envía y se redirige al usuario a la página de productos.
WEB-VAL-05	Verificar que un usuario estándar puede registrarse usando un proveedor social (GitHub) desde un botón en la página de registro.	La UI redirige a GitHub, autentica al usuario y registra la cuenta, mostrando el dashboard de productos.
WEB-VAL-06	Comprobar que un usuario estándar puede registrarse usando un proveedor	La UI redirige a Google, autentica al usuario y

	social (Google) desde un botón en la página de registro.	registra la cuenta, mostrando el dashboard de productos
WEB-VAL-07	Verificar que un usuario estándar puede iniciar sesión con credenciales válidas desde el formulario de login y es redirigido a la vista principal de productos.	La UI muestra la vista principal de productos.
WEB-VAL-08	Verificar que un usuario estándar puede iniciar sesión usando Google desde un botón en la página de login.	La UI redirige a GitHub, autentica al usuario y carga la vista principal de productos.
WEB-VAL-09	Asegurar que un usuario estándar puede iniciar sesión usando GitHub desde un botón en la página de login.	La UI redirige a GitHub, autentica al usuario y carga la vista principal de productos.
WEB-VAL-10	Comprobar que un administrador puede iniciar sesión desde el formulario de login y es redirigido al dashboard de administración.	La UI muestra el dashboard de administrador.
WEB-VAL-11	Asegurar que un administrador puede iniciar sesión usando Google desde un botón en la página de login.	La UI redirige a Google, autentica al administrador y carga el dashboard de admin.
WEB-VAL-12	Validar que un administrador puede iniciar sesión usando GitHub desde un botón en la página de login.	La UI redirige a Github, autentica al administrador y carga el dashboard de admin.
WEB-VAL-13	Verificar que la vista principal de productos para usuarios estándar muestra una lista paginada de productos con filtros por categoría y precio, accesible desde el header.	La UI muestra productos paginados, los filtros aplican correctamente y los resultados se actualizan.
WEB-VAL-14	Probar que un usuario estándar puede buscar productos mediante un campo de búsqueda semántica en el header del frontend.	La UI muestra productos relevantes, ordenados por relevancia según la búsqueda.

WEB-VAL-15	Validar que la página de detalles de un producto muestra nombre, descripción, precio, enlace a Amazon y reseñas asociadas, ordenadas por fecha.	La UI carga detalles completos, un enlace clickable a Amazon y reseñas en orden cronológico.
WEB-VAL-16	Comprobar que un usuario estándar puede marcar un producto como favorito desde la vista principal o usando un botón.	El producto se marca como favorito, la UI muestra un ícono actualizado y aparece en favoritos.
WEB-VAL-17	Verificar que un usuario estándar puede desmarcar un producto desde la sección de favoritos.	El producto se elimina de favoritos y la lista se actualiza.
WEB-VAL-18	Asegurar que el chatbot para usuarios, permite buscar productos mediante una consulta semántica (e.g., "Muéstrame productos de electrónica bajo \$100")	La UI muestra una lista de productos relevantes, con nombres, precios y enlaces a detalles.
WEB-VAL-19	Probar que el chatbot para usuarios, responde a consultas sobre el producto utilizando información de sus reseñas (e.g., "¿Qué dicen las reseñas sobre este producto?")	La UI muestra un resumen de las reseñas (e.g., opiniones positivas/negativas) en la ventana del chatbot.
WEB-VAL-20	Verificar que el chatbot para usuarios maneja consultas ambiguas (e.g., "Dame algo") mostrando un mensaje de aclaración en la UI	La UI muestra "Por favor, especifica tu consulta" en la ventana del chatbot, sin errores.
WEB-VAL-21	Validar que el dashboard de administradores muestra tanto visualizaciones interactivas (gráficas y mapas de calor) como estadísticas.	La UI renderiza gráficas, estadísticas y mapas de calor funcionales con datos actualizados.
WEB-VAL-22	Probar que un administrador puede eliminar un usuario desde la pestaña de gestión de usuarios, confirmando la acción en un modal.	La lista de usuarios se actualiza tras confirmación.
WEB-VAL-23	Verificar que un administrador puede modificar el rol de un usuario (e.g., de estándar a administrador) desde la pestaña de gestión de usuarios.	El cambio se refleja en la lista de usuarios.

WEB-VAL-24	Comprobar que un administrador puede generar y descargar un reporte en PDF acerca de las estadísticas del sistema desde una página dedicada.	La UI inicia la descarga de un PDF con datos correctos de productos y reseñas.
WEB-VAL-25	Probar que el chatbot para administradores responde a consultas analíticas (e.g., "¿Cuántos usuarios están registrados como administrador?").	La UI muestra el número exacto de usuarios en la ventana del chatbot, sin errores.
WEB-VAL-26	Comprobar que el chatbot para administradores permite generar y descargar un reporte en PDF (e.g., "Genera un reporte de los productos con categoría Deportes que cuestan menos de 20 euros") mediante function calling desde el dashboard.	La UI inicia la descarga de un PDF con datos de productos, mostrando un mensaje de confirmación.
WEB-VAL-27	Verificar que el chatbot para administradores permite generar un gráfico interactivo (e.g., "Genera un gráfico de reseñas por categoría") mediante function calling desde el dashboard.	La UI muestra un gráfico interactivo (e.g., barras o líneas)
WEB-VAL-28	Validar que el chatbot para administradores permite eliminar un usuario o cambiar su rol mediante function calling (e.g., "Elimina al usuario con correo user@example.com") desde el dashboard.	La UI muestra un mensaje de confirmación, el usuario se elimina y la lista de usuarios se actualiza.
WEB-VAL-29	Asegurar que el header de navegación permite a usuarios estándar acceder a la vista principal, favoritos y búsqueda en todas las vistas tras el login.	El header muestra opciones relevantes y permite navegar sin errores.
WEB-VAL-30	Probar que la interfaz es responsive, y las vistas (bienvenida, login, registro, productos, dashboard) se adaptan a móviles, tablets y escritorios.	La UI se renderiza correctamente en todos los dispositivos, con elementos accesibles.
WEB-VAL-31	Comprobar que el chatbot para usuarios soporta consultas en múltiples idiomas (e.g., español, inglés) para buscar productos o reseñas.	La UI muestra resultados correctos en la ventana del chatbot, independientemente del

		idioma.
--	--	---------

Tabla 4.53 Pruebas de sistema para la aplicación

Todas las pruebas de sistema y validación alfa realizadas sobre la aplicación web, detalladas en las tablas anteriores, han sido superadas con éxito, verificando que el sistema cumple con los requisitos establecidos y funciona correctamente en todos los aspectos evaluados.

4.6 EL PRODUCTO DEL DESARROLLO

A continuación, se muestra el producto final, explicando con imágenes el funcionamiento del mismo.

4.6.1 SISTEMA DE DATOS

El sistema de datos de la aplicación se sustenta en una infraestructura contenerizada que garantiza un entorno aislado, escalable y coordinado para el procesamiento de datos en tiempo real. Esta infraestructura utiliza Docker para gestionar tres contenedores clave: Kafka, que facilita la transmisión de mensajes entre componentes; ZooKeeper, que asegura la coordinación y consistencia del clúster de mensajería; y Kafka-UI, que proporciona una interfaz visual para monitorear el estado de los tópicos y el clúster. Los contenedores en ejecución, observables mediante una herramienta de gestión, evidencian la actividad operativa del sistema durante el procesamiento de datos de productos y reseñas.



Figura 4.68 Contenedores del sistema de datos en ejecución

El sistema de datos de la aplicación orquesta el flujo continuo de datos de productos y reseñas, garantizando un almacenamiento eficiente que habilita funcionalidades de búsqueda y análisis en tiempo real. A continuación, se detalla su funcionamiento operativo, ilustrado con capturas que reflejan el comportamiento dinámico del sistema durante la ejecución.

Funcionamiento del Productor

El Productor inicia el flujo de datos al procesar archivos de entrada y enviar información filtrada a través de un sistema de mensajería distribuida. Lee los datos secuencialmente, crea un índice auxiliar para optimizar el acceso a las reseñas y transmite los datos aprobados a los canales correspondientes. Este proceso asegura una entrega ordenada y estable, con monitorización constante para detectar anomalías en el flujo. La interfaz de Kafka-UI muestra los tópicos activos con el conteo de mensajes enviados, confirmando la transmisión en tiempo real.

Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size
IN __consumer_offsets	50	0	1	599	585 Bytes
products	1	0	1	1	6 KB
reviews	1	0	1	8	4 KB

Figura 4.69 Tópicos Activos en Kafka-UI

El Productor registra cada operación en una consola, mostrando mensajes como “Procesando producto...” y “Enviado a tópicos”, lo que permite rastrear el progreso en tiempo real y diagnosticar posibles interrupciones. Este registro continuo, combinado con un mecanismo de reanudación desde el último punto procesado usando el archivo `last_sent.json`, asegura la continuidad del flujo de datos incluso en escenarios de fallo.

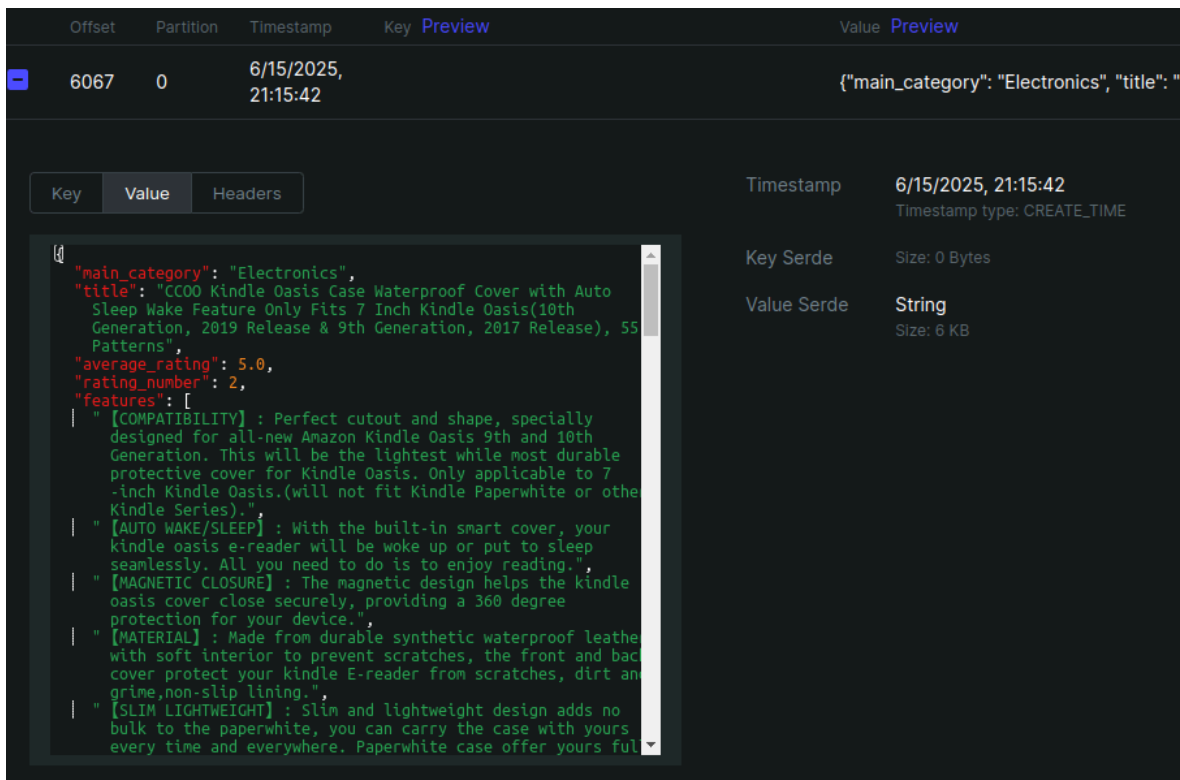
```
--- Procesando línea 63050 del archivo ---  
Producto actual: B096RJG2X6, Último producto enviado: B06XG3BXG1  
Procesando producto B096RJG2X6...  
Procesando producto B096RJG2X6...  
Aplicando filtros a producto B096RJG2X6, número de reseñas: 8  
Producto aprobado.  
Producto B096RJG2X6 aprobado y será enviado.  
Producto B096RJG2X6 aprobado. Preparando para enviar...  
Enviado a products: B096RJG2X6  
Producto B096RJG2X6 enviado exitosamente.  
Enviado a reviews: B096RJG2X6  
Enviado a reviews: B096RJG2X6  
Enviado a reviews: B096RJG2X6  
Enviado a reviews: B096RJG2X6  
Enviado a reviews: B096RJG2X6  
Enviado a reviews: B096RJG2X6  
Enviado a reviews: B096RJG2X6  
Enviado a reviews: B096RJG2X6  
Se enviaron 8 reviews para el producto B096RJG2X6.
```

Figura 4.70 Consola del Productor Mostrando Procesamiento y Envío de Dato

Funcionamiento del Consumidor

El Consumidor recoge los mensajes enviados por el Productor y los transfiere al sistema de almacenamiento final. Procesa los datos en dos fases secuenciales, primero productos y

luego reseñas, asegurando vinculación correcta. Un mensaje en un tópico, visible en Kafka-UI, muestra el contenido procesado, como datos de un producto o reseña



Offset	Partition	Timestamp	Key	Value
6067	0	6/15/2025, 21:15:42		{"main_category": "Electronics", "title": "CC00 Kindle Oasis Case Waterproof Cover with Auto Sleep Wake Feature Only Fits 7 Inch Kindle Oasis(10th Generation, 2019 Release & 9th Generation, 2017 Release), 55 Patterns", "average_rating": 5.0, "rating_number": 2, "features": [{"COMPATIBILITY": "Perfect cutout and shape, specially designed for all-new Amazon Kindle Oasis 9th and 10th Generation. This will be the lightest while most durable protective cover for Kindle Oasis. Only applicable to 7-inch Kindle Oasis.(will not fit Kindle Paperwhite or other Kindle Series).", "AUTO_WAKE/SLEEP": "With the built-in smart cover, your kindle oasis e-reader will be woke up or put to sleep seamlessly. All you need to do is to enjoy reading.", "MAGNETIC_CLOSURE": "The magnetic design helps the kindle oasis cover close securely, providing a 360 degree protection for your device.", "MATERIAL": "Made from durable synthetic waterproof leather with soft interior to prevent scratches, the front and back cover protect your kindle E-reader from scratches, dirt and grime,non-slip lining.", "SLIM_LIGHTWEIGHT": "Slim and lightweight design adds no bulk to the paperwhite, you can carry the case with yours every time and everywhere. Paperwhite case offer yours full"}]}

Key Value Headers

```
"main_category": "Electronics",
"title": "CC00 Kindle Oasis Case Waterproof Cover with Auto
Sleep Wake Feature Only Fits 7 Inch Kindle Oasis(10th
Generation, 2019 Release & 9th Generation, 2017 Release), 55
Patterns",
"average_rating": 5.0,
"rating_number": 2,
"features": [
| " [COMPATIBILITY] : Perfect cutout and shape, specially
designed for all-new Amazon Kindle Oasis 9th and 10th
Generation. This will be the lightest while most durable
protective cover for Kindle Oasis. Only applicable to 7
-inch Kindle Oasis.(will not fit Kindle Paperwhite or other
Kindle Series).",
| " [AUTO_WAKE/SLEEP] : With the built-in smart cover, your
kindle oasis e-reader will be woke up or put to sleep
seamlessly. All you need to do is to enjoy reading.",
| " [MAGNETIC_CLOSURE] : The magnetic design helps the kindle
oasis cover close securely, providing a 360 degree
protection for your device.",
| " [MATERIAL] : Made from durable synthetic waterproof leather
with soft interior to prevent scratches, the front and back
cover protect your kindle E-reader from scratches, dirt and
grime,non-slip lining.",
| " [SLIM_LIGHTWEIGHT] : Slim and lightweight design adds no
bulk to the paperwhite, you can carry the case with yours
every time and everywhere. Paperwhite case offer yours full
```

Timestamp

6/15/2025, 21:15:42
Timestamp type: CREATE_TIME

Key Serde

Size: 0 Bytes

Value Serde

String
Size: 6 KB

Figura 4.71 Contenido de un Mensaje de Tópico en Kafka-UI Procesado por el Consumidor

Cada mensaje se envía al backend mediante solicitudes autenticadas. Los logs del Consumidor detallan una solicitud POST, incluyendo endpoint, carga útil y respuesta, confirmando la transferencia exitosa

```
Initializing Kafka consumer...
Kafka consumer initialized successfully.
API headers set: {'Authorization': 'Bearer eyJhbGciOiJIUzI1NiIsInR5c
czfQ.3WlsTdLX2yMJx6 k82evoqM3w-2Ch6nrJ40-vvWdXmE', 'Content-Type': '
=== STARTING PRODUCTS PROCESSING PHASE ===
Processing product: B096RJG2X6
Sending POST to http://localhost:5000/api/v0/products/ with payload:
  "main_category": "Electronics",
  "title": "CC00 Kindle Oasis Case Waterproof Cover with Auto Sleep",
  "average_rating": 5.0,
  "rating_number": 2,
  "features": [
    "\u00a0COMPATIBILITY\u00a0: Perfect cutout and shape, specially
r Kindle Oasis. Only applicable to 7-inch Kindle Oasis.(will not fit
    "\u00a0AUTO WAKE/SLEEP\u00a0: With the built-in smart cover, you
    "\u00a0MAGNETIC CLOSURE\u00a0: The magnetic design helps the kin
    "\u00a0MATERIAL\u00a0: Made from durable synthetic waterproof le
```

Figura 4.72 Logs del Consumidor Detallando una Solicitud POST Exitosa

Integración con el Backend

El backend recibe los datos del consumidor, los valida y los almacena en una base de datos relacional, procesados previamente para generar representaciones numéricas que habilitan la búsqueda semántica en la aplicación. Este paso asegura que los datos estén disponibles para consultas rápidas y precisas por parte de los usuarios. Una tabla en PostgreSQL muestra filas de productos o reseñas, con sus atributos correspondientes, verificando que el almacenamiento se completó correctamente.

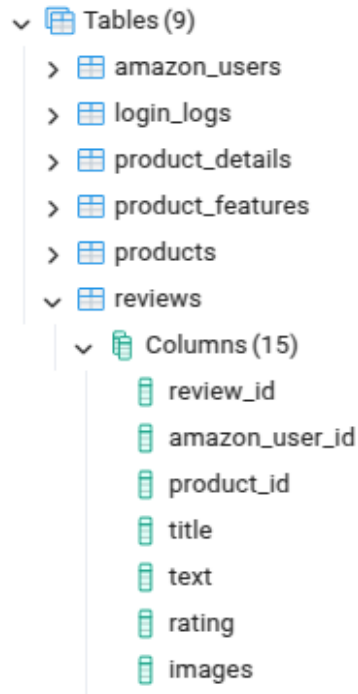


Figura 4.73 Tablas almacenadas en PostgreSQL

El sistema de datos garantiza un flujo continuo y escalable de información mediante un sistema de mensajería monitoreado en tiempo real, que asegura alta disponibilidad y capacidad para gestionar volúmenes crecientes de datos. Esta infraestructura robusta, soportada por una coordinación precisa entre componentes, consolida su rol como pilar fundamental de la aplicación de comercio electrónico, habilitando funcionalidades críticas de búsqueda y análisis. El sistema de datos garantiza un flujo continuo y escalable de información mediante un sistema de mensajería monitoreado en tiempo real, que asegura alta disponibilidad y capacidad para gestionar volúmenes crecientes de datos. Esta infraestructura robusta, soportada por una coordinación precisa entre componentes, consolida su rol como pilar fundamental de la aplicación de comercio electrónico, habilitando funcionalidades críticas de búsqueda y análisis.

4.6.2 LA APLICACIÓN WEB

USUARIOS NORMALES

La aplicación web sirve como el segundo componente clave del sistema, proporcionando una experiencia interactiva y adaptativa. El proceso inicia en una pantalla de bienvenida que presenta la plataforma y dirige a los usuarios hacia la página de login para comenzar su interacción.

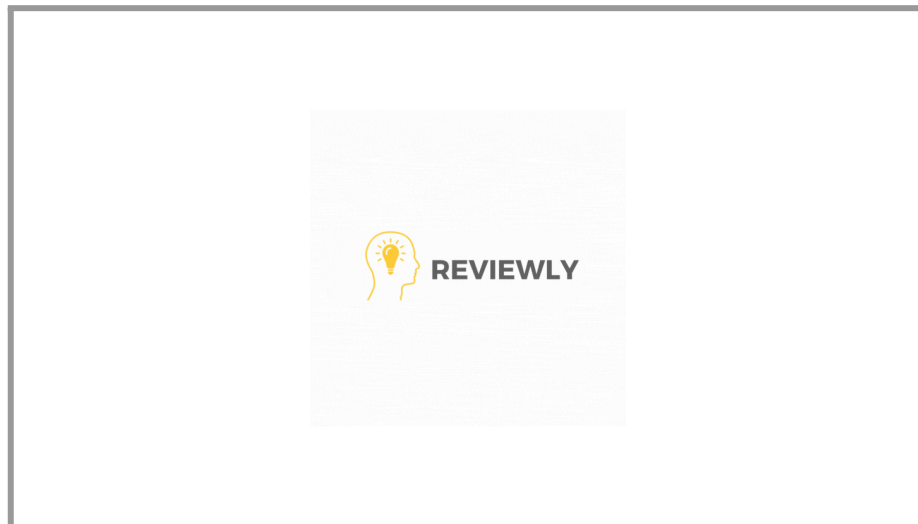


Figura 4.74 Bienvenida a la aplicación

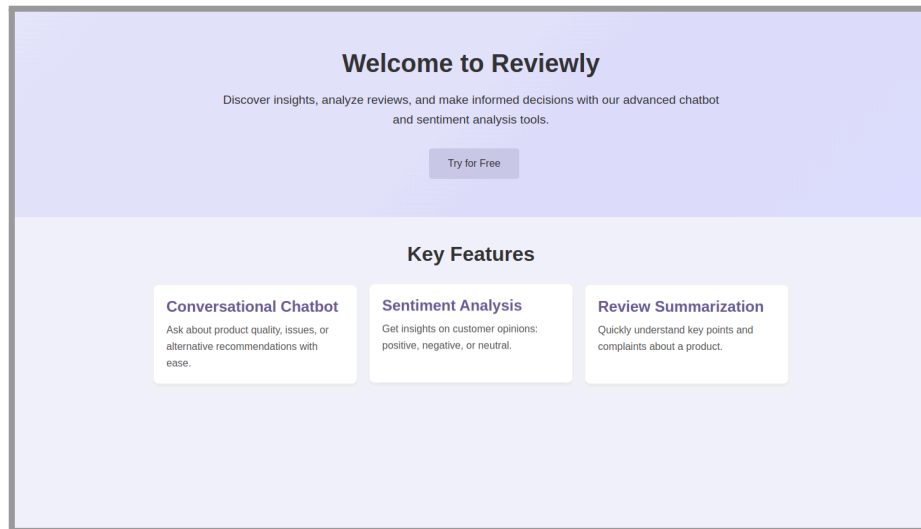


Figura 4.75 Landing page

Desde la página inicial, los usuarios acceden a la página de login, donde pueden ingresar sus credenciales para autenticarse y acceder al sistema. Esta etapa asegura un control de acceso básico antes de redirigir al usuario según su rol.

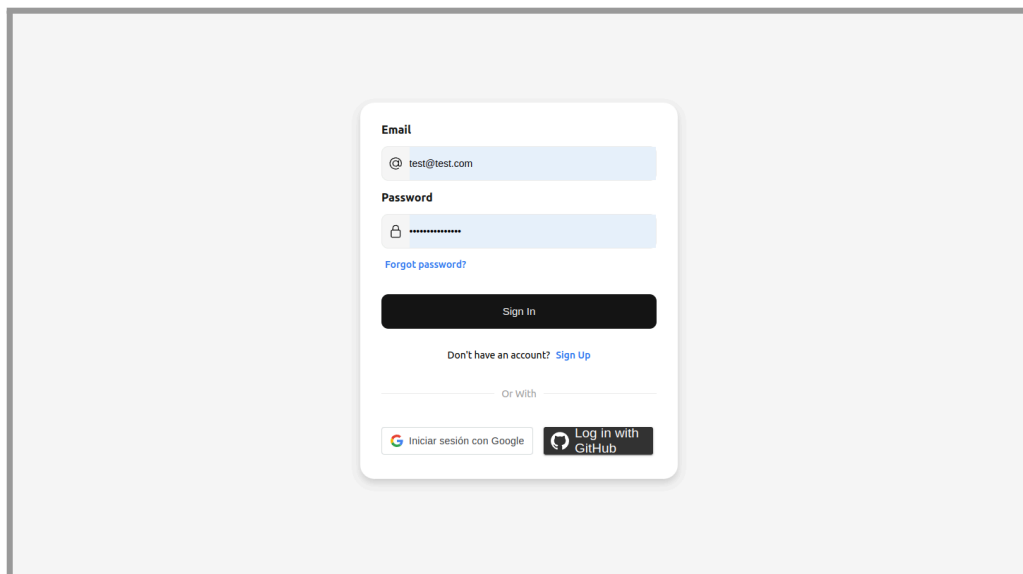
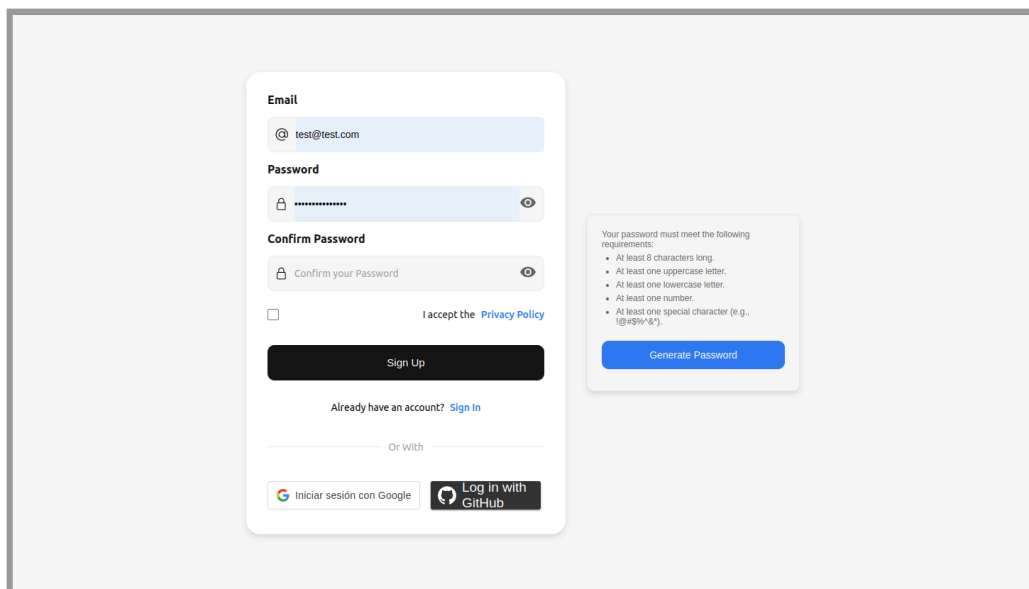


Figura 4.76 Página de login

Alternativamente, los nuevos usuarios pueden clicar en "*Sign up*" para ir a la página de inicio de sesión, donde pueden registrarse proporcionando la información necesaria para crear una cuenta, permitiendo su incorporación al sistema. Esta página ofrece flexibilidad al permitir el registro de manera tradicional, ingresando manualmente datos como correo electrónico y contraseña, o mediante proveedores sociales como Google o Github, lo que simplifica el proceso. Además, el sistema incluye una funcionalidad para generar contraseñas seguras automáticamente, asegurando que los usuarios puedan optar por una opción robusta si lo desean, facilitando así una experiencia de registro adaptable y segura.



The screenshot displays a 'Sign Up' form with the following elements:

- Email:** A text input field containing 'test@test.com'.
- Password:** A text input field with masked characters and a visibility toggle icon.
- Confirm Password:** A text input field with the placeholder 'Confirm your Password' and a visibility toggle icon.
- Privacy Policy:** A checkbox followed by the text 'I accept the [Privacy Policy](#)'.
- Sign Up:** A large black button.
- Sign In:** A link labeled 'Already have an account? [Sign In](#)'.
- Or With:** A horizontal separator line with the text 'Or With'.
- Google:** A button labeled 'Iniciar sesión con Google'.
- Github:** A button labeled 'Log in with GitHub'.

To the right of the form, a box lists password requirements:

- Your password must meet the following requirements:
- At least 8 characters long.
- At least one uppercase letter.
- At least one lowercase letter.
- At least one number.
- At least one special character (e.g., !@#\$%^&*).

Below these requirements is a blue button labeled 'Generate Password'.

Figura 4.77 Página de inicio de sesión

Una vez autenticados, los usuarios estándar acceden a la vista principal, que presenta un catálogo completo de productos. Esta pantalla inicial ofrece una visión general del listado disponible, permitiendo a los usuarios comenzar su exploración de manera intuitiva.

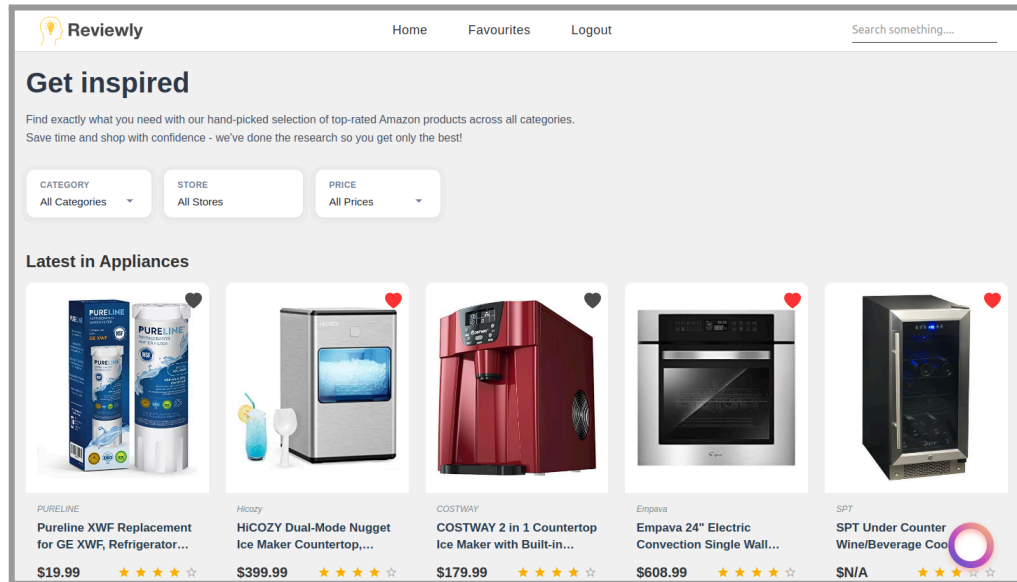


Figura 4.78 Página principal de los usuarios

Desde la vista principal, los usuarios pueden realizar una **búsqueda de un producto** ingresando términos o descripciones específicas en la barra superior derecha. Esta acción activa el sistema de búsqueda, que procesa la solicitud para localizar productos relevantes dentro del catálogo. Además, cuando los usuarios introducen un término, en lugar de sus búsquedas anteriores, se mostrarán resultados rápidos que coincidan con el término introducido, mejorando la eficiencia de la navegación.

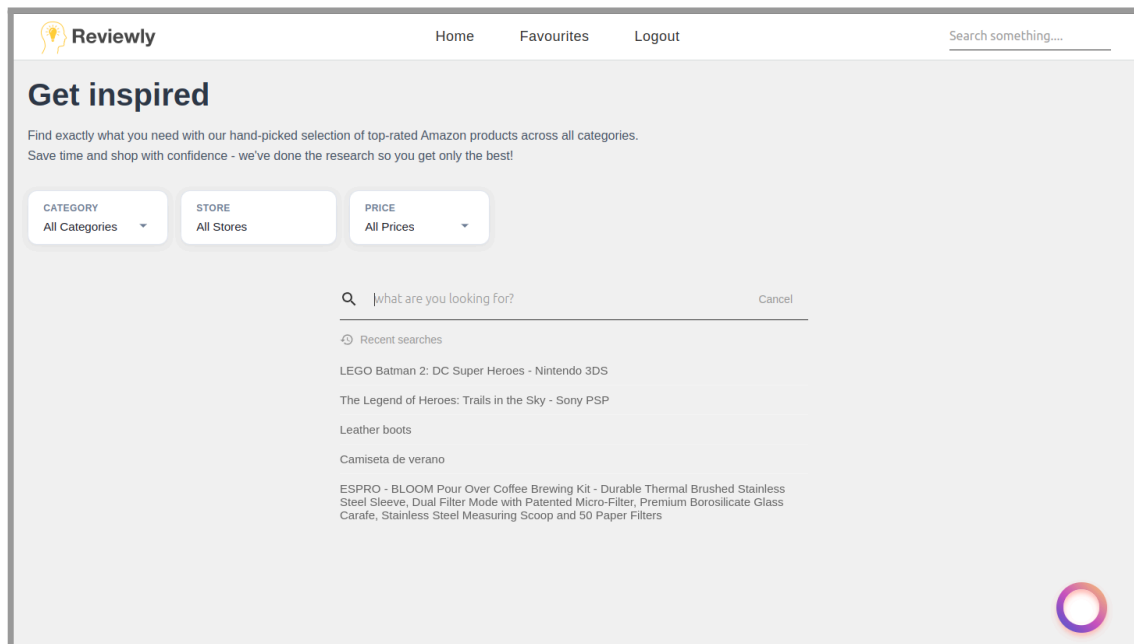


Figura 4.79 Búsqueda de producto con búsquedas recientes

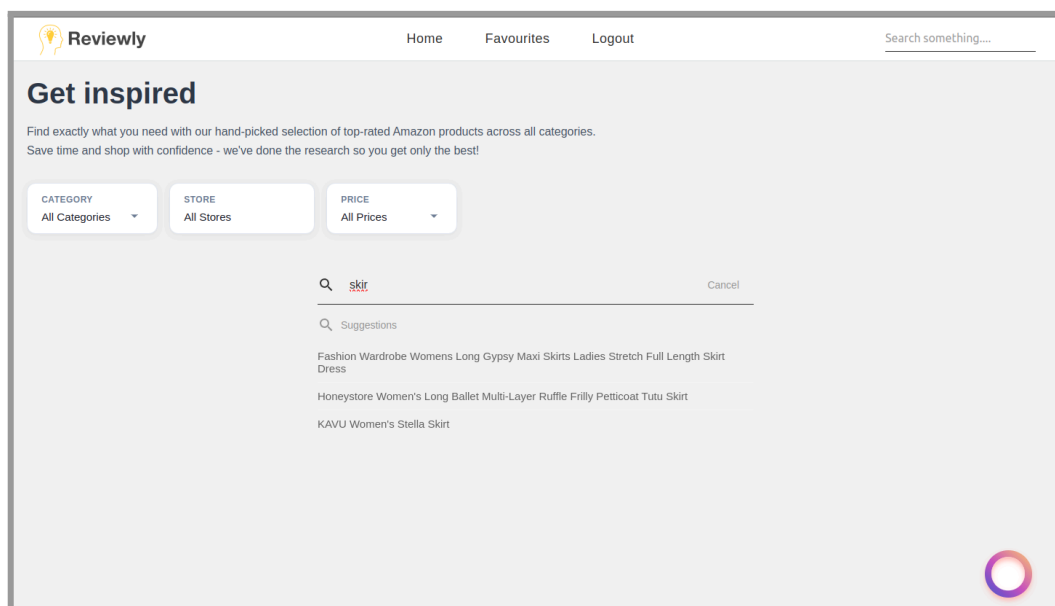


Figura 4.80 Búsqueda de producto con sugerencias de búsqueda

Tras la búsqueda, los usuarios reciben los resultados en una lista ordenada que muestra los productos encontrados. Esta sección permite revisar las opciones disponibles y seleccionar aquellos de interés para obtener más detalles.

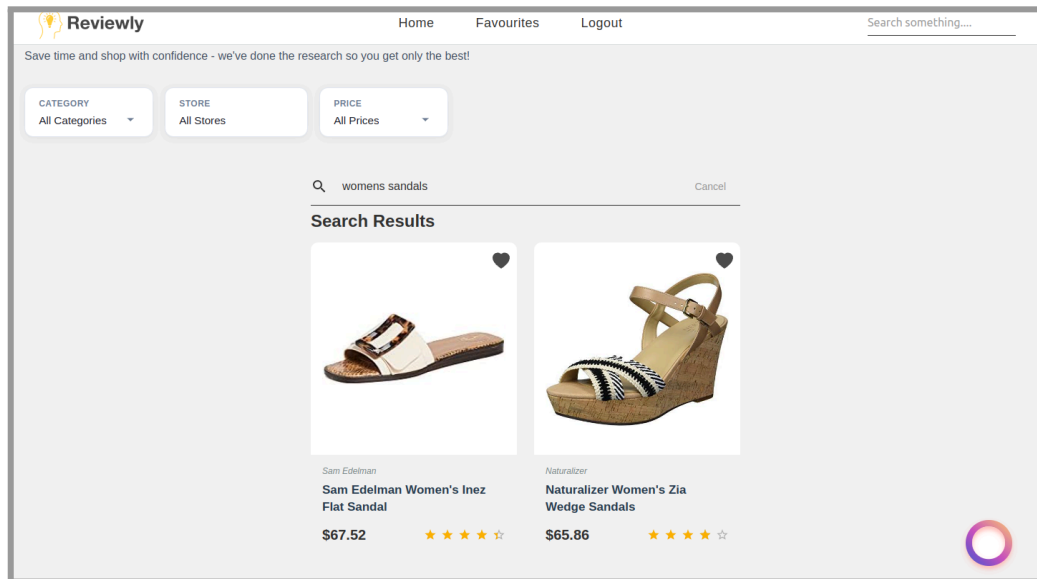


Figura 4.81 Resultados de búsqueda de producto

Al seleccionar un producto de la lista de resultados, los usuarios accedan a la vista detallada del producto, donde se presenta información completa sobre el artículo elegido. Esta pantalla incluye imágenes de alta calidad, enlace a amazon, reviews y especificaciones técnicas. Además, los usuarios pueden realizar consultas específicas al chatbot integrado, optimizando la experiencia de compra y permitiendo una interacción fluida con el sistema

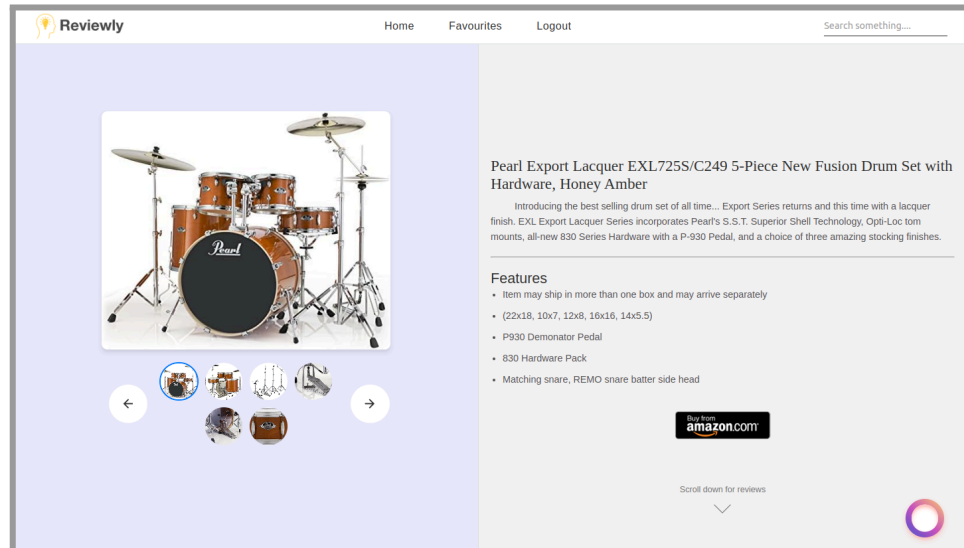


Figura 4.82 Vista detallada de producto

Además, los usuarios pueden realizar consultas específicas al chatbot integrado, optimizando la experiencia de compra y permitiendo una interacción fluida con el sistema. Las respuestas proporcionadas por el chatbot se pueden generar a partir del análisis de las reseñas disponibles del producto, indicando de forma clara de qué opiniones o valoraciones se ha extraído la información mostrada.

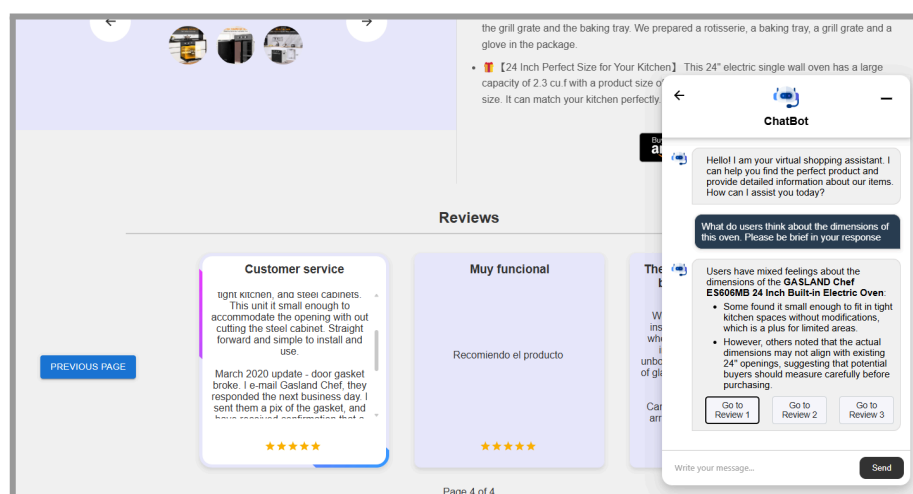


Figura 4.83 Consulta del usuario al chatbot con respuesta basada en reseñas del producto

Antes de interactuar con el chatbot, los usuarios acceden a la **pantalla inicial del chatbot**, donde pueden seleccionar el proveedor de IA con el que desean interactuar, ofreciendo flexibilidad para elegir entre diferentes modelos de inteligencia artificial según sus preferencias o necesidades específicas.

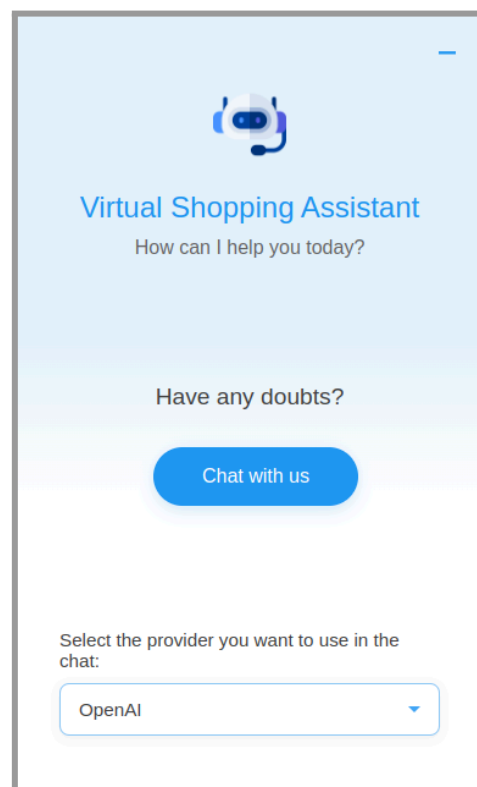


Figura 4.84 Landing page del chatbot

Además, los usuarios pueden realizar una **búsqueda de un producto con el chatbot**, interactuando directamente con esta herramienta para refinar sus criterios. La búsqueda se puede realizar en cualquier idioma, ampliando su accesibilidad. El chatbot asiste al procesar descripciones, filtrar resultados y ofrecer recomendaciones personalizadas basadas en preferencias previas, mejorando la experiencia de búsqueda.

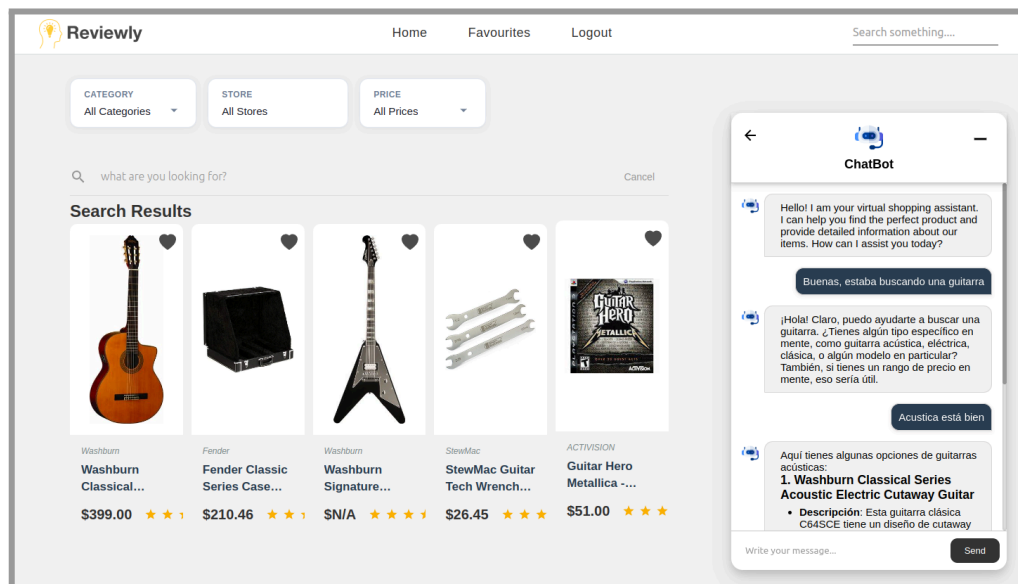


Figura 4.85 Búsqueda de producto mediante el chatbot

Finalmente, los usuarios tienen la opción de **guardar productos como favoritos**, seleccionando aquellos que desean conservar para un acceso posterior. Esta funcionalidad, apoyada por el chatbot, facilita la gestión de preferencias y sugiere nuevos productos según el historial del usuario.

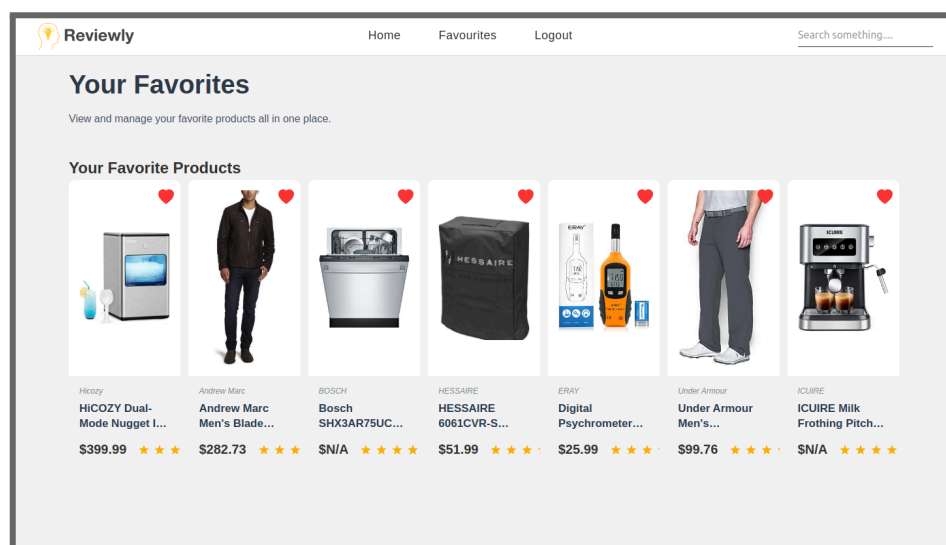


Figura 4.86 Página de favoritos

ADAPTABILIDAD A DISPOSITIVOS MÓVILES

La aplicación está diseñada con un enfoque responsive, garantizando una experiencia de usuario óptima tanto en ordenadores de escritorio como en dispositivos móviles.

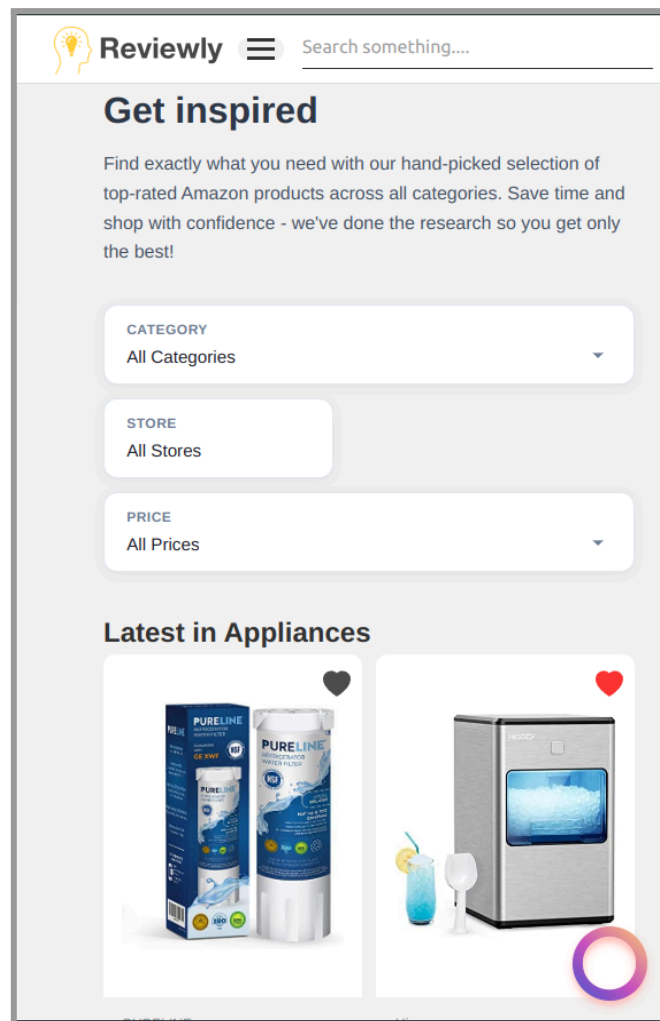


Figura 4.87 Vista móvil del catálogo de productos

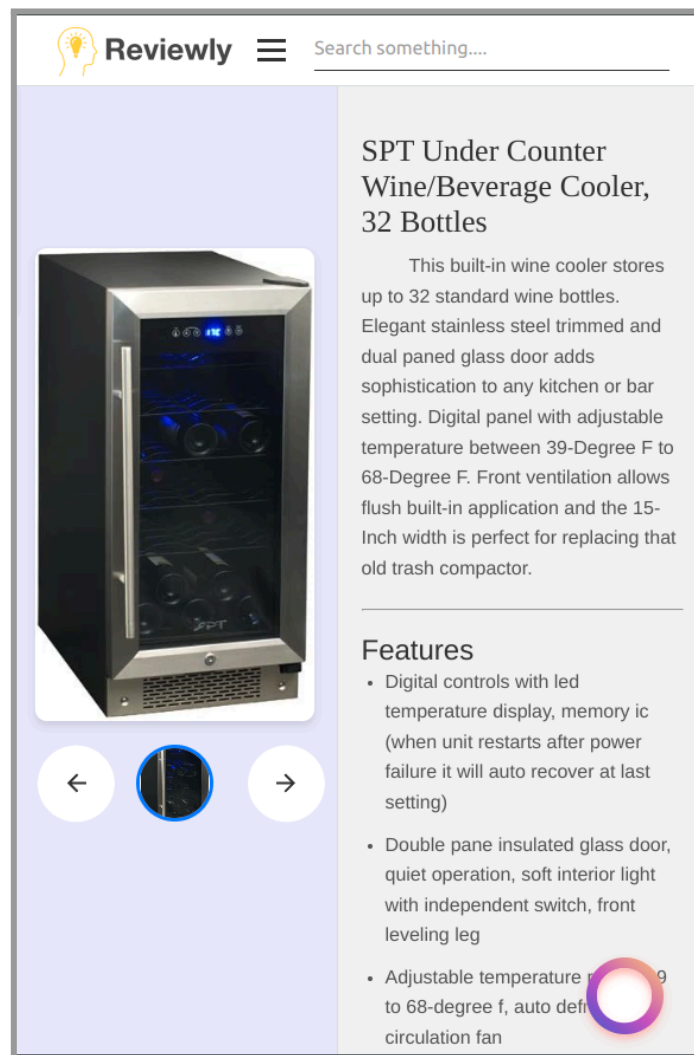


Figura 4.88 Vista móvil del detalle de producto

USUARIOS ADMINISTRADORES

Tras la autenticación, los administradores ingresan a una interfaz centralizada conocida como el dashboard de administración, que actúa como el núcleo de control para supervisar y gestionar el sistema. Esta plataforma ofrece una visión integral del funcionamiento general, estableciendo el punto de partida para las tareas administrativas, incluyendo un heatmap de logs del usuario que muestra patrones de actividad y un gráfico que detalla los tiempos de ejecución de la búsqueda de productos con percentiles, facilitando el análisis del rendimiento.

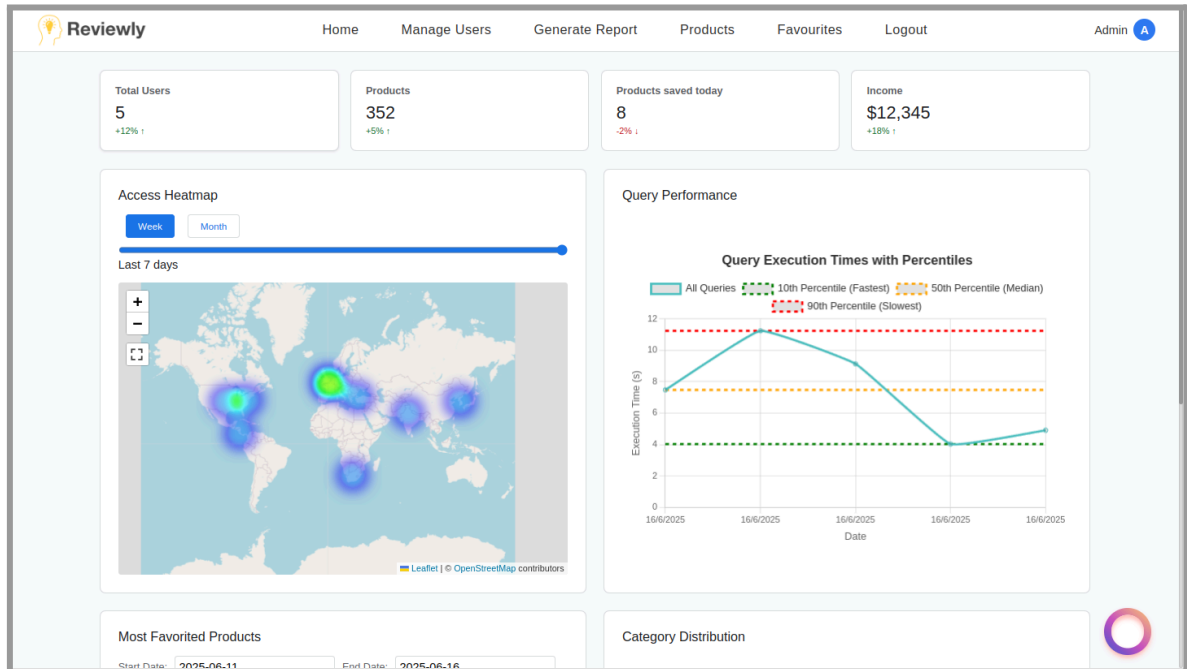


Figura 4.89 Página principal del administrador

Dentro del dashboard, los administradores pueden acceder a herramientas visuales que destacan los productos más guardados en favoritos junto con una distribución de categorías de estos productos, proporcionando una representación gráfica clara que ayuda a identificar tendencias de preferencia entre los usuarios.

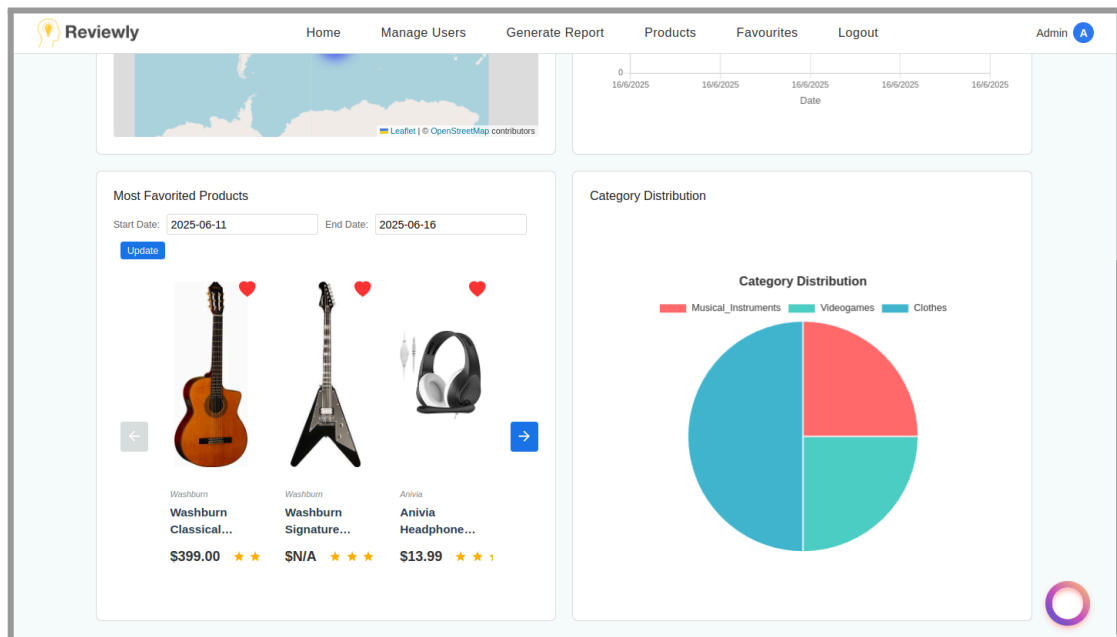


Figura 4.90 Productos más guardados en favoritos en la página del administrador

Desde el dashboard, los administradores pueden navegar a la página de manage users, donde gestionan la lista de usuarios registrados. Esta sección permite realizar tareas como visualizar perfiles, modificar roles o eliminar cuentas según sea necesario.

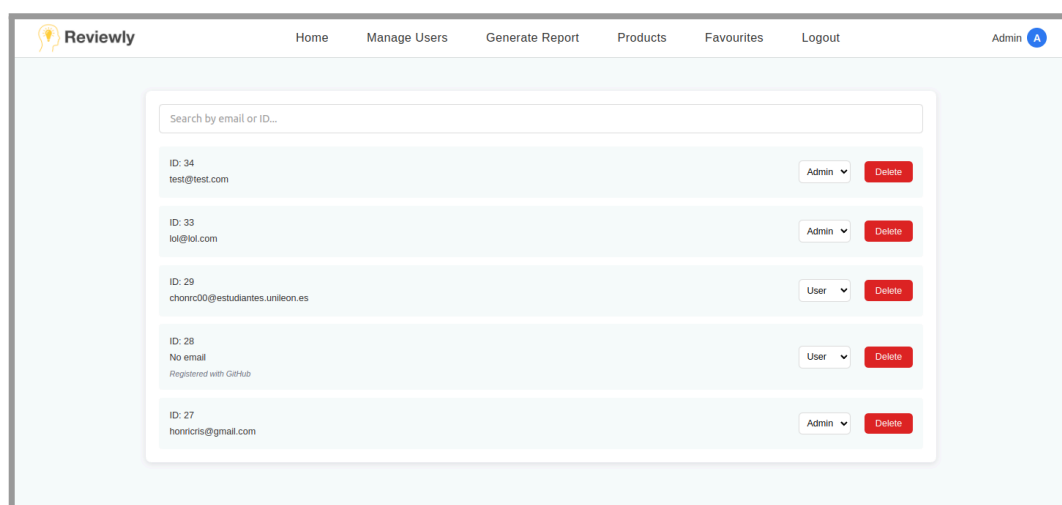


Figura 4.91 Página de manejo de usuarios

La página de generate report permite a los administradores crear informes detallados sobre el rendimiento del sistema, incluyendo datos de usuarios, productos y actividad general. Esta funcionalidad soporta la generación de reportes personalizados para análisis más profundos.

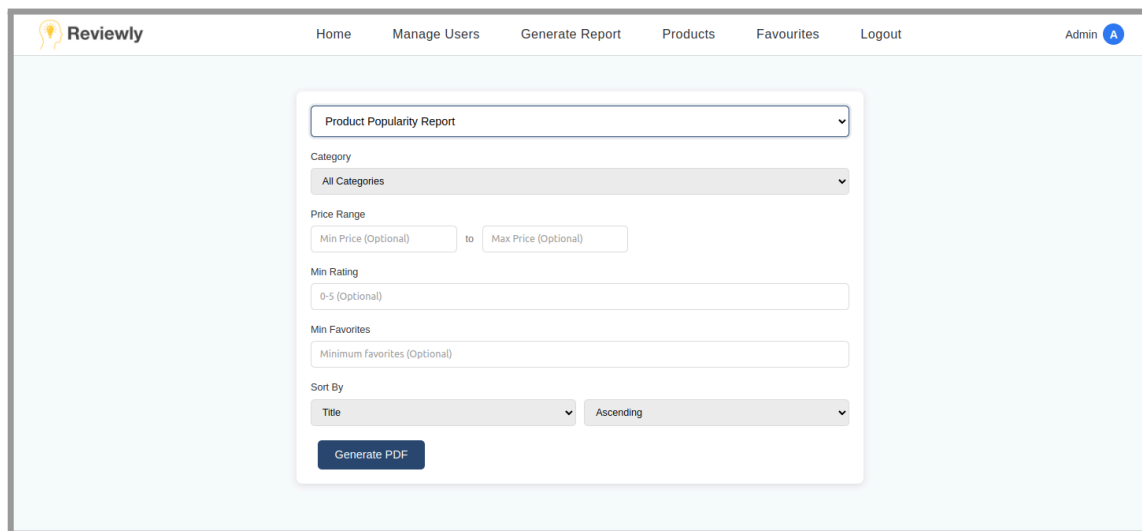


Figura 4.92 Página de creación de informes

Los administradores también pueden interactuar con el chatbot para gestionar usuarios, como en el caso de eliminar cuentas específicas. Por ejemplo, el chatbot puede procesar una solicitud para eliminar a todos los usuarios con correos "@test.com", ejecutando la acción de manera eficiente tras confirmación. Para ello, el chatbot primero utiliza una función de búsqueda seleccionada mediante function calling para identificar a los usuarios que coincidan con el criterio, y posteriormente emplea una segunda función distinta, también elegida con function calling, para proceder con la eliminación de los usuarios encontrados.

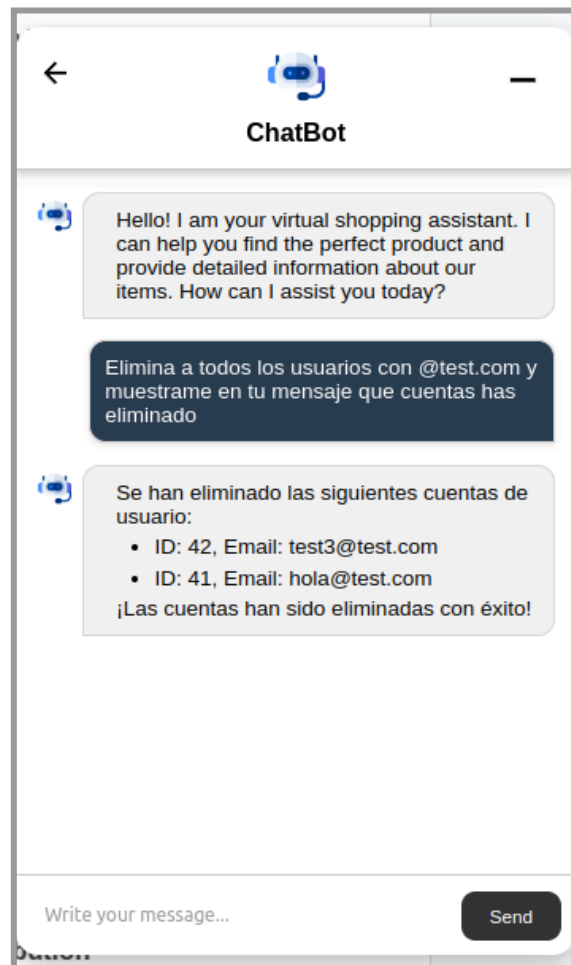


Figura 4.93 Ejemplo de manejo de usuarios con el chatbot

Asimismo, el chatbot permite generar reportes personalizados, sirviendo como una manera simplificada de la pestaña de generar informe. Al solicitar un informe con lenguaje natural, el chatbot puede crear un documento detallado con datos actualizados, como estadísticas de productos o resúmenes de actividad, adaptándose a las necesidades del administrador.

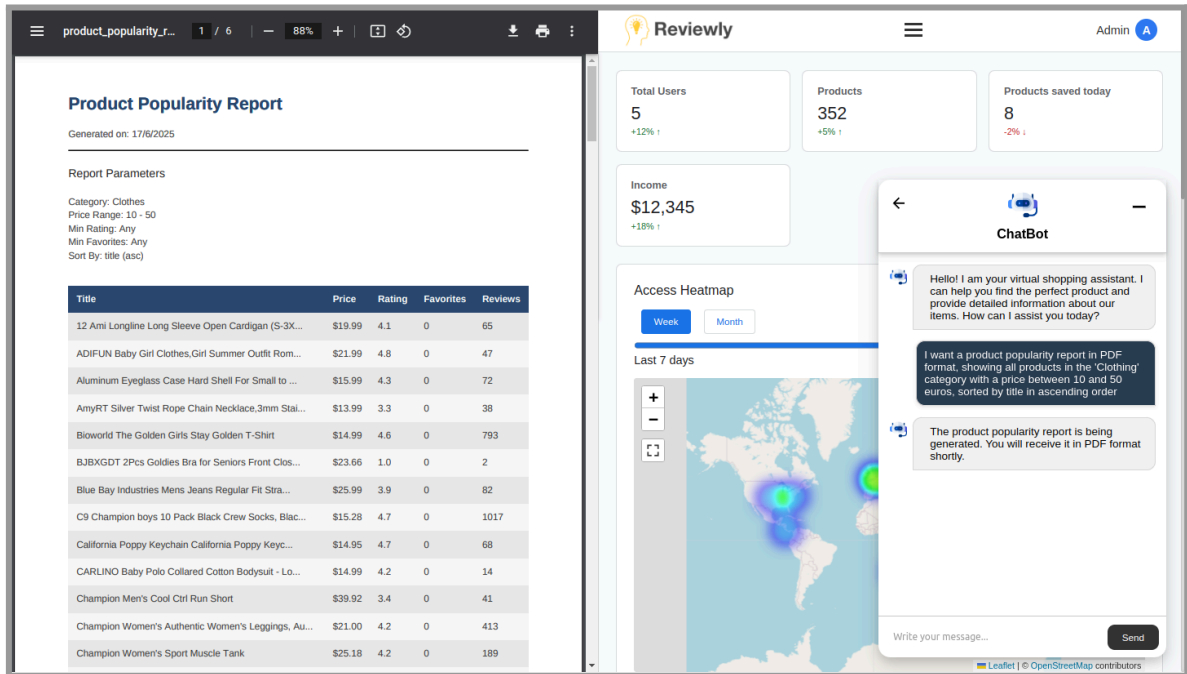


Figura 4.94 Ejemplo de generación de un reporte con el chatbot

Además, el chatbot ofrece una completa funcionalidad para generar gráficos interactivos, permitiendo a los administradores buscar datos utilizando todas sus funciones, como análisis de usuarios, productos, reseñas o actividad del sistema, y presentarlos visualmente. Por ejemplo, puede mostrar la distribución de reseñas por categoría de producto o cualquier otro dato solicitado, ayudando a identificar áreas de interés o mejora de manera efectiva.

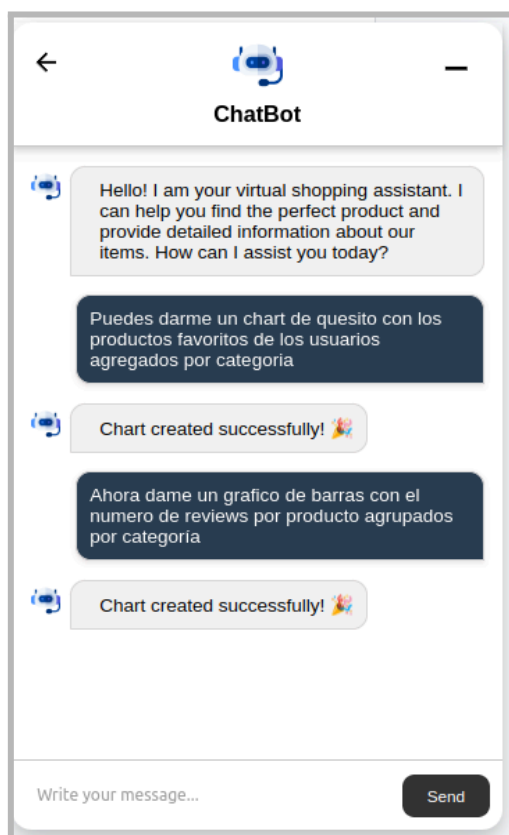


Figura 4.95 Ejemplo de consulta para crear gráficos

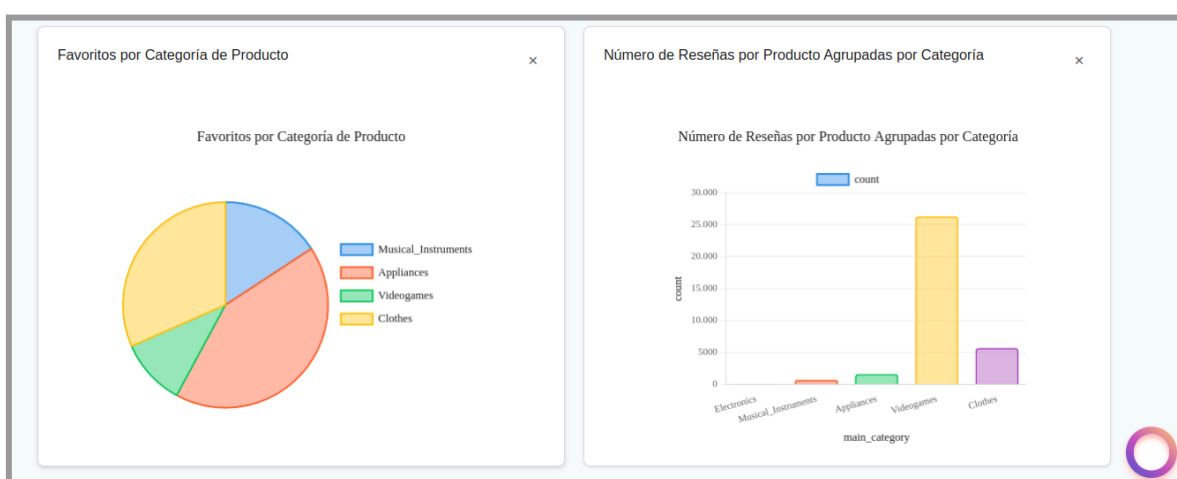


Figura 4.96 Gráficos creados por el chatbot

5 CONCLUSIONES

Como se ha mencionado anteriormente, este proyecto aborda tres problemas principales en el contexto de las plataformas de comercio electrónico: ahorrar tiempo a los consumidores, construir un motor de búsqueda eficiente y proporcionar a los administradores capacidades analíticas avanzadas mediante lenguaje natural. A continuación, se presenta un resumen de cómo se han abordado estos problemas.

El primer problema, ahorrar tiempo a los consumidores, se resolvió mediante el desarrollo de una aplicación web que utiliza un chatbot basado en inteligencia artificial generativa y Retrieval-Augmented Generation (RAG). Este sistema permite a los usuarios realizar búsquedas de productos mediante lenguaje natural, devolviendo tanto productos como reseñas de productos relevantes para responder a las consultas, eliminando la necesidad de múltiples consultas manuales o filtrados complejos. La integración de embeddings semánticos y búsqueda vectorial asegura que las recomendaciones sean rápidas, precisas y relevantes, mejorando significativamente la experiencia del usuario.

El segundo problema, construir un motor de búsqueda eficiente, se logró implementando tecnologías avanzadas como el modelo `stella_en_v5` para la generación de embeddings y una base de datos PostgreSQL con soporte para vectores (`pgvector`). Estas herramientas permitieron crear un sistema robusto que entrega resultados personalizados con alta precisión mediante búsquedas semánticas basadas en la comprensión del contexto.

Adicionalmente, se empleó Apache Kafka para la ingesta y procesamiento de datos en tiempo real, asegurando que la plataforma pueda manejar grandes volúmenes de información de manera eficiente y manteniendo los datos de productos y usuarios actualizados.

Por último, el tercer problema, proporcionar capacidades analíticas avanzadas a los administradores, se abordó mediante la integración de Function Calling en el chatbot, permitiendo la generación de reportes personalizados, gráficos interactivos y mapas de

calor a partir de consultas en lenguaje natural. Estas funcionalidades facilitan la toma de decisiones basada en datos, optimizando la gestión de la plataforma.

5.1 APORTACIONES REALIZADAS

Esta sección presenta las principales contribuciones realizadas en el desarrollo de la plataforma, destacando la integración de tecnologías avanzadas para mejorar la experiencia del usuario y la eficiencia del sistema. Se describen las soluciones implementadas para optimizar la interacción, búsqueda, procesamiento de datos, interfaz de usuario y despliegue, enfocándose en ofrecer un sistema funcional, escalable y centrado en las necesidades del usuario.

- **Implementación de IA Generativa y Chatbot con RAG:** La integración de modelos de lenguaje de gran escala (LLMs), como GPT-4o-mini a través de OpenRouter, junto con Retrieval-Augmented Generation (RAG) y Function Calling, permitió crear un asistente conversacional eficiente. Este chatbot permite a los usuarios interactuar con la plataforma mediante lenguaje natural, ofreciendo respuestas rápidas y personalizadas para búsquedas de productos y consultas administrativas, incluyendo la devolución de productos y reseñas relevantes. La integración con Function Calling añade la capacidad de ejecutar tareas analíticas complejas, como generar reportes en PDF o gráficos dinámicos. Mejorando la experiencia del usuario y la eficiencia administrativa en tiempo real.
- **Búsqueda Semántica Eficiente con stella_en_v5 y pgvector:** La combinación del modelo stella_en_v5 para generar embeddings y la base de datos PostgreSQL con soporte para vectores (pgvector) asegura búsquedas semánticas rápidas y relevantes, basadas en la comprensión del contexto en lugar de coincidencias de palabras clave. Esto supera las limitaciones de los motores de búsqueda tradicionales, mejorando la precisión de las recomendaciones de productos y reduciendo el tiempo necesario para encontrar artículos relevantes.

- **Procesamiento de Datos en Tiempo Real con Apache Kafka:** Apache Kafka gestiona la ingesta y distribución de datos en tiempo real, permitiendo que la plataforma procese grandes volúmenes de información de manera eficiente. Esto asegura que los datos de productos y usuarios estén siempre actualizados, mejorando la escalabilidad y el rendimiento del sistema.
- **Frontend Intuitivo con React y React Router:** La interfaz de usuario desarrollada con React ofrece una experiencia fluida y responsiva, adaptable a diferentes dispositivos. React Router facilita la navegación entre secciones de la aplicación, mejorando la accesibilidad y usabilidad para usuarios y administradores.
- **Despliegue Robusto con Docker:** La contenerización de los servicios (frontend, backend, base de datos y Kafka) mediante Docker y docker-compose.yml asegura un entorno de desarrollo y producción consistente y portable. Esto simplifica el desarrollo, las pruebas y el escalado, garantizando un sistema robusto y fácil de mantener.

5.2 TRABAJOS FUTUROS

En esta sección se describen las mejoras y funcionalidades planificadas para el desarrollo futuro de la aplicación, con el objetivo de optimizar su rendimiento, escalabilidad y experiencia de usuario. Estas propuestas abarcan desde el despliegue en la nube hasta la integración de técnicas avanzadas de inteligencia artificial para enriquecer las capacidades del sistema.

- **Despliegue en un Host Público:** Implementar el despliegue de la aplicación en un host público utilizando servicios en la nube como AWS, Google Cloud o Azure. Esto incluiría la configuración de un clúster de Kubernetes para orquestar los contenedores Docker, asegurando alta disponibilidad, escalabilidad automática y acceso global, lo que permitiría a los usuarios interactuar con la plataforma desde cualquier ubicación.

- **Análisis de Sentimientos de Reseñas:** Incorporar el análisis de sentimientos de las reseñas de productos, un campo existente en la base de datos pero no implementado. Esto implicaría usar modelos de procesamiento de lenguaje natural para clasificar las reseñas como positivas, negativas o neutrales, proporcionando a los usuarios y administradores información adicional sobre la percepción de los productos y mejorando la toma de decisiones.
- **Mejora de la Personalización del Chatbot:** Incorporar modelos de aprendizaje automático para personalizar aún más las respuestas del chatbot según el historial de búsqueda y preferencias del usuario, utilizando técnicas como aprendizaje por refuerzo para optimizar las interacciones y ofrecer recomendaciones más específicas.
- **Mejora del Proceso de Creación de Embeddings:** Sustituir el modelo `stella_en_v5` por un modelo más rápido o gestionado, como un modelo ligero basado en transformers optimizados (por ejemplo, un modelo gestionado en la nube como Vertex AI Embeddings), para reducir significativamente el tiempo de generación de embeddings para los productos, mejorando la eficiencia en la indexación de grandes volúmenes de datos.
- **Optimización del Motor de Búsqueda:** Implementar modelos de embeddings más avanzados o técnicas de re-ranking para mejorar la precisión de las búsquedas semánticas, especialmente en casos de consultas ambiguas o con múltiples intenciones, y explorar la integración de aprendizaje continuo para actualizar los embeddings dinámicamente.
- **Comparación de Productos:** Incorporar consultas del tipo “*Which is better, A or B?*”, permitiendo a los usuarios comparar productos directamente en base a sus características, reseñas y métricas relevantes. Esta funcionalidad mejorará la experiencia de compra al ofrecer una visión clara de las diferencias y ventajas entre productos similares.

- **Interfaz de Usuario Avanzada:** Incorporar características como modo oscuro, soporte multilingüe y personalización de la interfaz en el frontend con React, además de integrar visualizaciones interactivas más avanzadas, como paneles de control personalizables para administradores.

5.3 PROBLEMAS ENCONTRADOS

Esta sección detalla los principales desafíos técnicos enfrentados durante el desarrollo de la aplicación, junto con las soluciones implementadas para superarlos. Estos problemas, relacionados con la integración de servicios externos, el rendimiento de procesos críticos y la configuración de componentes clave, han sido abordados para garantizar la estabilidad y funcionalidad del sistema.

- **Problemas con EdenAI y Migración a OpenRouter:** Inicialmente, se utilizó EdenAI para las consultas del chatbot, lo que funcionaba bien para interacciones básicas. Sin embargo, al implementar Function Calling, EdenAI presentó problemas significativos, ya que no formateaba correctamente las respuestas para adaptarlas a los diferentes proveedores de modelos de lenguaje. Esto generó inconsistencias y errores en las respuestas, lo que llevó a migrar a OpenRouter. Esta transición redujo la dependencia de un proveedor intermedio y mejoró la compatibilidad y fiabilidad del sistema, aunque requirió ajustes en el código para integrar la nueva API.
- **Lentitud en la Generación de Mapas de Calor:** Para la generación de mapas de calor, se realizaba una llamada a un servicio de terceros para localizar direcciones IP, lo que resultaba extremadamente lento y afectaba el rendimiento. Para solucionar este problema, se migró a MaxMind con la base de datos GeoLite, que permitió procesar las geolocalizaciones de manera local y más rápida, mejorando significativamente la eficiencia en la generación de mapas de calor.
- **Dificultades con Embeddings y el Motor de Búsqueda:** La comprensión inicial del funcionamiento de los embeddings y la creación del motor de búsqueda

presentó desafíos, particularmente en la configuración del modelo `stella_en_v5` y la integración con `pgvector`. La generación de embeddings para grandes volúmenes de productos resultó ser un proceso lento, y la optimización de la búsqueda vectorial requirió un aprendizaje profundo sobre la indexación y la gestión de bases de datos vectoriales. Estos obstáculos se superaron mediante pruebas iterativas y ajustes en los parámetros del modelo, aunque el tiempo de procesamiento de embeddings sigue siendo un área de mejora.

5.4 OPINIÓN PERSONAL

Desde mi perspectiva, este proyecto ha sido una experiencia inmensamente enriquecedora. Aunque la aplicación tiene áreas de mejora, como la optimización de los tiempos de generación de embeddings o la incorporación de funcionalidades adicionales como el análisis de sentimientos, creo que refleja de manera efectiva cómo se pueden integrar herramientas de inteligencia artificial en aplicaciones modernas de comercio electrónico. La combinación de un chatbot conversacional, búsqueda semántica y capacidades analíticas avanzadas demuestra el potencial de la IA para transformar la experiencia del usuario y la gestión de plataformas. Personalmente, he aprendido muchísimo durante este proceso, ya que partía sin conocimientos previos en este campo. La realización de este proyecto me ha permitido adquirir un conocimiento profundo sobre IA generativa, embeddings, bases de datos vectoriales y tecnologías modernas como Docker y Kafka, lo que ha sido un logro muy gratificante y una base sólida para futuros desarrollos.

En resumen, el proyecto logró abordar con éxito los tres problemas identificados, entregando una solución innovadora que combina IA generativa, búsqueda semántica y herramientas analíticas avanzadas. A pesar de los desafíos encontrados, como los problemas con EdenAI, la generación de mapas de calor y la complejidad de los embeddings, se implementaron soluciones efectivas que fortalecieron el sistema. Las aportaciones de cada componente, junto con las mejoras propuestas y el aprendizaje personal adquirido, aseguran un claro potencial para elevar futuras versiones de la



plataforma, consolidándose como una herramienta moderna y que refleja las ventajas de aplicar inteligencia artificial en aplicaciones web.

REFERENCIAS

- [1] Amazon (2024) *Product Advertising API 5.0 Documentation*. Available at: <https://webservices.amazon.com/paapi5/documentation/> (Accessed: 15 March 2024).
- [2] Amazon Reviews'23 (2023) *Amazon Reviews'23 Dataset*. Available at: <https://amazon-reviews-2023.github.io/> (Accessed: 17 March 2025).
- [3] Amazon Web Services (2025) *Amazon Kinesis*. Available at: https://aws.amazon.com/pm/kinesis/?trk=a9988395-5d7d-4711-bea3-ca8d96b1able&sc_channel=ps&ef_id=Cj0KCQjwucDBBhDxARIsANqFdr0_96-NLoxv2D3cNVzuMJlwZDCCEK-goe3VyZFOqvhJQg-qsiSj8o8aAuLBEALw_wcB:G:s&s_kwcid=AL!4422!3!651541907446!e!!g!!amazon%20kinesis!19828211262!146491696785&gad_campaignid=19828211262&gbraid=0AAAAADjHtp_0zBFiputS8469XY5YFTCP9&gclid=Cj0KCQjwucDBBhDxARIsANqFdr0_96-NLoxv2D3cNVzuMJlwZDCCEK-goe3VyZFOqvhJQg-qsiSj8o8aAuLBEALw_wcB (Accessed: 23 May 2025).
- [4] Angular (2024) *Angular: The modern web developer's platform*. Available at: <https://angular.dev/> (Accessed: 14 March 2024).
- [5] Anthropic (2024) *Claude: Next-generation AI assistant*. Available at: <https://www.anthropic.com/index/claude> (Accessed: 30 May 2025).
- [6] Artificial intelligence (2025) *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Artificial_intelligence (Accessed: 17 June 2025).
- [7] AWS (2025) *Large Language Model*. Available at: <https://aws.amazon.com/what-is/large-language-model/> (Accessed: 17 March 2025).
- [8] Axios (2025) *Axios Documentation – Intro*. Available at: <https://axios-http.com/docs/intro> (Accessed: 30 May 2025).

- [9] Boletín Oficial del Estado (1996) *Real Decreto Legislativo 1/1996, de 12 de abril, por el que se aprueba el texto refundido de la Ley de Propiedad Intelectual*. Available at: <https://www.boe.es/buscar/act.php?id=BOE-A-1996-8930> (Accessed: 9 May 2025).4
- [10] Boletín Oficial del Estado (2018) *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*. Available at: <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673> (Accessed: 9 May 2025).
- [11] Chart.js (2025) *Chart.js – Open source HTML5 Charts for your website*. Available at: <https://www.chartjs.org/> (Accessed: 30 May 2025).
- [12] Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2018) ‘BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding’, *arXiv*. Available at: <https://arxiv.org/abs/1810.04805> (Accessed: 14 March 2024).
- [13] Días Laborables (2025) *Calendario de días laborables en España*. Available at: <https://www.dias-laborables.es/Castilla%20y%20Le%C3%B3n.htm> (Accessed: 3 April 2025).
- [14] Django Software Foundation (2024) *Django: The web framework for perfectionists with deadlines*. Available at: <https://www.djangoproject.com/> (Accessed: 14 March 2024).
- [15] Eden AI (2024) *Eden AI: AI APIs and solutions*. Available at: <https://www.edenai.co/> (Accessed: 15 March 2024).
- [16] Hugging Face (2024) *Hugging Face: The AI community building the future*. Available at: <https://huggingface.co/> (Accessed: 15 March 2024).
- [17] Hugging Face (2024) *deepseek-ai/DeepSeek-R1*. Available at: <https://huggingface.co/deepseek-ai/DeepSeek-R1> (Accessed: 14 March 2024).
- [18] Hugging Face (2024) *blevlabs/stella_en_v5*. Available at: https://huggingface.co/blevlabs/stella_en_v5 (Accessed: 14 March 2024).

- [19] IBM (2025) *Apache Kafka cluster*. Available at: <https://www.ibm.com/docs/es/oala/1.3.8?topic=components-apache-kafka-cluster> (Accessed: 23 May 2025).
- [20] Indeed (2025) *Salarios y empleos en España*. Available at: <https://es.indeed.com/career/salaries?from=gnav-menu-homepage> (Accessed: 29 April 2025).
- [21] jsPDF (2025) *jsPDF – Generate PDF files in JavaScript*. Available at: <https://github.com/parallax/jsPDF> (Accessed: 30 May 2025).
- [22] Lardinois, F. (2012) ‘In battle with Amazon, Walmart unveils Polaris, a semantic search engine for products’, *TechCrunch*. Available at: <https://techcrunch.com/2012/08/30/in-battle-with-amazon-walmart-unveils-polaris-a-semantic-search-engine-for-products/> (Accessed: 23 March 2025).
- [23] Leaflet (2025) *Leaflet – an open-source JavaScript library for interactive maps*. Available at: <https://leafletjs.com/> (Accessed: 30 May 2025).
- [24] Liu, X. (2022) ‘Demonstration of supply chain management in big data analysis from Walmart, Toyota, and Amazon’, in *International Conference on Management Engineering and Economic Analysis (MEEA)*, Vol. 34, pp. 1198-1203.
- [25] Llama (2024) *Llama*. Available at: <https://www.llama.com/> (Accessed: 14 March 2024).
- [26] Maio, N. and Re, B. (2020) ‘How Amazon's e-commerce works?’, *International Journal of Technology for Business (IJTB)*, 2(1), pp. 8–13.
- [27] MariaDB Foundation (2024) *MariaDB: Open source relational database*. Available at: <https://mariadb.org/> (Accessed: 14 March 2024).
- [28] MaxMind (2025) *GeoLite2 Free Geolocation Data*. Available at: <https://dev.maxmind.com/geoip/geolite2-free-geolocation-data> (Accessed: 30 May 2025).

- [29] MaxMind (2025) *maxminddb: Python Reader for the MaxMind DB Format*. Available at: <https://maxminddb.readthedocs.io/en/latest/> (Accessed: 30 May 2025).
- [30] MDN Web Docs (2024) *Express/Node introduction*. Available at: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction (Accessed: 14 March 2024).
- [31] Microsoft (2025) *Generate embeddings for retrieval-augmented generation (RAG)*. Azure Architecture Center. Available at: <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-generate-embeddings> (Accessed: 14 March 2025).
- [32] Ministerio para la Transformación Digital y de la Función Pública (2025) *Anteproyecto de Ley para el Buen Uso y la Gobernanza de la Inteligencia Artificial*. Available at: https://avance.digital.gob.es/_layouts/15/HttpHandlerParticipacionPublicaAnexos.ashx?k=19128 (Accessed: 9 May 2025).
- [33] MongoDB, Inc. (2024) *MongoDB: The developer data platform*. Available at: <https://www.mongodb.com/> (Accessed: 14 March 2024).
- [34] MongoDB (2025) *What are vector databases?* Available at: <https://www.mongodb.com/resources/basics/databases/vector-databases#mongodb-atlas-vector-search-a-gamechanger> (Accessed: 13 March 2025).
- [35] MySQL (2024) *MySQL: The world's most popular open source database*. Available at: <https://www.mysql.com/> (Accessed: 14 March 2024).
- [36] Nielsen, M. (2015) *Neural Networks and Deep Learning*. Available at: <http://neuralnetworksanddeeplearning.com/> (Accessed: 30 May 2025).
- [37] OpenAI (2023) *GPT-4 Technical Report*. Available at: <https://openai.com/research/gpt-4> (Accessed: 30 May 2025).

- [38] OpenAI (2024) *Embeddings*. OpenAI Documentation. Available at: <https://platform.openai.com/docs/guides/embeddings> (Accessed: 14 March 2024).
- [39] OpenAI (2024) *OpenAI: Creating safe and beneficial AI*. Available at: <https://openai.com/> (Accessed: 15 March 2024).
- [40] OpenAI (2025) *Function calling*. Available at: <https://platform.openai.com/docs/guides/function-calling?api-mode=chat> (Accessed: 17 March 2025).
- [41] OpenRouter (2024) *OpenRouter: AI model router and API*. Available at: <https://openrouter.ai/> (Accessed: 15 March 2024).
- [42] Pallets Projects (2024) *Flask Documentation (1.1.x)*. Available at: <https://flask.palletsprojects.com/en/stable/> (Accessed: 14 March 2024).
- [43] Paradigma Digital (2022) *Comunicación entre microservicios con Apache Kafka*. Available at: <https://www.paradigmadigital.com/dev/comunicacion-microservicios-apache-kafka/> (Accessed: 23 May 2025).
- [44] Pgvector (2024) *pgvector: Open-source vector similarity search for PostgreSQL*. GitHub. Available at: <https://github.com/pgvector/pgvector> (Accessed: 14 March 2024).
- [45] PostgreSQL Global Development Group (2024) *PostgreSQL: The world's most advanced open source relational database*. Available at: <https://www.postgresql.org/> (Accessed: 14 March 2024).
- [46] Python Software Foundation (2024) *Welcome to Python.org*. Available at: <https://www.python.org/> (Accessed: 14 March 2024).
- [47] React (2024) *React: The library for web and native user interfaces*. Available at: <https://react.dev/> (Accessed: 14 March 2024).

- [48] Reactive Programming (2022) *Estilos arquitectónicos - Cliente-Servidor*. Available at: <https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/cliente-servidor> (Accessed: 26 May 2025).
- [49] SBERT (2024) *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Available at: <https://sbert.net/> (Accessed: 14 March 2024).
- [50] Simon Bångs (2025) *jsPDF-AutoTable – Table plugin for jsPDF*. Available at: <https://github.com/simonbengtsson/jsPDF-AutoTable> (Accessed: 30 May 2025).
- [51] Statista (2025) *Generative AI - Worldwide*. Available at: <https://www.statista.com/outlook/tmo/artificial-intelligence/generative-ai/worldwide> (Accessed: 10 March 2025).
- [52] Swagger (2021) *OpenAPI Specification*. Available at: <https://swagger.io/specification/> (Accessed: 3 April 2025).
- [53] Trotman, A., Degenhardt, J. and Kallumadi, S. (2017) ‘The Architecture of eBay Search’, in eCOM@SIGIR. August, p. 88.
- [54] Vaswani, A. et al. (2017) *Attention is All You Need*. Available at: <https://arxiv.org/abs/1706.03762> (Accessed: 30 May 2025).
- [55] Vite (2025) *Next Generation Frontend Tooling*. Available at: <https://vite.dev/> (Accessed: 30 May 2025).
- [56] Vue.js (2024) *Vue.js: The progressive JavaScript framework*. Available at: <https://vuejs.org/> (Accessed: 14 March 2024).
- [57] Walmart Global Tech (2023) *Walmart's generative AI search puts more time back in customers' hands*. Available at: https://tech.walmart.com/content/walmart-global-tech/en_us/blog/post/walmarts-generative-ai-search-puts-more-time-back-in-customers-hands.html (Accessed: 23 March 2025).



[58] Wikipedia (2025) Data set. Available at: https://en.wikipedia.org/wiki/Data_set
(Accessed: 29 June 2025).